ระบบจัดตารางอ่านหนังสืออัตโนมัติ

Smart Study Planner

`

สหรัฐ เวียงแก้ว

โครงงานนี้เป็นส่วนหนึ่งของการศึกษา

หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการข้อมูลและนวัตกรรมซอฟต์แวร์

ภาควิชาคณิตศาสตร์ สถิติ และคอมพิวเตอร์ คณะวิทยาศาสตร์

มหาวิทยาลัยอุบลราชธานี

ปีการศึกษา 2568

ลิขสิทธิ์ของมหาวิทยาลัยอุบลราชธานี

โครงงาน	:	ระบบจัดตารางอัตโนมัติ Smart Study Planner
โดย	:	สหรัฐ เวียงแก้ว
อาจารย์ที่ปรึกษา	:	ผศ.ดร.ไพชยนต์ คงไชย
ระดับการศึกษา	:	วิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการข้อมูลและนวัตกรรมซอฟต์แวร์
ปีการศึกษา	:	2568

ได้รับการพิจารณาให้เป็นส่วนหนึ่งของการศึกษา

ตามหลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการข้อมูลและนวัตกรรมซอฟต์แวร์

คณะกรรมการสอบประเมินความรู้โครงงานของคอมพิวเตอร์

..... อาจารย์ที่ปรึกษา

(ผศ.ดร.ไพชยนต์ คงไชย)

..... กรรมการ

(ชื่อกรรมการ)

..... กรรมการ

(ชื่อกรรมการ)

..... หัวหน้าภาควิชา

(รศ.ชาญชัย ศุภอรรถกร)

วันที่ ...../...../.....

กิตติกรรมประกาศ

การพัฒนาโครงงานระบบจัดตารางอัตโนมัติ สำเร็จลุล่วงได้ด้วยความกรุณาและความช่วยเหลืออย่างดียิ่งจาก ดร.ไพชยนต์ คงไชย อาจารย์ที่ปรึกษาปริญญานิพนธ์ ที่ได้สละเวลาอันมีค่าในการให้คำปรึกษา คำแนะนำ และข้อคิดเห็นที่เป็นประโยชน์ ตลอดจนช่วยตรวจสอบและแก้ไขข้อบกพร่องต่าง ๆ ที่เกิดขึ้นระหว่างการพัฒนาระบบ ทำให้ผลงานดำเนินไปได้ด้วยดีและมีความสมบูรณ์ยิ่งขึ้น ผู้จัดทำขอกราบขอบพระคุณเป็นอย่างสูงไว้ ณ โอกาสนี้

ขอขอบคุณ อาจารย์ประจำสาขาวิทยาการข้อมูลและนวัตกรรมซอฟต์แวร์ คณะวิทยาศาสตร์ มหาวิทยาลัยอุบลราชธานี ทุกท่านที่ได้ประสิทธิ์ประสาทวิชาความรู้ และให้คำแนะนำทางวิชาการอันเป็นพื้นฐานสำคัญในการพัฒนาโครงงานนี้

ขอขอบคุณเพื่อนนักศึกษาและรุ่นพี่ ที่คอยให้ความช่วยเหลือ ให้คำแนะนำ และเป็นกำลังใจในการแก้ไขปัญหาอุปสรรคต่าง ๆ ระหว่างการดำเนินงาน จนทำให้โครงงานนี้สำเร็จลุล่วงไปด้วยดี

สุดท้ายนี้ ขอกราบขอบพระคุณบิดา มารดา และสมาชิกในครอบครัว ที่เป็นผู้ให้การสนับสนุนในทุก ๆ ด้าน ทั้งทางทรัพย์สินและกำลังใจอันสำคัญยิ่ง ตลอดระยะเวลาการศึกษาและการพัฒนาโครงงานครั้งนี้จนสำเร็จลุล่วงด้วยดี

นาย สหรัฐ เวียงแก้ว

วันที่ 16 กุมภาพันธ์ ปี 2569

โครงงาน	:	ระบบจัดตารางอัตโนมัติ Smart Study Planner
โดย	:	สหรัฐ เวียงแก้ว
อาจารย์ที่ปรึกษา	:	ผศ.ดร.ไพชยนต์ คงไชย
ระดับการศึกษา	:	วิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการข้อมูลและนวัตกรรมซอฟต์แวร์
ปีการศึกษา	:	2568

บทคัดย่อ

โครงงานนี้มีวัตถุประสงค์เพื่อพัฒนา ระบบจัดตารางอ่านหนังสืออัจฉริยะ (Smart Study Planner) สำหรับนักเรียนและนักศึกษา ช่วยให้วางแผนการอ่านได้อย่างเป็นระบบและมีประสิทธิภาพ โดยอ้างอิงจากเวลาว่าง ความสำคัญของวิชา และระยะเวลาที่เหลือก่อนสอบ เพื่อแก้ไขปัญหาการจัดการเวลาอ่านหนังสือที่ไม่เหมาะสม ระบบนี้พัฒนาในรูปแบบ เว็บแอปพลิเคชัน โดยใช้ Django Framework สำหรับทั้ง Backend และ Frontend, PostgreSQL สำหรับฐานข้อมูล, และ Supabase สำหรับจัดเก็บข้อมูลออนไลน์ นอกจากนี้ยังนำโมเดลภาษา Gemini มาช่วยในการสร้างคำแนะนำและตอบคำถามเชิงวางแผน เพื่อเพิ่มความสะดวกและยืดหยุ่นในการใช้งาน

ผู้ใช้เพียงกรอกข้อมูลพื้นฐาน เช่น รายวิชา วันสอบ และเวลาว่าง ระบบจะสร้างตารางอ่านหนังสือให้อัตโนมัติ แสดงผลในรูปแบบตารางรายวันหรือรายสัปดาห์

ผลลัพธ์ที่คาดหวังคือ ผู้เรียนจะสามารถจัดการเวลาอ่านหนังสือได้เหมาะสมขึ้น ลดความเครียด และทบทวนเนื้อหาได้ครบถ้วน อีกทั้งระบบยังมีความยืดหยุ่นในการปรับแผนตามการเปลี่ยนแปลงในชีวิตประจำวัน Smart Study Planner จึงเป็นเครื่องมือสำคัญที่สนับสนุนการเรียนรู้ด้วยตนเองในยุคดิจิทัลได้อย่างมีประสิทธิภาพ

คำสำคัญ: การจัดตารางอ่านหนังสือ วางแผนการเรียน เว็บแอปพลิเคชัน การบริหารเวลา Django

Topic	:	Smart Study Planner
Author	:	Saharat Wiangkaew
Advisor	:	Phaichayon Kongchai
Degree	:	Bachelor of Science (Data Science and Software Innovation)
Academic Year	:	2025

Abstract

This project aims to develop a Smart Study Planner system to help students effectively organize their study schedules. It references users' actual free time, subject importance, and remaining time before exams, addressing the common problem of inefficient study time management among learners.

The system is developed as a web application, utilizing Django Framework for both backend and frontend, PostgreSQL for database management, and Supabase for online data storage. Additionally, a large language model (GPT or Gemini) is integrated to provide planning-oriented recommendations and answer questions, enhancing user convenience and flexibility.

Users can input basic information such as subjects to be studied, exam dates, and daily free time, and the system will automatically generate a study schedule, presented in an easy-to-understand format like daily or weekly tables.

The expected outcomes are that students will manage their study time more effectively, reduce pre-exam stress, and be more likely to cover all course material. The system also offers flexibility to adjust plans if daily schedules change. This Smart Study Planner is therefore designed to be a crucial tool supporting self-directed learning in the digital age.

Keywords: Study Scheduling Learning Planning Web Application Time Management Django

สารบัญ

กิตติกรรมประกาศ 3
บทคัดย่อ 4
Abstract 5
สารบัญ 6
สารบัญภาพ 11
สารบัญตาราง 14
บทที่ 1
บทนำ 15
1.1 ที่มาและเหตุผล 15
1.2 วัตถุประสงค์ 15
1.3 ขอบเขตของโครงงาน 16
1.3.1 ผู้ใช้งานระบบ (นักศึกษา) 16
1.3.2 ระบบ (System) 16
1.4 ประโยชน์ที่คาดว่าจะได้รับ 17
1.5 เครื่องมือที่ใช้ในการพัฒนา 17
1.5.1 ฮาร์ดแวร์ 17
1.5.2 ซอฟต์แวร์ (Software) 17
1.5.3 ภาษาโปรแกรมที่ใช้ 18
1.6.4 ฐานข้อมูลที่ใช้ 19
1.6 แผนการดำเนินการ 19
บทที่ 2
ทฤษฎีที่เกี่ยวข้อง 20
2.1 ความรู้เกี่ยวกับ Visual Studio Code 20
2.1.1 ข้อดีของ Visual Studio Code 20
2.2 ความรู้เกี่ยวกับ Django Framework 20
2.2.1 ข้อดีของ Django Framework 21
2.3 ความรู้เกี่ยวกับ PostgreSQL 21
2.4 ความรู้เกี่ยวกับภาษา Python 21
2.5 ความรู้เกี่ยวกับ AI Gemini 21
2.6 ความรู้เกี่ยวกับ GitHub 21
2.6.1 ข้อดีของ GitHub 21
บทที่ 3
การออกแบบระบบ 22
3.1 โครงสร้างภาพรวมของระบบ (System Architecture) 22
3.2 System requirement 23

3.3 การออกแบบหน้าจอ 24
3.3.1 ส่วนของหน้า Landing Page 24
3.3.2 ส่วนของหน้าลงทะเบียน 25
3.3.3 ส่วนของหน้าลงชื่อเข้าสู่ระบบ 26
3.3.4 ส่วนของตารางการอ่าน 27
3.3.5 ส่วนของแผนการอ่าน 28
3.3.6 ส่วนของสถิติภาพรวม 28
3.3.7 ส่วนของการสอบที่ใกล้จะถึง 29
3.3.8 ส่วนของการซิงค์กับ Google Calendar 29
3.3.9 ส่วนของการเพิ่มวิชา 30
3.3.10 ส่วนของการกำหนดเวลาว่าง 31
3.3.11 ส่วนของการตั้งค่าการอ่าน 32
3.3.12 ส่วนของข้อมูลผู้ใช้ 33
3.3.13 ส่วนของหน้าเริ่มอ่าน 34
3.3.14 ส่วนของหน้าหมดเวลาการอ่าน 35
3.3.15 ส่วนของการสรุปเนื้อหา 36
3.3.16 ส่วนของแบบทดสอบ 37
3.3.17 ส่วนของคะแนนการทำบททดสอบ 38
3.3.18 ส่วนของการเฉลยแบบทดสอบ 39
3.3.19 ส่วนของสุมดบันทึกเนื้อหาที่สรุป 40
3.3.20 ส่วนของการแจ้งปัญหา 41
3.4 Use case diagram 42
3.5 Class diagram 45
3.6 Sequence diagram 49
3.6.1 Sequence Diagram ของการสมัครสมาชิก 51
3.6.2 Sequence Diagram ของการเข้าสู่ระบบ 52
3.6.3 Sequence Diagram ของการเพิ่มรายวิชาและไฟล์ 53
3.6.4 Sequence Diagram ของการกำหนดเวลาว่าง 54
3.6.5 Sequence Diagram ของการตั้งค่าการเรียน 55
3.6.6 Sequence Diagram ของการจัดตารางเรียนอัตโนมัติ 56
3.6.7 Sequence Diagram ของการสรุปเนื้อหาด้วย AI 57
3.6.8 Sequence Diagram ของการสร้างแบบทดสอบด้วย AI 58
3.6.9 Sequence Diagram ของการแสดงตารางแผนการอ่าน (homepage) 59
3.6.10 Sequence Diagram ของการแก้ไขข้อมูลส่วนตัว 60
3.6.11 Sequence Diagram ของระบบการแจ้งเตือน 61
3.7 Entity Relationship Diagram 62
บทที่ 4

การพัฒนาระบบ 65
4.1 การพัฒนาระบบการลงทะเบียนและลงชื่อเข้าใช้ 65
4.1.1 การพัฒนาระบบการบันทึกข้อมูลการลงทะเบียน(register_view) 65
4.1.2 การพัฒนาระบบฟอร์มสำหรับผู้ใช้ 65
4.1.3 การพัฒนาระบบการลงทะเบียน(register_view) 66
4.1.4 การพัฒนาระบบการลงชื่อเข้าใช้ 67
4.2 การพัฒนาระบบตารางแผนการอ่านและกิจกรรมในแผนการอ่าน 68
4.2.1 การพัฒนาระบบการบันทึกข้อมูลตารางแผนการอ่าน 68
4.2.2 การพัฒนาระบบการบันทึกข้อมูลตารางกิจกรรมในแผนการอ่าน 69
4.2.3 การพัฒนาระบบหน้าแรก 70
4.3 การพัฒนาระบบเพิ่มวิชา 73
4.3.1 การพัฒนาระบบการบันทึกข้อมูลรายวิชา 73
4.3.2 การพัฒนาระบบการบันทึกข้อมูลไฟล์เอกสาร 74
4.3.3 การพัฒนาระบบฟอร์มสำหรับการเพิ่มไฟล์ 75
4.3.4 การพัฒนาระบบบบเพิ่มวิชา 76
4.3.5 การพัฒนาระบบการลบข้อมูลไฟล์ 78
4.4 การพัฒนาระบบกำหนดเวลาว่าง 79
4.4.1 การพัฒนาการบันทึกข้อมูลเวลาว่าง 79
4.4.2 การพัฒนาระบบกำหนดเวลาว่าง 80
4.5 การพัฒนาระบบการตั้งค่าการอ่าน 82
4.6 การพัฒนาระบบ AI 83
4.6.1 การพัฒนาฟังก์ชันการจัดตารางด้วย AI 83
4.6.2 การพัฒนาฟังก์ชันการสรุปเนื้อหาหลังอ่านด้วย AI 87
4.6.3 การพัฒนาฟังก์ชันการสร้างแบบทดสอบหลังอ่านด้วย AI 88
4.7 การพัฒนาระบบเชื่อมต่อ Google Calendar 90
4.7.1 การพัฒนาระบบเชื่อมต่อ Google Calendar (google_calendar.py) 90
4.7.2 การพัฒนาระบบเชื่อมต่อ Google Calendar (views.py) 92
4.8 การพัฒนาระบบแจ้งเตือน 93
4.8.1 การพัฒนาระบบแจ้งเตือน (context_processors.py) 93
4.8.2 การพัฒนาระบบการบันทึกข้อมูลการแจ้งเตือน 94
4.8.3 การพัฒนาระบบการจัดการสถานะการแจ้งเตือน 95
4.9 การพัฒนาระบบสรุปเนื้อหา 96
4.9.1 การพัฒนาระบบการบันทึกข้อมูลสรุปเนื้อหา 96
4.9.2 การพัฒนาระบบ API สำหรับเรียก AI สรุปเนื้อหา 97
4.9.3 การพัฒนาระบบหน้าแสดงสรุปเนื้อหา 98
4.9.4 การพัฒนาระบบหน้าประวัติการสรุปเนื้อหา 99
4.10 การพัฒนาระบบสร้างแบบทดสอบ 100

4.10.1 การพัฒนาระบบการบันทึกข้อมูลแบบทดสอบ 100
4.10.2 การพัฒนาระบบการบันทึกข้อมูลผลการทดสอบ 101
4.10.3 การพัฒนาระบบ API สำหรับเรียก AI สร้างแบบทดสอบ 102
4.10.4 การพัฒนาระบบการบันทึกผลการสอบ 103
4.10.5 การพัฒนาระบบหน้าแสดงผลคะแนนสอบ 104
4.10.6 การพัฒนาระบบหน้าแสดงเฉลยแบบทดสอบ 105
4.10 การพัฒนาระบบยันตัวตนด้วยอีเมล (backends.py) 106
บทที่ 5
การทดสอบระบบ 107
5.1 การทดสอบฟังก์ชันสมัครสมาชิกและลงชื่อเข้าสู่ระบบ 108
5.1.1 การทดสอบฟังก์ชันลงทะเบียน 108
5.1.2 การทดสอบฟังก์ชันลงชื่อเข้าสู่ระบบ 108
5.2 การทดสอบฟังก์ชันการเพิ่มรายวิชาและไฟล์การเรียนการสอบ 109
5.3 การทดสอบฟังก์ชันการกำหนดเวลาว่าง 109
5.4 การทดสอบฟังก์ชันตั้งค่าการอ่าน 110
5.5 การทดสอบฟังก์ชันการจัดตารางอัตโนมัติด้วย AI 110
5.6 การทดสอบตารางแผนการอ่าน, แผนการอ่านวันนี้, สถิติภาพรวม, การสอบที่ใกล้จะถึง 111
5.7 การทดสอบฟังก์ชันการจับเวลาในการอ่าน 111
5.8 การทดสอบฟังก์ชันการสรุปเนื้อหาและสร้างแบบทดสอบ 112
5.9 การทดสอบฟังก์ชันข้อเสนอแนะและแจ้งปัญหา 112
บทที่ 6
สรุปและข้อเสนอแนะ 113
6.1 สรุปความสามารถของระบบ 113
6.2 ปัญหาและอุปสรรคในการพัฒนา 114
6.3 แนวทางในการพัฒนาต่อ 114
บรรณานุกรม 115
ภาคผนวก ก
การติดตั้งเครื่องมือที่ใช้พัฒนาโปรแกรม 117
ก.1 ขั้นตอนการติดตั้งโปรแกรม Visual Studio Code หรือ VS Code 117
ก.2 ขั้นตอนการติดตั้ง Python 120
ก.3 ขั้นตอนการติดตั้ง Git 122
ก.4 ขั้นตอนการติดตั้ง Django และ Library ที่เกี่ยวข้อง 124
ก.5 ขั้นตอนการติดตั้ง PostgreSQL 125
ภาคผนวก ข
คู่มือการติดตั้งระบบ 128
ข.1 ส่วนของการดาวน์โหลดซอร์สโค้ด 128
ข.2 ส่วนของการสร้างและเปิดใช้งาน Virtual Environment 129
ข.3 ส่วนของการติดตั้งไลบรารีที่จำเป็น 129

ข.4 การตั้งค่า Environment Variables 130
ข.5 การตั้งค่าฐานข้อมูล 130
ข.6 การรันเซิร์ฟเวอร์ 131
ภาคผนวก ค
คู่มือการใช้งานของระบบ 132
ค.1.1 ส่วนของหน้าแรก 132
ค.1.2 ส่วนของหน้าสมัครสมาชิก 133
ค.1.3 ส่วนของหน้าเข้าสู่ระบบ 134
ค.1.4 ส่วนของหน้า Home page 135
ค.1.5 ส่วนของหน้าเพิ่มวิชา 136
ค.1.6 ส่วนของหน้ากำหนดเวลาว่าง 137
ค.1.7 ส่วนของหน้าตั้งค่าการเรียน 138
ค.1.8 ส่วนของตารางแผนการอ่าน 139
ค.1.9 ส่วนของแผนการอ่านวันนี้ 140
ค.1.10 ส่วนของสถิติภาพรวม 141
ค.1.11 ส่วนของการสอบที่ใกล้จะถึง 142
ค.1.12 ส่วนของการซิงค์กับ Google Calendar 142
ค.1.13 ส่วนของการเริ่มอ่าน 143
ค.1.14 ส่วนของการสรุปเนื้อหาและการทำแบบทดสอบ 144
ค.1.15 ส่วนของสมุดบันทึกสรุป 145
ค.1.16 ส่วนของการแก้ไขโปรไฟล์ 146
ประวัติผู้พัฒนา 147

สารบัญภาพ

ภาพประกอบที่ 3.1 สถาปัตยกรรมของระบบจัดตารางอัตโนมัติ 22
ภาพประกอบที่ 3.2 หน้า Landing Page 24
ภาพประกอบที่ 3.3 หน้าลงทะเบียน 25
ภาพประกอบที่ 3.4 หน้าลงชื่อเข้าสู่ระบบ 26
ภาพประกอบที่ 3.5 ตารางการอ่าน 27
ภาพประกอบที่ 3.6 แผนการอ่าน 28
ภาพประกอบที่ 3.7 สถิติภาพรวม 28
ภาพประกอบที่ 3.8 สถิติภาพรวม 29
ภาพประกอบที่ 3.9 ซิงค์กับ Google Calendar 29
ภาพประกอบที่ 3.10 30
ภาพประกอบที่ 3.11 31
ภาพประกอบที่ 3.12 32
ภาพประกอบที่ 3.13 33
ภาพประกอบที่ 3.14 34
ภาพประกอบที่ 3.15 35
ภาพประกอบที่ 3.16 36
ภาพประกอบที่ 3.17 37
ภาพประกอบที่ 3.18 38
ภาพประกอบที่ 3.19 39
ภาพประกอบที่ 3.20 40
ภาพประกอบที่ 3.21 41
ภาพประกอบที่ 3.22 Usecase Diagram ของระบบจัดตารางอัตโนมัติ Smart Study Planner 43
ภาพประกอบที่ 3.23 Class Diagram ของของระบบจัดตารางอัตโนมัติz Smart Study Planner 46
ภาพประกอบที่ 3.24 Sequence Diagram ของการสมัครสมาชิก 51
ภาพประกอบที่ 3.25 Sequence Diagram ของการเข้าสู่ระบบ 52
ภาพประกอบที่ 3.27 Sequence Diagram ของการกำหนดเวลาว่าง 54
ภาพประกอบที่ 3.28 Sequence Diagram ของการตั้งค่าการเรียน 55
ภาพประกอบที่ 3.29 Sequence Diagram ของการจัดตารางเรียนอัตโนมัติ 56
ภาพประกอบที่ 3.30 Sequence Diagram ของการสรุปเนื้อหาด้วย AI 57
ภาพประกอบที่ 3.31 Sequence Diagram ของการสร้างแบบทดสอบด้วย AI 58
ภาพประกอบที่ 3.32 Sequence Diagram ของการแสดงตารางแผนการอ่าน (homepage) 59
ภาพประกอบที่ 3.33 Sequence Diagram ของการแก้ไขข้อมูลส่วนตัว 60
ภาพประกอบที่ 3.34 Sequence Diagram ของระบบการแจ้งเตือน 61
ภาพประกอบที่ 4.1 การพัฒนาระบบการบันทึกข้อมูลการลงทะเบียน (models.py) 65
ภาพประกอบที่ 4.2 การพัฒนาระบบฟอร์มสำหรับผู้ใช้ (forms.py) 65

ภาพประกอบที่ 4.3 การพัฒนาระบบลงทะเบียน (views.py) 66
ภาพประกอบที่ 4.4 การพัฒนาระบบการลงชื่อเข้าใช้ (views.py) 67
ภาพประกอบที่ 4.5 การพัฒนาระบบการบันทึกข้อมูลตารางแผนการอ่าน (models.py) 68
ภาพประกอบที่ 4.6 การพัฒนาระบบการบันทึกข้อมูลตารางกิจกรรมในแผนการอ่าน (models.py) 69
ภาพประกอบที่ 4.7 การพัฒนาระบบหน้าแรก (homepage_view) 72
ภาพประกอบที่ 4.8 การพัฒนาระบบการบันทึกข้อมูลรายวิชา (models.py) 73
ภาพประกอบที่ 4.9 การพัฒนาระบบการพัฒนาระบบการบันทึกข้อมูลไฟล์เอกสาร (models.py) 74
ภาพประกอบที่ 4.10 การพัฒนาระบบฟอร์มสำหรับการเพิ่มไฟล์ (forms.py) 75
ภาพประกอบที่ 4.11 การพัฒนาระบบเพิ่มวิชา (views.py) 77
ภาพประกอบที่ 4.12 การพัฒนาระบบการลบข้อมูลไฟล์ (views.py) 78
ภาพประกอบที่ 4.13 การพัฒนาระบบการบันทึกข้อมูลเวลาว่าง (models.py) 79
ภาพประกอบที่ 4.14 การพัฒนาระบบกำหนดเวลาว่าง (views.py) 81
ภาพประกอบที่ 4.15 การพัฒนาระบบการตั้งค่าการอ่าน (views.py) 82
ภาพประกอบที่ 4.16 การพัฒนาระบบ 86
ภาพประกอบที่ 4.17 การพัฒนาฟังก์ชันการสรุปเนื้อหาหลังอ่านด้วย AI (ai_service.py) 87
ภาพประกอบที่ 4.18 การพัฒนาระบบ 89
ภาพประกอบที่ 4.19 การพัฒนาระบบ 91
ภาพประกอบที่ 4.20 การพัฒนาระบบเชื่อมต่อ Google Calendar (views.py) 92
ภาพประกอบที่ 4.21 การพัฒนาระบบการตั้งค่าการแจ้งเตือน 93
ภาพประกอบที่ 4.22 การพัฒนาระบบการบันทึกข้อมูลการแจ้งเตือน (models.py) 94
ภาพประกอบที่ 4.23 การพัฒนาระบบการจัดการสถานะการแจ้งเตือน (views.py) 95
ภาพประกอบที่ 4.24 การพัฒนาระบบการบันทึกข้อมูลสรุปเนื้อหา (models.py) 96
ภาพประกอบที่ 4.25 การพัฒนาระบบ API สำหรับเรียก AI สรุปเนื้อหา (views.py) 97
ภาพประกอบที่ 4.26 การพัฒนาระบบหน้าแสดงสรุปเนื้อหา (views.py) 98
ภาพประกอบที่ 4.27 การพัฒนาระบบหน้าประวัติการสรุปเนื้อหา (views.py) 99
ภาพประกอบที่ 4.28 การพัฒนาระบบการบันทึกข้อมูลแบบทดสอบ (models.py) 100
ภาพประกอบที่ 4.29 การพัฒนาระบบการบันทึกข้อมูลผลการทดสอบ (models.py) 101
ภาพประกอบที่ 4.30 การพัฒนาระบบ API สำหรับเรียก AI สร้างแบบทดสอบ (views.py) 102
ภาพประกอบที่ 4.31 การพัฒนาระบบการบันทึกผลการสอบ (views.py) 103
ภาพประกอบที่ 4.32 การพัฒนาระบบหน้าแสดงผลคะแนนสอบ (views.py) 104
ภาพประกอบที่ 4.33 การพัฒนาระบบหน้าแสดงเฉลยแบบทดสอบ (views.py) 105
ภาพประกอบที่ 4.34 การพัฒนาระบบ 106
ภาพที่ ก.1 หน้าเว็บดาวน์โหลด Visual Studio Code 117
ภาพที่ ก.2 หน้าต่าง License Agreement 118
ภาพที่ ก.3 หน้าต่าง Select Additional Tasks 118
ภาพที่ ก.4 หน้าต่างReady To Install 119
ภาพที่ ก.5 หน้าต่างการติดตั้งเสร็จสิ้น 119

ภาพที่ ก.6 หน้าเว็บดาวน์โหลด Python 120
ภาพที่ ก.7 ภาพตัวเลือกที่ต้องการดาวน์โหลด 120
ภาพที่ ก.8 ภาพเมื่อทำการติดตั้ง Python 121
ภาพที่ ก.9 ภาพ ติดตั้งpython สำเร็จ 121
ภาพที่ ก.10 ตรวจสอบการติดตั้ง 122
ภาพที่ ก.11 หน้าเว็บดาวน์โหลด Git 122
ภาพที่ ก.12 ตัวเลือกการดาวน์โหลด 123
ภาพที่ ก.13 ภาพการติดตั้ง 123
ภาพที่ ก.14 ตรวจสอบการติดตั้ง 124
ภาพที่ ก.14 การติดตั้ง Django 124
ภาพที่ ก.14 การติดตั้ง Library 125
ภาพที่ ก.16 หน้าเว็บดาวน์โหลด PostgreSQL 125
ภาพที่ ก.17 ตัวเลือกการดาวน์โหลด 126
ภาพที่ ก.18 ตัวเลือกเวอร์ชั่นดาวน์โหลด 126
ภาพที่ ก.19 ภาพการติดตั้ง 127
ภาพที่ ข.1 หน้าเว็บ Repository ของโปรเจ็ค 128
ภาพที่ ค.1.1 ภาพหน้า 132
ภาพที่ ค.1.2 ภาพหน้าสมัครสมาชิก 133
ภาพที่ ค.1.3 ภาพหน้าเข้าสู่ระบบ 134
ภาพที่ ค.1.4 ภาพหน้า Home page 135
ภาพที่ ค.1.5 ภาพหน้าเพิ่มวิชา 136
ภาพที่ ค.1.6 ภาพหน้ากำหนดเวลาว่าง 137
ภาพที่ ค.1.7 ภาพหน้าตั้งค่าการเรียน 138
ภาพที่ ค.1.8 ภาพหน้าตารางแผนการอ่าน 139
ภาพที่ ค.1.9 ภาพหน้าแผนการอ่านวันนี้ 140
ภาพที่ ค.1.10 ภาพหน้าแผนการอ่านวันนี้ 141
ภาพที่ ค.1.11 ภาพหน้าการสอบที่ใกล้จะถึง 142
ภาพที่ ค.1.12 ภาพหน้าการซิงค์กับ Google Calendar 142
ภาพที่ ค.1.13 ภาพหน้าเริ่มอ่าน 143
ภาพที่ ค.1.14 ภาพการสรุปเนื้อหา 144
ภาพที่ ค.1.15 ภาพสมุดบันทึกสรุป 145
ภาพที่ ค.1.16 ภาพการแก้ไขโปรไฟล์ 146

สารบัญตาราง

ตารางที่ 1.1 ขั้นตอนการดำเนินงาน 19
ตารางที่ 3.1 สัญลักษณ์ของ Use case Diagram 42
ตารางที่ 3.2 อธิบาย Use Case หน้าที่ของระบบ 44
ตารางที่ 3.3 สัญลักษณ์ของ Class Diagram 45
ตารางที่ 3.4 อธิบาย Class Diagram ของคลาสระบบ 47
ตารางที่ 3.5 สัญลักษณ์ของ Sequence Diagram 49
ตารางที่ 3.6 สัญลักษณ์ของ Entity Relationship Diagram 62
ตารางที่ 3.7 รายละเอียด Entity ที่เกี่ยวข้องกับระบบ 64
ตารางที่ 5.1 ผลการทดสอบการลงทะเบียน 108
ตารางที่ 5.2 ผลการทดสอบการลงชื่อเข้าสู่ระบบ 108
ตารางที่ 5.3 ผลการทดสอบการเพิ่มรายวิชาและไฟล์การเรียนการสอบ 109
ตารางที่ 5.4 ผลการทดสอบการกำหนดเวลาว่าง 109
ตารางที่ 5.5 ผลการทดสอบการตั้งค่าการอ่าน 110
ตารางที่ 5.6 ผลการทดสอบการจัดตารางอัตโนมัติด้วย AI 110
ตารางที่ 5.7 ผลการทดสอบแผนการอ่าน, แผนการอ่านวันนี้, สถิติภาพรวม, การสอบที่ใกล้จะถึง 111
ตารางที่ 5.8 ผลการทดสอบการการจับเวลาในการอ่าน 111
ตารางที่ 5.9 ผลการทดสอบการการสรุปเนื้อหาและสร้างแบบทดสอบ 112
ตารางที่ 5.15 ผลการทดสอบข้อเสนอแนะและแจ้งปัญหา 112
ตารางที่ ข.1 คำสั่งติดตั้ง โปรเจค 128
ตารางที่ ข.2 คำสั่งเข้าสู่โฟลเดอร์โปรเจกต์ 129
ตารางที่ ข.3 คำสั่งสร้าง Virtual Environment 129
ตารางที่ ข.4 คำสั่งเปิดใช้งาน Virtual Environment 129
ตารางที่ ข.5 คำสั่งติดตั้งไลบรารี 129
ตารางที่ ข.6 คำสั่งตั้งค่า Environment Variables 130
ตารางที่ ข.7 คำสั่งสร้างไฟล์ Migration 130
ตารางที่ ข.8 คำสั่งอัปเดตฐานข้อมูล 130
ตารางที่ ข.9 คำสั่งรันเซิร์ฟเวอร์ 131

บทที่ 1
บทนำ

1.1 ที่มาและเหตุผล

นักศึกษาจำนวนมากในปัจจุบันประสบปัญหาในการวางแผนการอ่านหนังสืออย่างมีประสิทธิภาพ โดยเฉพาะในช่วงสอบที่มีเนื้อหาจำนวนมากและเวลาจำกัด การอ่านแบบไม่มีแผน หรือการวางแผนด้วยตนเองโดยไม่เข้าใจลำดับความสำคัญของเนื้อหา อาจนำไปสู่การเตรียมตัวสอบที่ไม่พร้อม ทำให้ผลการเรียนอาจไม่เป็นไปตามที่คาดหวัง และเกิดความเครียดจากการจัดการเวลาที่ไม่ดี

ระบบการจัดตารางอ่านหนังสือส่วนใหญ่ในปัจจุบัน เช่น การใช้กระดาษ ปฏิทิน หรือแอปพลิเคชันปฏิทินทั่วไป ยังคงเป็นแบบพื้นฐาน ไม่สามารถปรับเปลี่ยนตามพฤติกรรมผู้ใช้ หรือแนะนำตามความยากง่ายของวิชาได้ ทำให้ผู้ใช้ต้องใช้เวลาและแรงในการวางแผนด้วยตนเอง ซึ่งอาจไม่เหมาะสมกับแต่ละบุคคล

ด้วยเหตุนี้ เราจึงมีแนวคิดที่จะพัฒนาระบบ Smart Study Planner ที่สามารถจัดตารางอ่านหนังสือให้เหมาะสมกับเป้าหมายของผู้ใช้ โดยอิงจากข้อมูลต่าง ๆ เช่น ระยะเวลาในการเตรียมตัว วิชาที่ต้องสอบ ความยากของเนื้อหา และพฤติกรรมการอ่านของผู้ใช้ ระบบนี้จะช่วยให้ผู้ใช้สามารถเตรียมตัวสอบได้อย่างมีประสิทธิภาพและตรงจุดมากยิ่งขึ้น ลดความเครียดและเพิ่มโอกาสในการประสบความสำเร็จในการเรียน

1.2 วัตถุประสงค์

1.2.1 เพื่อพัฒนาและออกแบบระบบ Smart Study Planner ที่สามารถจัดตารางอ่านหนังสือส่วนบุคคลได้อย่างมีประสิทธิภาพ โดยพิจารณาจากข้อมูล เช่น ระยะเวลาในการเตรียมตัว วิชาที่ต้องสอบ ความยากของเนื้อหา และพฤติกรรมการอ่านของผู้ใช้

1.2.2 เพื่อสร้างกลไกการปรับตารางอ่านหนังสืออัตโนมัติ ที่สามารถปรับเปลี่ยนแผนการอ่านได้ตามความคืบหน้า หรือเมื่อมีการเปลี่ยนแปลงปัจจัยภายนอก เช่น วันสอบเลื่อน เพื่อให้ผู้ใช้สามารถปรับตัวได้ทันท่วงที

1.2.3 เพื่อประเมินประสิทธิภาพและความพึงพอใจของผู้ใช้งาน ต่อระบบ Smart Study Planner ในด้านความสะดวกในการใช้งาน การช่วยเพิ่มประสิทธิภาพในการจัดตารางอ่านหนังสือ และลดความกังวลในการเตรียมตัวสอบ

1.3 ขอบเขตของโครงงาน

1.3.1 ผู้ใช้งานระบบ (นักศึกษา)

1.3.1.1 สมัครสมาชิกและเข้าสู่ระบบเพื่อใช้งานระบบ
1.3.1.2 จัดการข้อมูลส่วนตัว (ชื่อ, ระดับการศึกษา, สาขา ฯลฯ)
1.3.1.3 จัดการรายวิชาที่ต้องสอบ (เพิ่ม, ลบ, แก้ไขข้อมูลวิชา)
1.3.1.4 ระบุวันสอบและช่วงเวลาว่างในแต่ละวัน
1.3.1.5 เพิ่มไฟล์การเรียนการสอนหรือเอกสารประกอบการอ่าน
1.3.1.6 สร้างตารางอ่านหนังสืออัตโนมัติจากระบบ
1.3.1.7 ดูตารางอ่านหนังสือที่ระบบสร้างให้อัตโนมัติ
1.3.1.8 ปรับแก้ไขตารางอ่านหนังสือที่ระบบคำนวณมาให้
1.3.1.9 ติ๊กเครื่องหมาย “อ่านแล้ว” เพื่อบันทึกความคืบหน้าของแต่ละวิชา
1.3.1.10 ดูความคืบหน้าการอ่านของแต่ละวิชา
1.3.1.11 ดูสรุปเนื้อหาหลังจากทำเครื่องหมายว่าอ่านแล้ว
1.3.1.12 ทำแบบทดสอบหลังเรียนเพื่อตรวจสอบความเข้าใจ
1.3.1.13 รับการแจ้งเตือนกำหนดการอ่านหนังสือ
1.3.1.14 แจ้งปัญหาหรือให้คำแนะนำเพิ่มเติมเกี่ยวกับระบบ

1.3.2 ระบบ (System)

1.3.2.1 ประมวลผลข้อมูลวิชาและเวลาว่างที่ผู้ใช้กรอก
1.3.2.2 คำนวณและสร้างตารางอ่านหนังสืออัตโนมัติ
1.3.2.3 ปรับแผนการอ่านใหม่อัตโนมัติเมื่อมีการเปลี่ยนแปลงข้อมูลหรือเวลาว่าง
1.3.2.4 คำนวณเปอร์เซ็นต์ความคืบหน้าการอ่านของแต่ละวิชา
1.3.2.5 สร้างสรุปเนื้อหาหลังจากผู้ใช้ทำเครื่องหมายว่าอ่านแล้ว
1.3.2.6 สร้างแบบทดสอบหลังเรียน
1.3.2.7 แจ้งเตือนความคืบหน้าของผู้ใช้ในแต่ละวิชา
1.3.2.8 แจ้งเตือนเมื่อใกล้ถึงวันสอบ

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1.4.1 มีระบบช่วยวางแผนการอ่านหนังสืออย่างมีระบบ
ระบบสามารถวิเคราะห์ข้อมูลเวลาว่างและวิชาที่ต้องสอบ เพื่อสร้างตารางอ่านหนังสือที่เหมาะสม ลดปัญหาการอ่านไม่ทันหรืออ่านซ้ำโดยไม่จำเป็น

1.4.2 มีระบบช่วยเพิ่มประสิทธิภาพในการเตรียมสอบ
ระบบสามารถจัดลำดับความสำคัญของเนื้อหาและแนะนำตารางอ่านหนังสือเฉพาะบุคคล ทำให้การเตรียมสอบมีความชัดเจนและมีประสิทธิภาพมากขึ้น

1.4.3 มีระบบช่วยลดภาระในการวางแผนด้วยตนเอง
ผู้ใช้งานไม่จำเป็นต้องจัดตารางเองด้วยกระดาษหรือแอปทั่วไป เพราะระบบสามารถประมวลผลและสร้างตารางให้อัตโนมัติ

1.4.4 มีระบบส่งเสริมการบริหารเวลาอย่างมีประสิทธิภาพ
ระบบช่วยให้ผู้ใช้ตระหนักถึงการจัดสรรเวลาอ่านหนังสืออย่างเหมาะสม และสามารถบาลานซ์เวลาในชีวิตประจำวันได้ดีขึ้น

1.4.5 มีระบบที่สามารถนำไปพัฒนาและต่อยอดได้ในอนาคต
ระบบมีโครงสร้างที่รองรับการเพิ่มฟีเจอร์ใหม่ เช่น การเชื่อมต่อกับ Google Calendar, การสร้างแบบทดสอบหลังการอ่าน, การใช้ AI วิเคราะห์พฤติกรรมการอ่าน เพื่อเพิ่มความชาญฉลาดของระบบ

1.5 เครื่องมือที่ใช้ในการพัฒนา

1.5.1 ฮาร์ดแวร์

1.5.1.1 คอมพิวเตอร์ส่วนบุคคล
ใช้สำหรับเขียนโปรแกรม พัฒนาเว็บแอปพลิเคชัน และทดสอบระบบ ทั้งในส่วน Frontend และ Backend รวมถึงการจัดการฐานข้อมูล

1.5.1.2 . อุปกรณ์เชื่อมต่ออินเทอร์เน็ต
ใช้สำหรับเชื่อมต่อระบบกับอินเทอร์เน็ตเพื่อทดสอบการใช้งานในสภาพแวดล้อมจริง และทดสอบการเข้าถึงระบบจากผู้ใช้ปลายทาง

1.5.2 ซอฟต์แวร์ (Software)

1.5.2.1 Visual Studio Code
ใช้เป็นเครื่องมือหลักในการพัฒนาเว็บแอปพลิเคชัน Smart Study Planner ทั้งในส่วน Frontend และ Backend

1.5.2.2 Python Interpreter
ใช้สำหรับรันภาษา Python และรองรับการทำงานของเฟรมเวิร์ก Django ซึ่งเป็นเทคโนโลยีหลักของระบบ

1.5.2.3 Django Framework
ใช้สำหรับพัฒนาเว็บแอปพลิเคชันแบบ Full-Stack รองรับการจัดการฐานข้อมูล การจัดการผู้ใช้ และการสร้าง API สำหรับระบบแนะนำตารางอ่านหนังสืออัจฉริยะ

1.5.2.4 PostgreSQL
ใช้เป็นฐานข้อมูล สำหรับจัดเก็บข้อมูลผู้ใช้ ตารางอ่านหนังสือ ข้อมูลวิชา วันสอบ และสถิติการอ่าน

1.5.2.5 Gemini (Google AI Platform)
ใช้เป็นระบบปัญญาประดิษฐ์สำหรับให้คำแนะนำเกี่ยวกับการจัดตารางอ่านหนังสือ และแสดงคำแนะนำเมื่อตารางเวลาของผู้ใช้อาจไม่เพียงพอ

1.5.2.6 Git และ GitHub
ใช้สำหรับควบคุมเวอร์ชันของซอร์สโค้ด (Version Control) และเป็นพื้นที่จัดเก็บโค้ดกลางสำหรับการพัฒนาร่วมกัน

1.5.2.7 Google Chrome เวอร์ชันล่าสุด
ใช้สำหรับทดสอบการทำงานของเว็บแอปพลิเคชัน และตรวจสอบการแสดงผลของระบบจากมุมมองของผู้ใช้จริง

1.5.3 ภาษาโปรแกรมที่ใช้

1.5.3.1 Python เวอร์ชัน 3.12

1.5.3.2 HTML5

1.5.3.3 CSS3

1.5.3.4 JavaScript

1.5.3.5 SQL

1.6.4 ฐานข้อมูลที่ใช้

1.6.4.1 PostgreSQL

1.6 แผนการดำเนินการ

โครงงาน Smart Study Planner – ระบบจัดตารางอ่านหนังสืออัตโนมัติ มีแผนการดำเนินงานโดยแบ่งออกเป็น 8 ขั้นตอน ดังนี้

ตารางที่ 1.1 ขั้นตอนการดำเนินงาน

	2568			2569				
แผนการดำเนินการ	ก.ค.	ส.ค.	ก.ย.	ต.ค.	พ.ย.	ธ.ค.	ม.ค.	ก.พ.
1. ศึกษาความเป็นไปได้	■	■						
2. นำเสนอหัวข้อ		■	■					
3. ศึกษาค้นคว้าข้อมูล			■	■	■			
4. ศึกษาการใช้เครื่องมือ				■	■	■		
5. วิเคราะห์และออกแบบ				■	■	■	■	
6. เขียนโปรแกรม				■	■	■	■	■
7. ทดสอบและแก้ปัญหา						■	■	■
8. จัดทำเอกสาร							■	■

บทที่ 2
ทฤษฎีที่เกี่ยวข้อง

บทนี้กล่าวถึงหลักการ ทฤษฎี และงานวิจัยที่เกี่ยวข้องกับการพัฒนาระบบจัดตารางอ่านหนังสืออัจฉริยะ (Smart Study Planner) โดยแบ่งออกเป็น 2 ส่วน คือ

ส่วนที่ 1: เนื้อหาพื้นฐานเกี่ยวกับทฤษฎีและเทคโนโลยีที่ใช้ในการพัฒนาระบบ ได้แก่ ความรู้พื้นฐานเกี่ยวกับ Visual Studio Code, Django Framework, PostgreSQL, Supabase, Docker, OAuth, Bootstrap, Python, AI (Gemini), และ GitHub

ส่วนที่ 2: เอกสารและงานวิจัยที่เกี่ยวข้องกับระบบบริหารจัดการและการพัฒนาระบบที่คล้ายคลึง

2.1 ความรู้เกี่ยวกับ Visual Studio Code

Visual Studio Code (VS Code) คือโปรแกรมแก้ไขโค้ด (Code Editor) แบบโอเพนซอร์สจากบริษัทไมโครซอฟต์ที่ได้รับความนิยมสูง ใช้งานได้ฟรีและรองรับหลายระบบปฏิบัติการ เช่น Windows, macOS, และ Linux รองรับภาษาการเขียนโปรแกรมหลายภาษา เช่น JavaScript, Python, PHP และอื่น ๆ นอกจากนี้ยังสามารถติดตั้งส่วนขยาย (Extensions) เพื่อเพิ่มความสามารถต่าง ๆ เช่น การเชื่อมต่อกับระบบควบคุมเวอร์ชัน Git, การตรวจสอบโค้ด (Linting), และ Debugger ทำให้การพัฒนาโปรแกรมรวดเร็วและมีประสิทธิภาพ

2.1.1 ข้อดีของ Visual Studio Code

2.1.1.1 รองรับการใช้งานหลายภาษาและแพลตฟอร์ม

2.1.1.2 มีระบบจัดการโค้ดอัตโนมัติ เช่น Format on Paste, Inline Variable Value

2.1.1.3 มี Integrated Terminal ช่วยให้ทำงานโดยไม่ต้องสลับหน้าต่าง

2.1.1.4 รองรับการรันคำสั่งหลายคำสั่งใน Task เดียว

2.2 ความรู้เกี่ยวกับ Django Framework

Django เป็นเฟรมเวิร์กสำหรับพัฒนาเว็บแอปพลิเคชันฝั่ง Backend ด้วยภาษา Python โดยมีเครื่องมือและโครงสร้างครบถ้วนสำหรับเชื่อมต่อฐานข้อมูล จัดการระบบผู้ใช้ และแสดงผลข้อมูลไปยังหน้าเว็บ Django มีจุดเด่นเรื่องการพัฒนาอย่างรวดเร็ว และรองรับการขยายระบบขนาดใหญ่ มีผู้ใช้งานในองค์กรชั้นนำเช่น Instagram และ Disqus

2.2.1 ข้อดีของ Django Framework

2.2.1.1 เอกสารประกอบและชุมชนผู้ใช้งานขนาดใหญ่
2.2.1.2 มีเครื่องมือครบถ้วนสำหรับพัฒนาเว็บแบบ Full-stack
2.2.1.3 เหมาะสำหรับการพัฒนาระบบที่ต้องการความเสถียรและความปลอดภัยสูง
2.2.1.4 สามารถใช้งานร่วมกับไลบรารีอื่นๆ เช่น Image Processing ได้ง่าย

2.3 ความรู้เกี่ยวกับ PostgreSQL

PostgreSQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์แบบโอเพนซอร์สที่ได้รับความนิยมสูง มีความเสถียรและสามารถปรับขนาดได้ดี รองรับฟีเจอร์ขั้นสูง เช่น การทำงานกับข้อมูลเชิงภูมิศาสตร์ (GIS), JSON และฟังก์ชันที่ซับซ้อน PostgreSQL เป็นฐานข้อมูลหลักที่ใช้สำหรับเก็บข้อมูลผู้ใช้ ตารางอ่านหนังสือ และข้อมูลอื่น ๆ ในระบบ

2.4 ความรู้เกี่ยวกับภาษา Python

Python เป็นภาษาการเขียนโปรแกรมระดับสูงที่อ่านง่ายและเขียนได้รวดเร็ว มีไลบรารีจำนวนมากรองรับงานหลากหลาย เช่น การวิเคราะห์ข้อมูล วิทยาศาสตร์ข้อมูล และพัฒนาเว็บ โดย Django Framework ก็เป็นหนึ่งในไลบรารีสำคัญที่พัฒนาด้วย Python

2.5 ความรู้เกี่ยวกับ AI Gemini

Gemini เป็นระบบปัญญาประดิษฐ์จาก Google ที่ใช้สำหรับวิเคราะห์และแนะนำตารางอ่านหนังสืออย่างอัจฉริยะ โดยใช้เทคนิคการประมวลผลภาษาธรรมชาติ (NLP) เพื่อเข้าใจพฤติกรรมผู้ใช้และสร้างแผนการเรียนที่เหมาะสมตามเป้าหมาย

2.6 ความรู้เกี่ยวกับ GitHub

GitHub เป็นระบบจัดการเวอร์ชันโค้ด (Version Control System) บนคลาวด์ ช่วยให้ทีมพัฒนาสามารถทำงานร่วมกันได้อย่างมีประสิทธิภาพ ตรวจสอบประวัติการแก้ไข และบริหารจัดการซอร์สโค้ดได้อย่างเป็นระบบ

2.6.1 ข้อดีของ GitHub

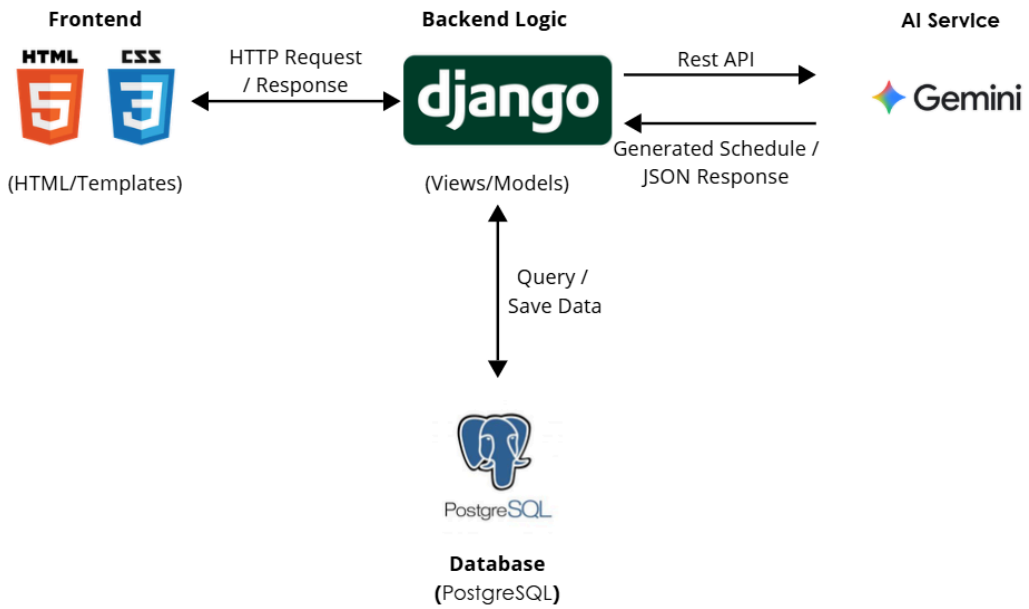
2.10.1.1 ตรวจสอบเวอร์ชันย้อนหลังของโค้ดได้
2.10.1.2 สนับสนุนการทำงานร่วมกันเป็นทีม
2.10.1.3 มีระบบ Pull Request สำหรับรีวิวโค้ดก่อนรวมเข้าระบบ

บทที่ 3

การออกแบบระบบ

ในบทนี้จะกล่าวถึงการวิเคราะห์และออกแบบระบบเว็บแอปพลิเคชั่น ระบบจัดตารางอัตโนมัติ ซึ่งการวิเคราะห์และออกแบบระบบที่ใช้ในการพัฒนาจะแบ่งออกเป็น n ดังนี้

3.1 โครงสร้างภาพรวมของระบบ (System Architecture)



ภาพประกอบที่ 3.1 สถาปัตยกรรมของระบบจัดตารางอัตโนมัติ

จากภาพประกอบที่ 3.1 สามารถอธิบายองค์ประกอบได้ดังนี้

1. django เป็นศูนย์กลางของระบบสำหรับการพัฒนาเว็บไซต์
2. Database (PostgreSQL) ทำหน้าที่จัดเก็บข้อมูลทั้งหมดของระบบ
3. AI Service ทำหน้าที่วิเคราะห์ข้อมูลเพื่อ จัดตารางเรียนอัตโนมัติ, สรุปเนื้อหาบทเรียน, และสร้างโจทย์แบบทดสอบ
4. Frontend ส่วนติดต่อผู้ใช้งาน ที่พัฒนาด้วย HTML, CSS, และ JavaScript ทำหน้าที่รับค่าจากผู้ใช้ และแสดงผลข้อมูลที่ได้จาก Django กลับมาในรูปแบบที่สวยงามและใช้งานง่าย

3.2 System requirement

3.2.1 ความต้องการของโปรแกรม (Functional Requirement)

3.2.1.1 ผู้ใช้งานทั่วไป/นักศึกษา(User/Student)

- สมัครสมาชิก
- ลงชื่อเข้าสู่ระบบ
- กรอกข้อมูลวิชา (ชื่อวิชา, ความยาก, วันที่สอบ)
- เพิ่มไฟล์การเรียนการสอน(.pdf .txt)
- กำหนดเวลาว่างในแต่ละวัน/สัปดาห์
- สร้างตารางอ่านหนังสืออัตโนมัติ
- ดูแผนการอ่าน (รายวัน / ปฏิทิน)
- ติ๊กว่าอ่านจบแล้วในแต่ละวิชา
- ทำแบบทดสอบหลังเรียน
- สรุปเนื้อหาหลังอ่าน
- ดูความคืบหน้าของแผน
- แก้ไขวิชา และ สร้างแผนใหม่
- รับการแจ้งเตือน (เวลาอ่านใกล้ถึง,มีเวลาเพิ่ม หรือใกล้ถึงวันสอบ)
- เชื่อมต่อ Google Calendar
- แสดงความคิดเห็นหรือให้คำแนะนำเพิ่มเติมเกี่ยวกับระบบ

3.2.1.2 ระบบ (System)

- ประมวลผลข้อมูลวิชา + เวลาว่าง
- คำนวณและจัดตารางอ่านหนังสือให้อัตโนมัติ
- ปรับแผนอ่านใหม่เมื่อมีการเปลี่ยนแปลง
- คำนวณเปอร์เซ็นต์ความคืบหน้าตามวิชา ที่ติ๊กแล้ว
- สร้างแบบทดสอบหลังเรียน
- สรุปเนื้อหาหลังผู้ใช้ติ๊กว่าอ่านแล้ว
- แจ้งเตือนความคืบหน้าในการอ่านแต่ละวิชา
- แจ้งเตือนเมื่อใกล้ถึงวันสอบ

3.2.2 ความต้องการด้านคุณภาพ (Non-functional Requirements)

- ระบบต้องสามารถสร้างตารางการอ่านได้
- ระบบต้องสามารถสรุปเนื้อหาได้
- ระบบต้องเชื่อมต่ออินเทอร์เน็ตก่อนใช้งานทุกครั้ง
- ระบบต้องมีการออกแบบหน้าจอที่ใช้งานง่ายและเข้าถึงฟีเจอร์ต่าง ๆ ได้อย่างรวดเร็ว
- ระบบมีการรักษาความปลอดภัยของรหัสผ่านของผู้ใช้งานrd

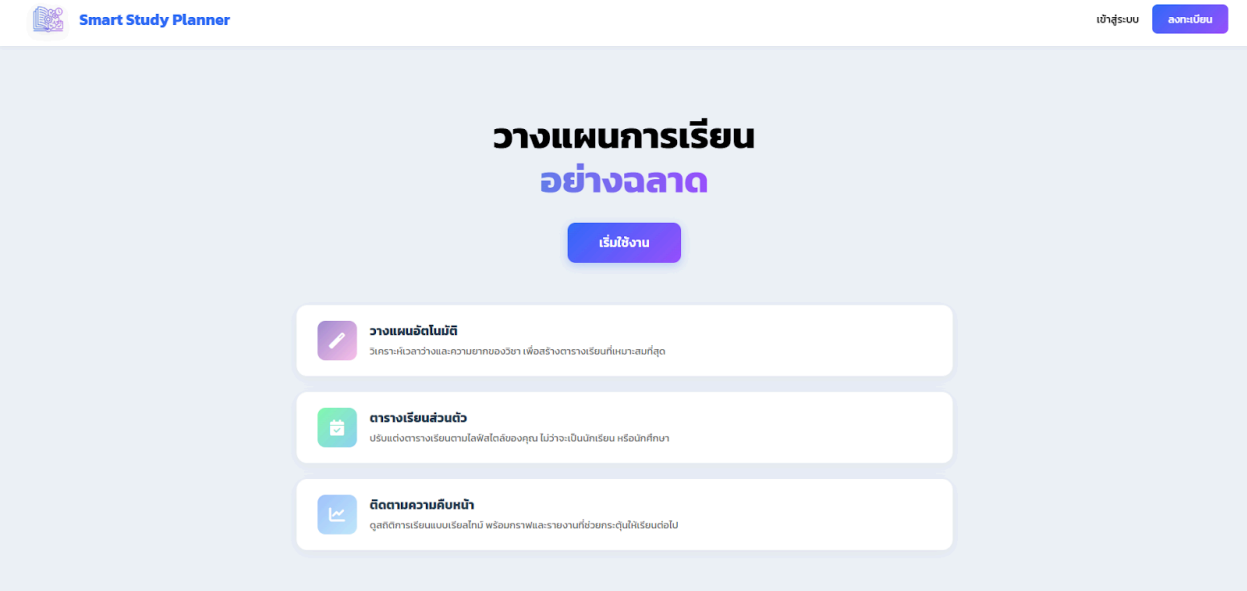
3.2.3 ความสามารถของระบบ (System Requirements)

- ระบบต้องสามารถสร้างตารางการอ่านได้อย่างถูกต้อง
- ระบบต้องสามารถ
- ระบบต้องสามารถส่งการแจ้งเตือนเพื่อเตือนให้ผู้ใช้รู้ได้ว่าต้องอ่านวิชาไหนบ้าง

- ระบบต้องสามารถให้ผู้ใช้ จัดการข้อมูลส่วนตัว ของตนเองได้ เช่น การอัปเดตข้อมูลส่วนตัว, การลบข้อมูล หรือ การเพิ่มข้อมูลส่วนตัว
- ระบบต้องสามารถสรุปเนื้อหาสำคัญให้ผู้ใช้เข้าใจง่าย
- ระบบต้องสามารถสร้างแบบทดสอบที่ถูกต้องได้

3.3 การออกแบบหน้าจอ

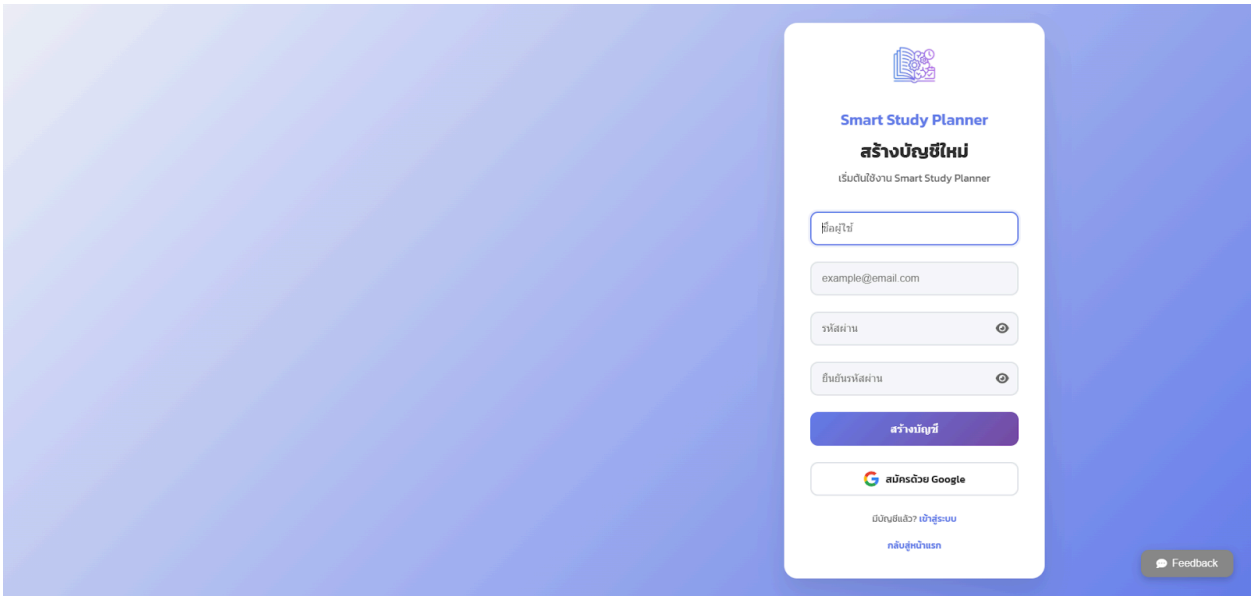
3.3.1 ส่วนของหน้า Landing Page



ภาพประกอบที่ 3.2 หน้า Landing Page

จากภาพประกอบที่ 3.2 แสดงหน้า

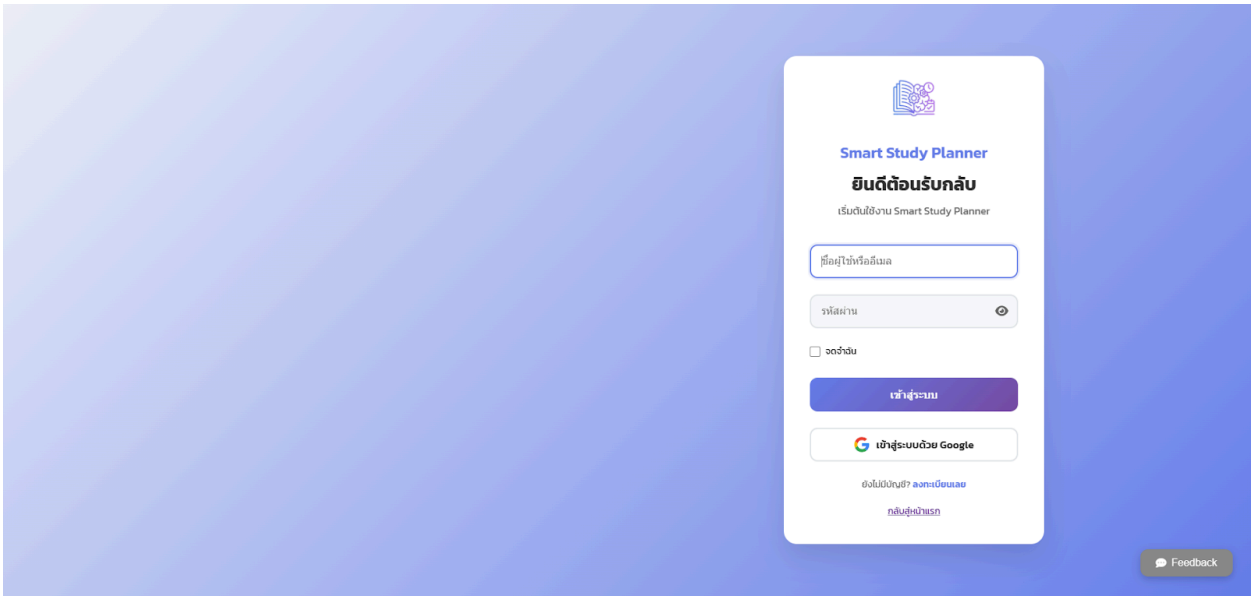
3.3.2 ส่วนของหน้าลงทะเบียน



ภาพประกอบที่ 3.3 หน้าลงทะเบียน

จากภาพประกอบที่ 3.3 แสดงหน้าลงทะเบียนของระบบ โดยเมื่อผู้ใช้งานต้องการใช้ระบบระบบจัดตารางอัตโนมัติ ผู้ใช้งานจำเป็นต้องลงทะเบียนหรือ สมัครสมาชิกในหน้านี้ก่อนจึงนำข้อมูลในส่วนนี้ไปกรอกที่หน้าลงชื่อเข้าสู่ระบบ

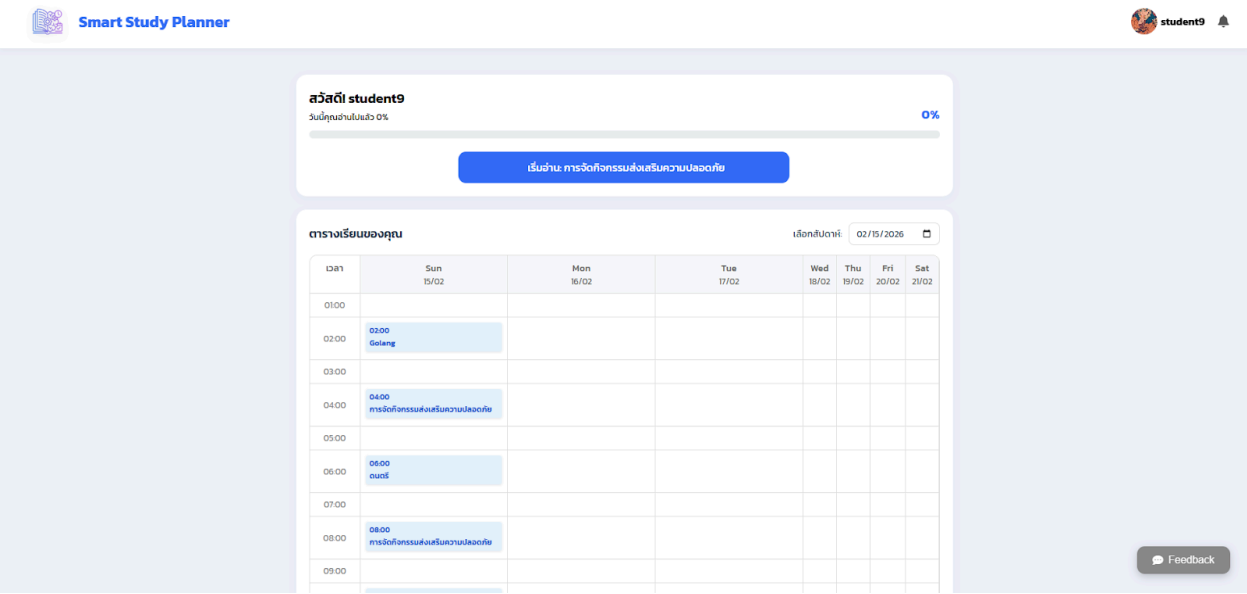
3.3.3 ส่วนของหน้าลงชื่อเข้าสู่ระบบ



ภาพประกอบที่ 3.4 หน้าลงชื่อเข้าสู่ระบบ

จากภาพประกอบที่ 3.4 แสดงหน้าเข้าสู่ระบบในกรณีที่ผู้ใช้งานยังไม่เคยลงชื่อเข้าใช้งานระบบ โดยที่ผู้ใช้งานจำเป็นต้องลงทะเบียนหรือ สมัครสมาชิกให้เรียบร้อยเสียก่อน จากนั้นจึงนำชื่อผู้ใช้งานและรหัสผ่านที่ตั้งไว้ในตอนที่สมัครสมาชิกมากรอกในหน้านี้ เพื่อทำการลงชื่อเข้าสู่ระบบ

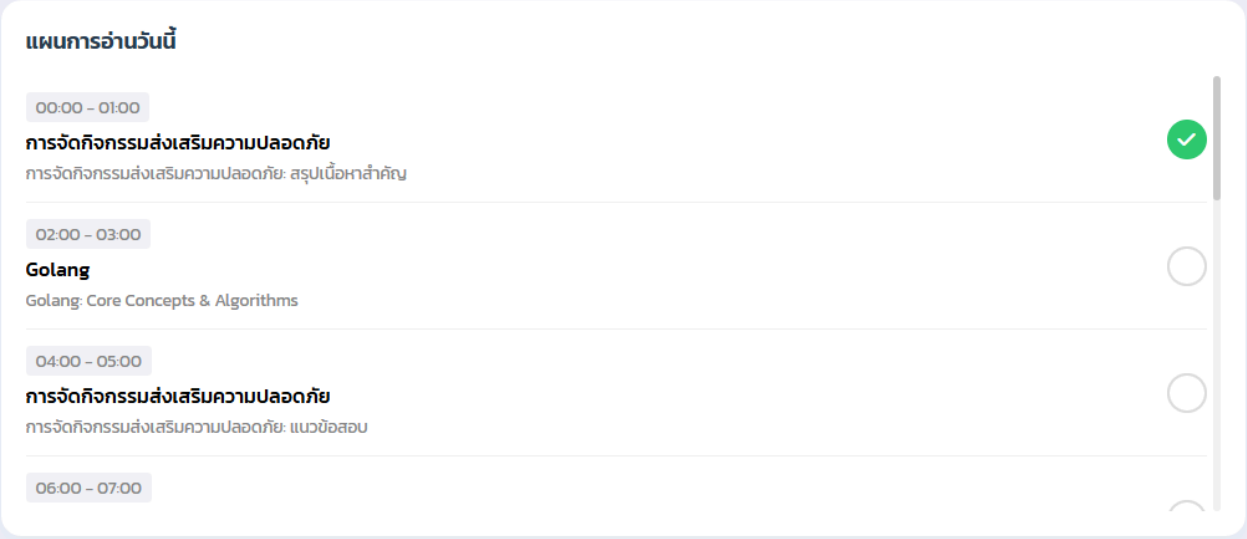
3.3.4 ส่วนของตารางการอ่าน



ภาพประกอบที่ 3.5 **ตารางการอ่าน**

จากภาพประกอบที่ 3.5 แสดงตารางการอ่าน โดยจะแสดงวันเวลาและวิชาที่ต้องอ่านในรูปแบบของปฏิทิน

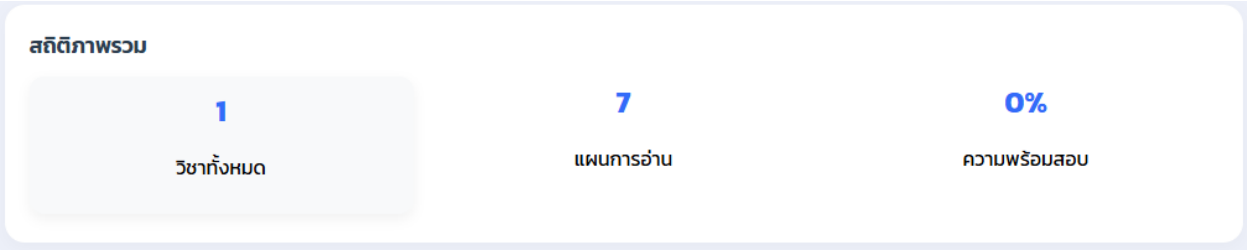
3.3.5 ส่วนของแผนการอ่าน



ภาพประกอบที่ 3.6 **แผนการอ่าน**

จากภาพประกอบที่ 3.6 แสดงหน้าแผนการอ่าน โดยจะแสดงแผนการอ่านในวันนั้นๆ หาหัวข้อไหนที่อ่านไปแล้วจะมีติ๊กถูกสีเขียว

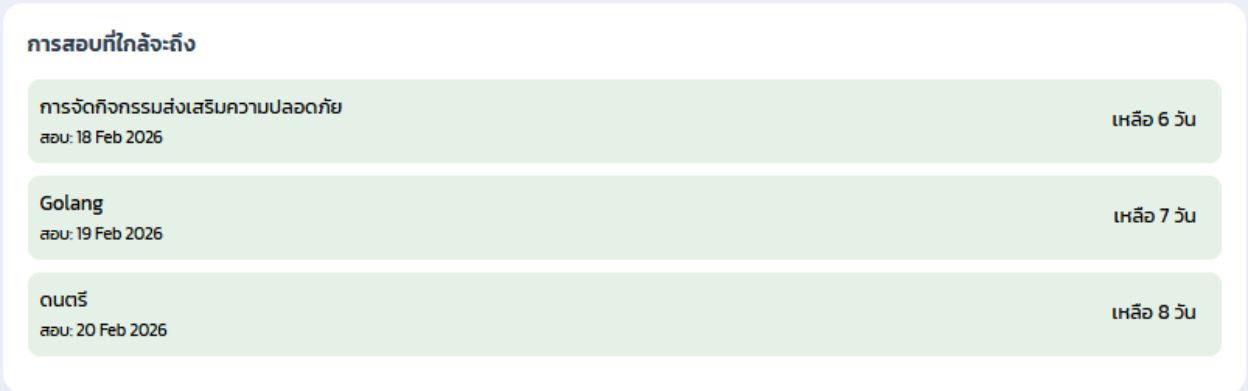
3.3.6 ส่วนของสถิติภาพรวม



ภาพประกอบที่ 3.7 **สถิติภาพรวม**

จากภาพประกอบที่ 3.7 แสดงหน้าสถิติภาพรวม โดยจะแสดงรายวิชาทั้งหมดที่ต้องอ่าน มีแผนการอ่านทั้งหมดกี่แผน และอ่านไปแล้วกี่เปอร์เซ็น

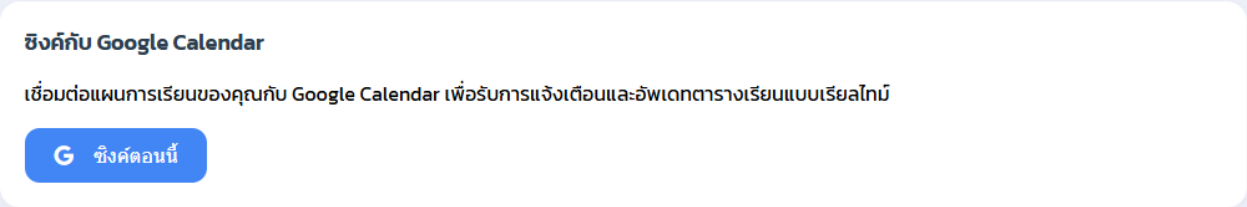
3.3.7 ส่วนของการสอบที่ใกล้จะถึง



ภาพประกอบที่ 3.8 **สถิติภาพรวม**

จากภาพประกอบที่ 3.8 แสดงหน้าการสอบที่ใกล้จะถึง โดยจะแสดงว่าเหลือเวลาในการอ่านอีกกี่วันก่อนที่จะถึงวันสอบ

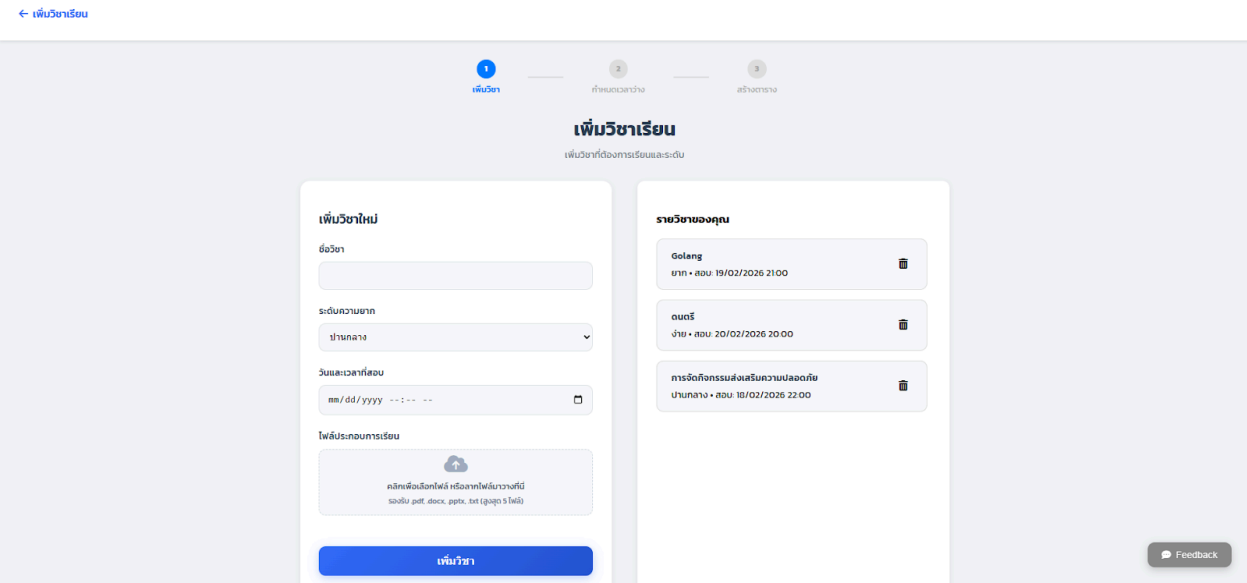
3.3.8 ส่วนของการซิงค์กับ Google Calendar



ภาพประกอบที่ 3.9 ซิงค์กับ Google Calendar

จากภาพประกอบที่ 3.9 แสดงหน้า**การซิงค์กับ Google Calendar โดยหากผู้ใช้กดซิงค์ ข้อมูลการอ่านทั้งหมดจะซิงค์และแสดงบน** Google Calendar

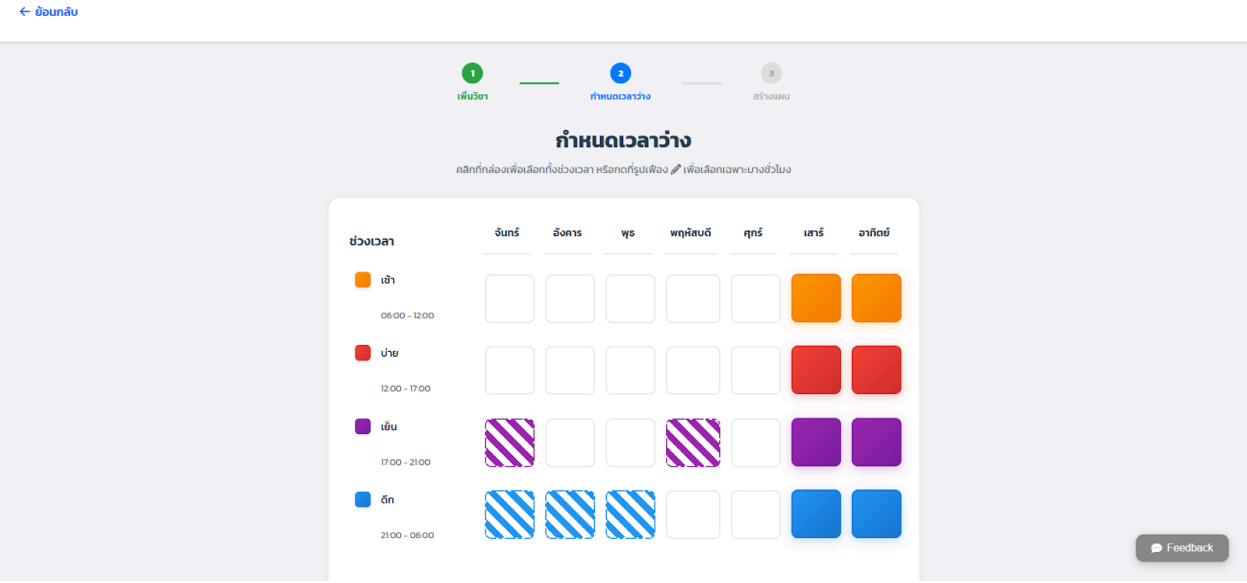
3.3.9 ส่วนของการเพิ่มวิชา



ภาพประกอบที่ 3.10

จากภาพประกอบที่ 3.10 แสดงหน้าเพิ่มวิชา โดยผู้ใช้สามารถเพิ่มวิชาที่จะอ่าน และไฟล์การเรียนการสอนได้ หนึ่งวิชาเพิ่มได้สูงสุก 5 ไฟล์ ขนาดไฟล์ม่เกิน 10 MB

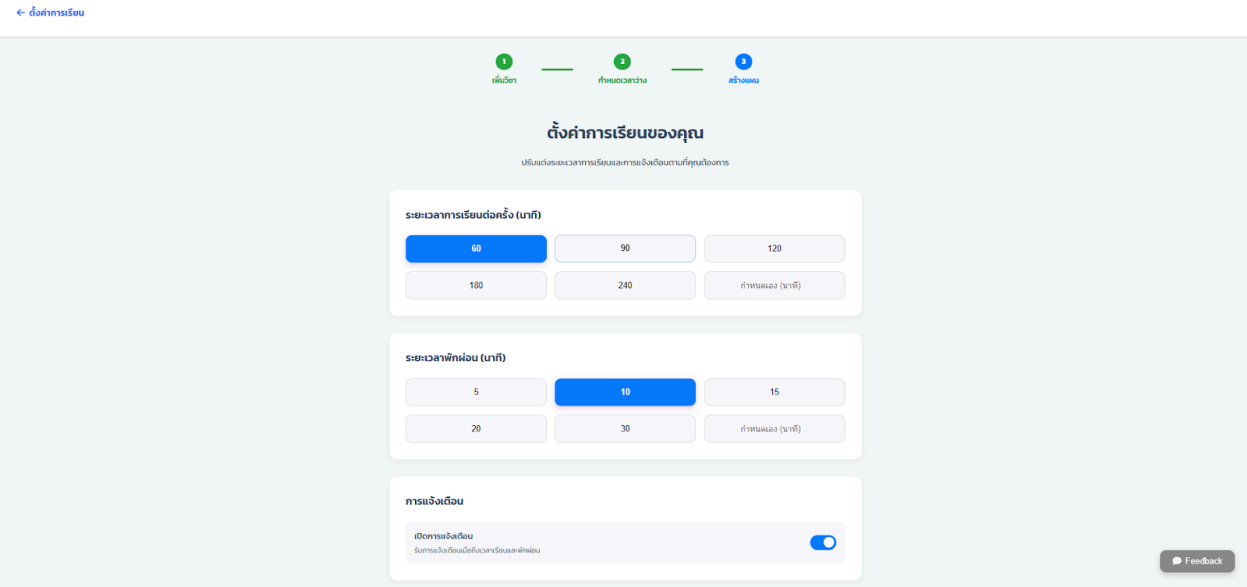
3.3.10 ส่วนของการกำหนดเวลาว่าง



ภาพประกอบที่ 3.11

จากภาพประกอบที่ 3.11 แสดงหน้ากำหนดเวลาว่าง โดยแบ่งเป็น 4 ช่วงเวลา ได้แก่ เช้า, บ่าย, เย็น, ดึก และสามารถเลือกเป็นรายชั่วโมงได้

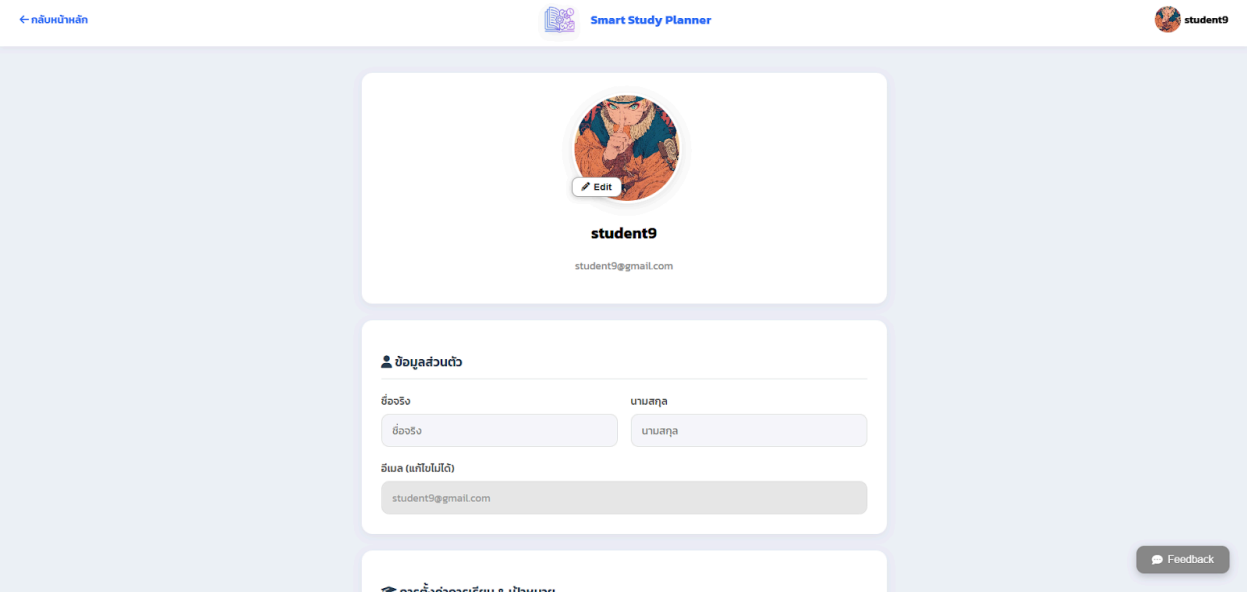
3.3.11 ส่วนของการตั้งค่าการอ่าน



ภาพประกอบที่ 3.12

จากภาพประกอบที่ 3.12 แสดงหน้าตั้งค่าการอ่าน โดยผู้ใช้สามารถเวลาในการอ่าน และเวลาพักในแต่ละครั้งได้

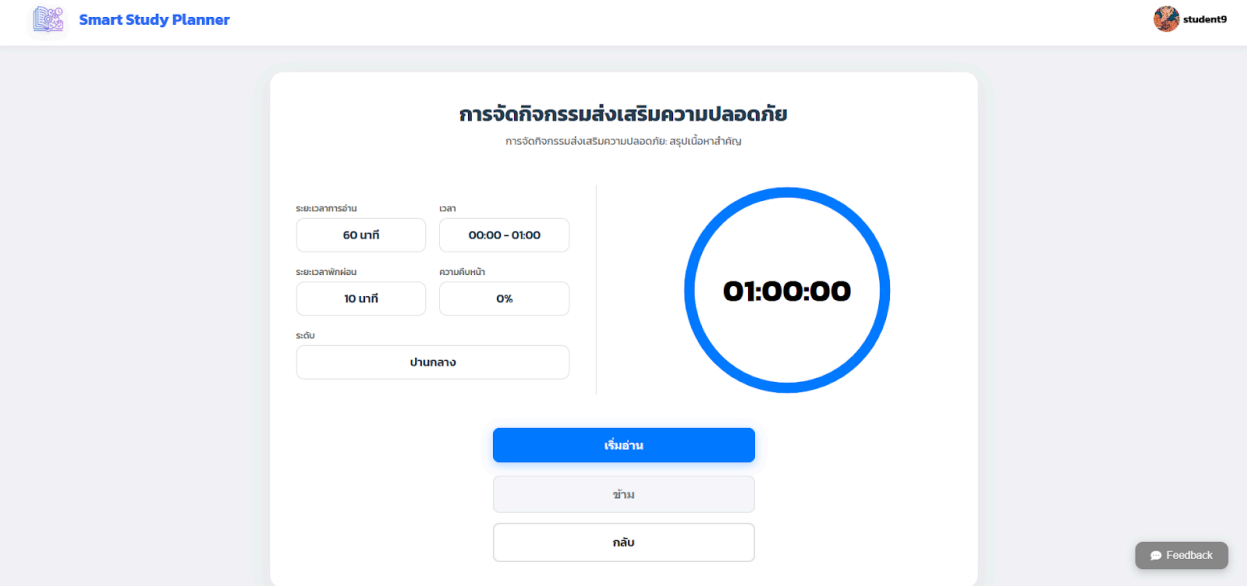
3.3.12 ส่วนของข้อมูลผู้ใช้



ภาพประกอบที่ 3.13

จากภาพประกอบที่ 3.13 แสดงหน้าข้อมูลผู้ใช้ โดยผู้ใช้สามารถ เปลี่ยนโปรไฟล์ และชื่อได้

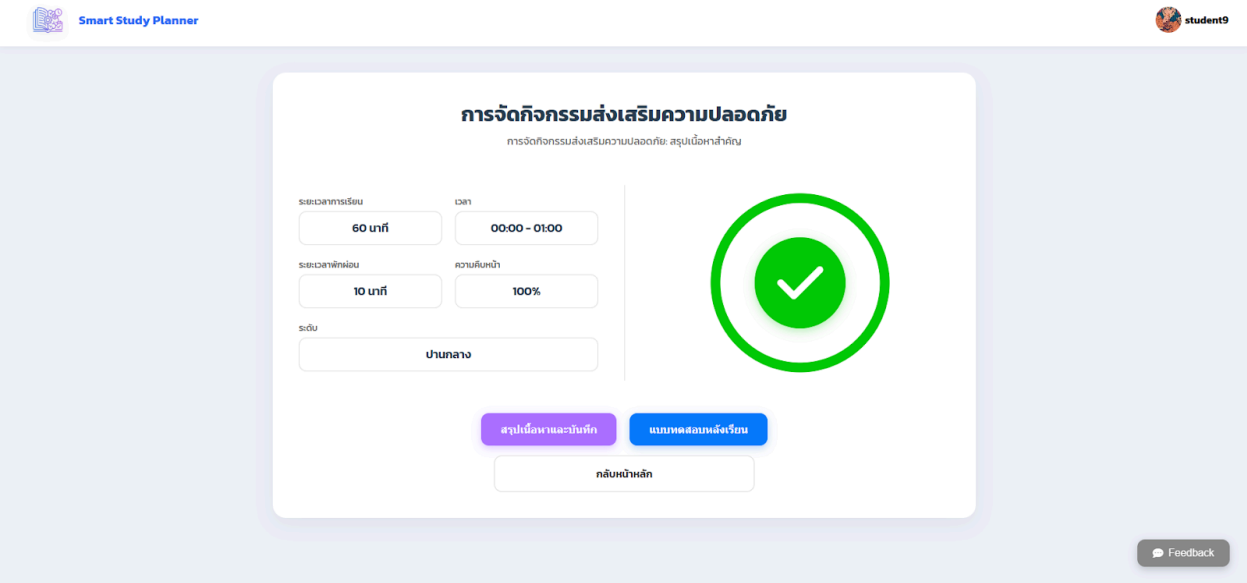
3.3.13 ส่วนของหน้าเริ่มอ่าน



ภาพประกอบที่ 3.14

จากภาพประกอบที่ 3.14 แสดงหน้าเริ่มอ่าน โดยจะแสดงรายละเอียด เช่น หัวข้อการอ่าน เวลาการอ่าน ช่วงเวลา ความคืบหน้า และระดับความอยากง่ายของวิชา

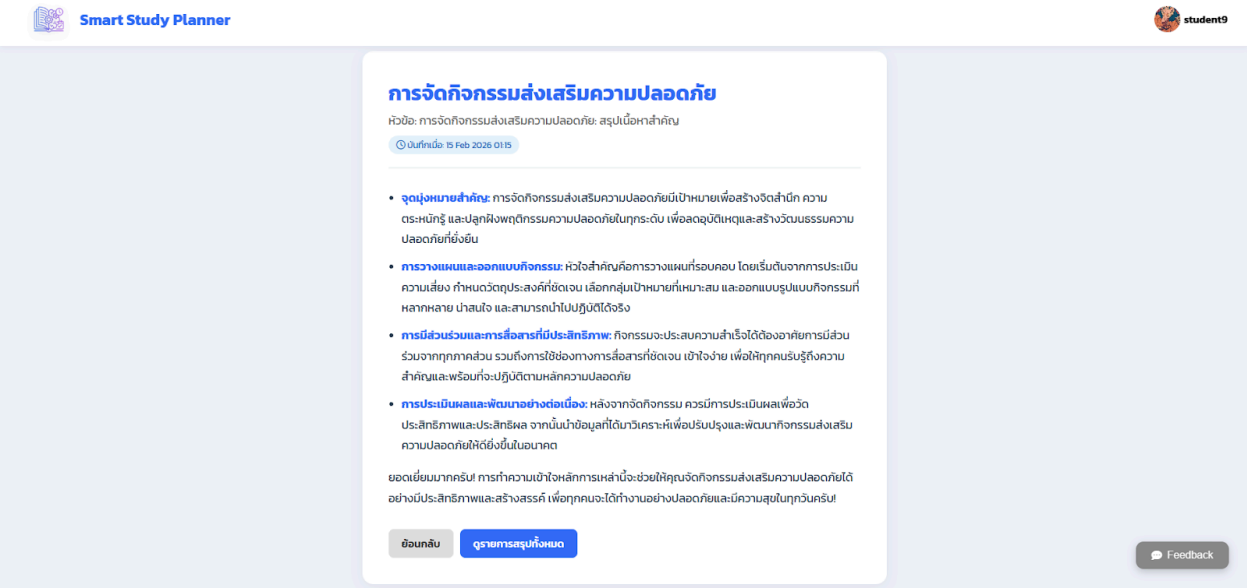
3.3.14 ส่วนของหน้าหมดเวลาการอ่าน



ภาพประกอบที่ 3.15

จากภาพประกอบที่ 3.15 แสดงหน้าหมดเวลาการอ่านในครั้งนั้นๆ สามารถเลือกได้ว่าจะสรุปเนื้อหาหรือทำแบบทดสอบหลังเรียน

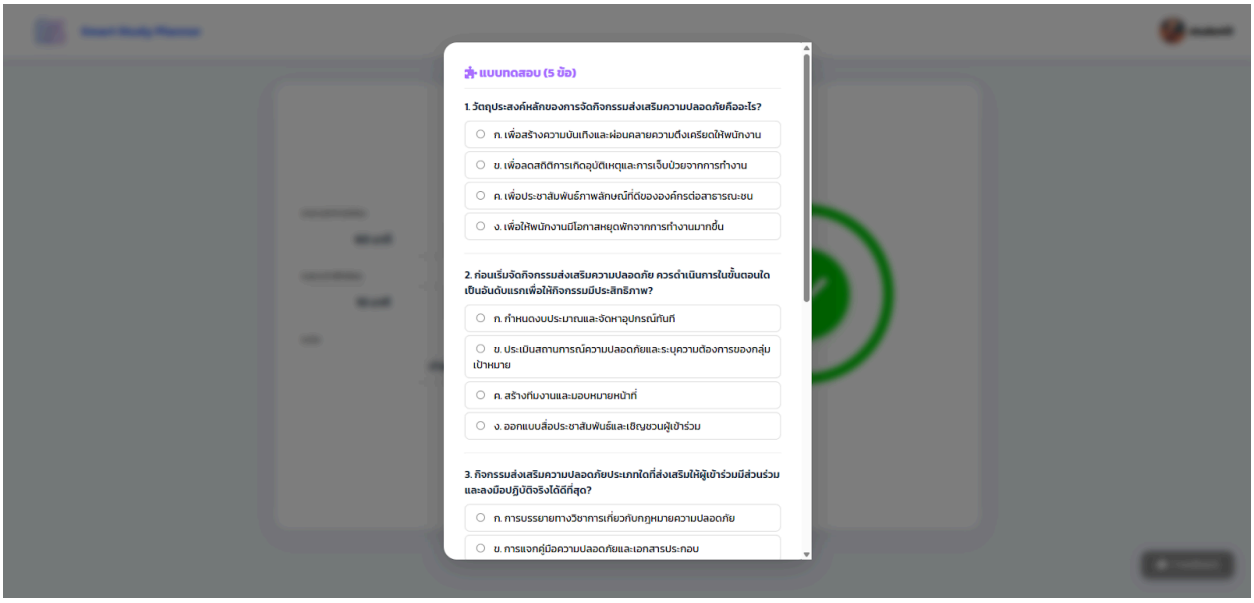
3.3.15 ส่วนของการสรุปเนื้อหา



ภาพประกอบที่ 3.16

จากภาพประกอบที่ 3.16 แสดงหน้าเนื้อหาที่สรุปโดย AI อ้างอิงจากเนื้อหาหรือหัวข้อที่อ่านในครั้งนั้น

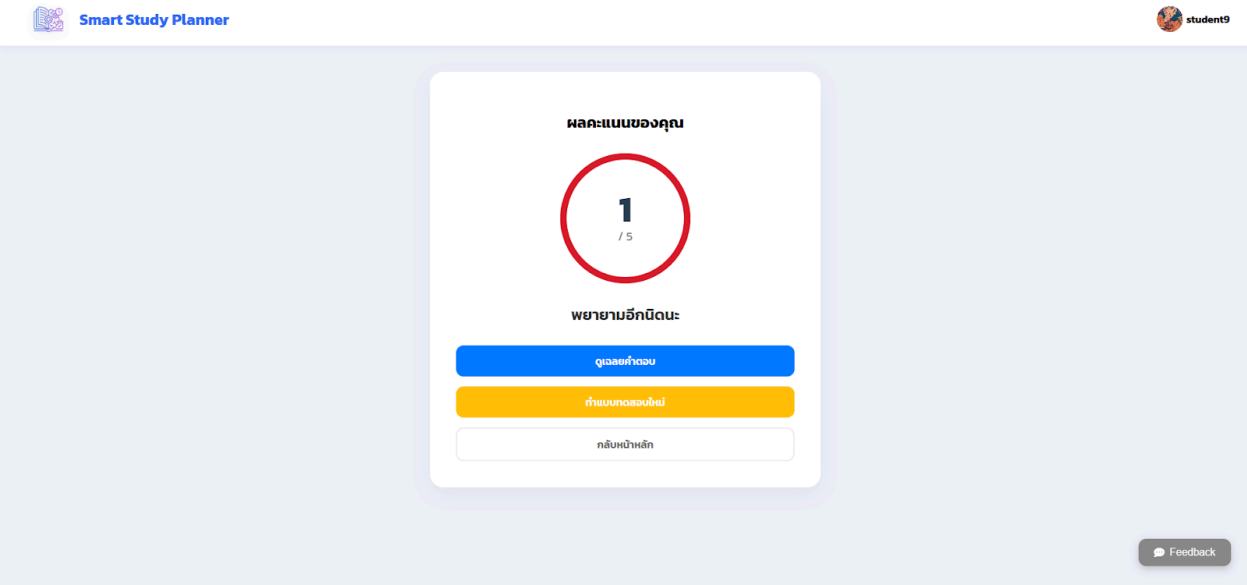
3.3.16 ส่วนของแบบทดสอบ



ภาพประกอบที่ 3.17

จากภาพประกอบที่ 3.17 แสดงหน้าแบบทดสอบที่ออกโดย AI ทั้งหมด 5 ข้อ อ้างอิงจากเนื้อหาหรือหัวข้อที่อ่านในครั้งนั้น

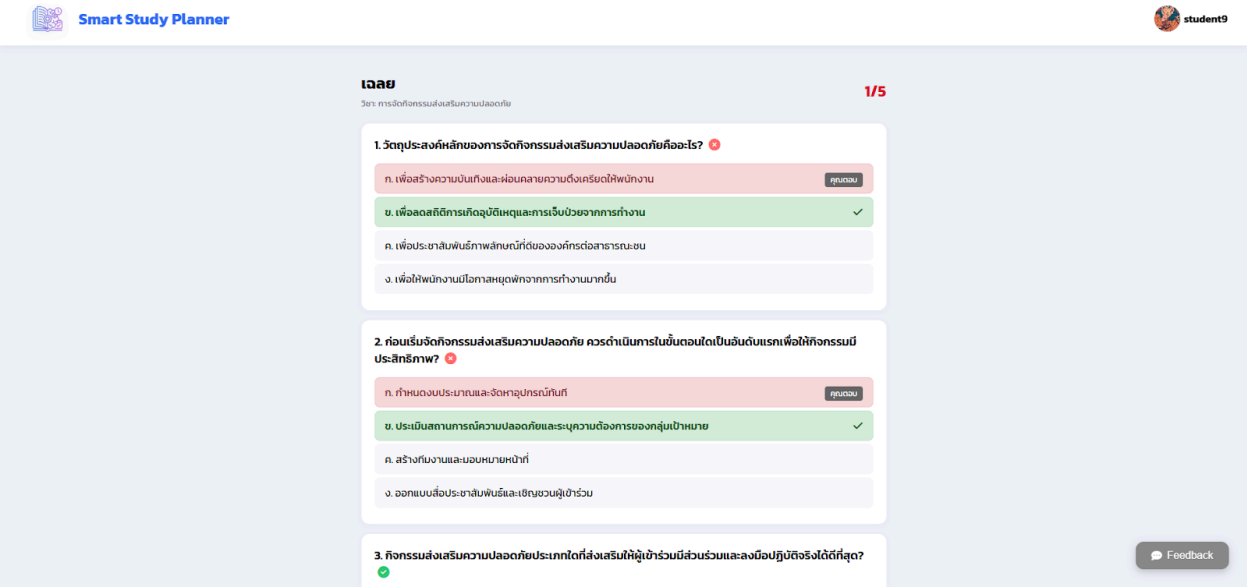
3.3.17 ส่วนของคะแนนการทำบททดสอบ



ภาพประกอบที่ 3.18

จากภาพประกอบที่ 3.18 แสดงหน้าคะแนนการทำบททดสอบ โดยผู้ใช้สามารถเลือกได้ว่าจะดูเฉลยหรือทำแบบทดสอบใหม่อีกครั้ง

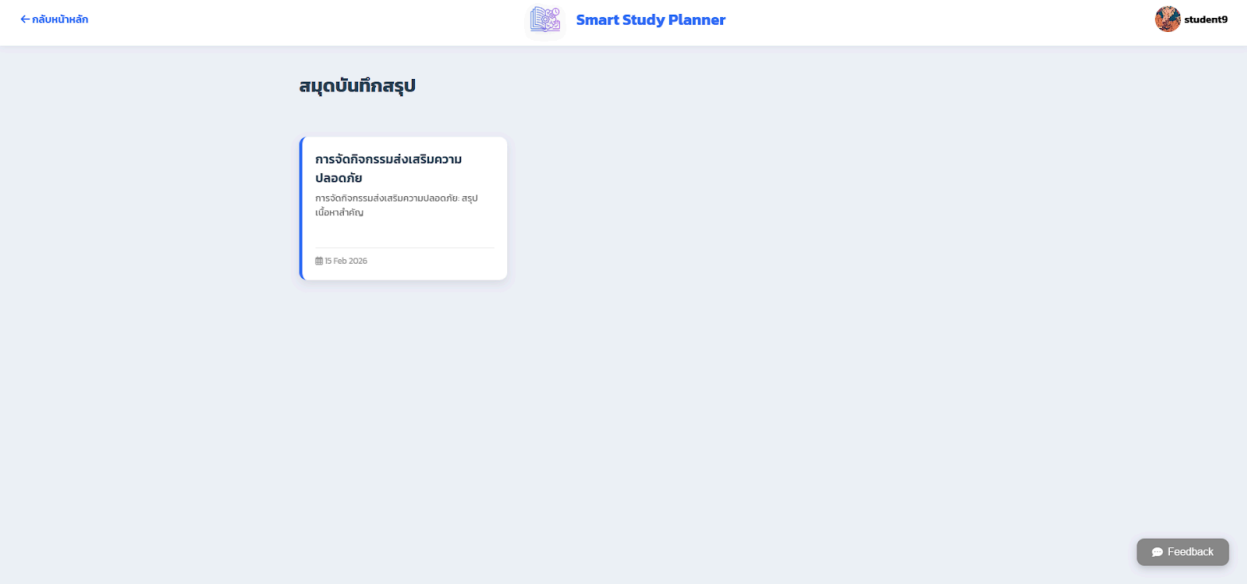
3.3.18 ส่วนของการเฉลยแบบทดสอบ



ภาพประกอบที่ 3.19

จากภาพประกอบที่ 3.19 แสดงหน้าเฉลยแบบทดสอบ โดยแสดงคำตอบที่ผู้ใช้เลือก และคำตอบที่ถูกต้อง

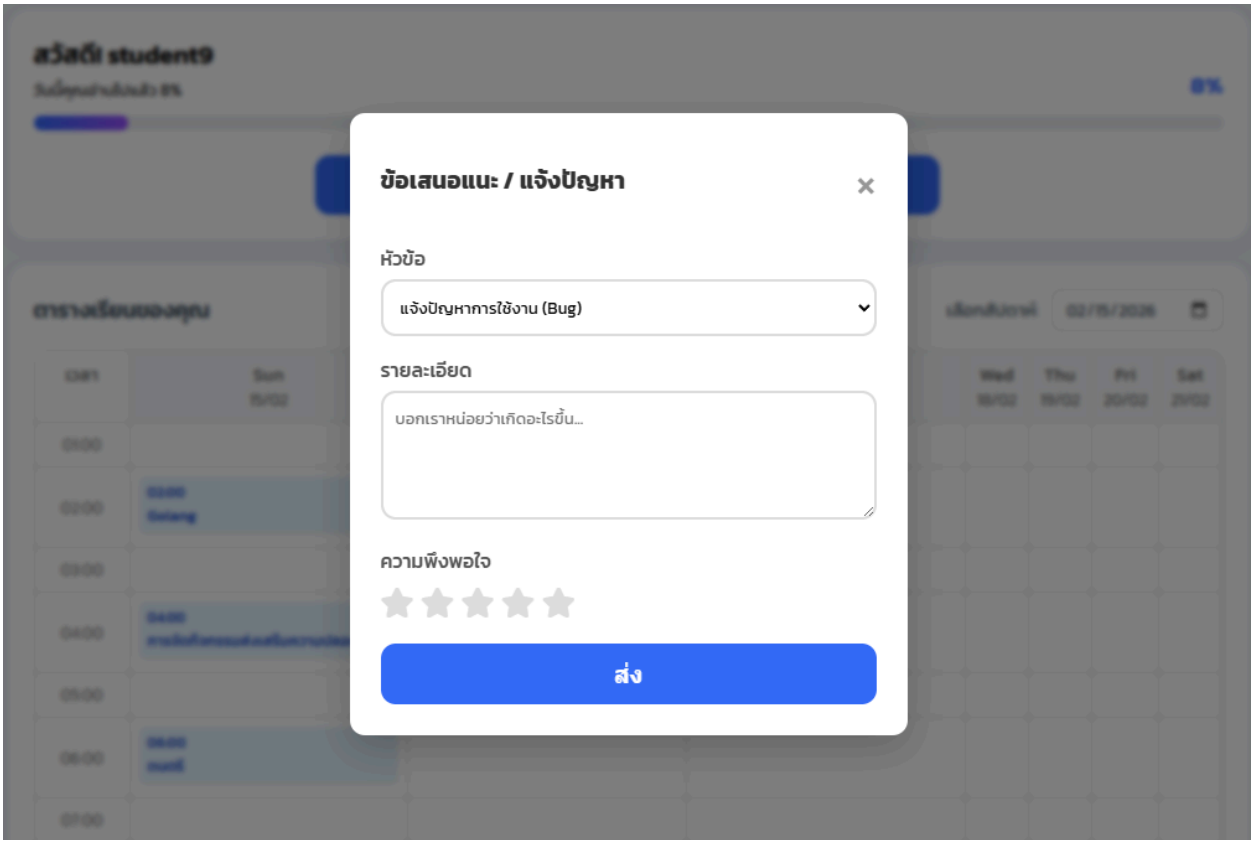
3.3.19 ส่วนของสุมดบันทึกเนื้อหาที่สรุป



ภาพประกอบที่ 3.20

จากภาพประกอบที่ 3.20 แสดงหน้าสุมดบันทึกเนื้อหาที่สรุป โดยจะบันทึกเนื้อหาที่ผู้ใช้สรุปไว้หลังการอ่าน เพื่อให้ผู้ใช้ย้อนกับมาอ่านเนื้อหาที่เคยสรุปไว้ได้

3.3.20 ส่วนของการแจ้งปัญหา



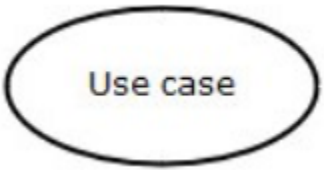
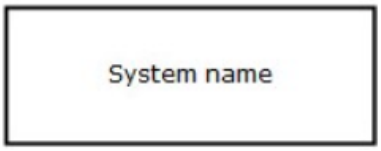
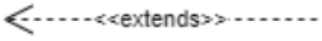
ภาพประกอบที่ 3.21

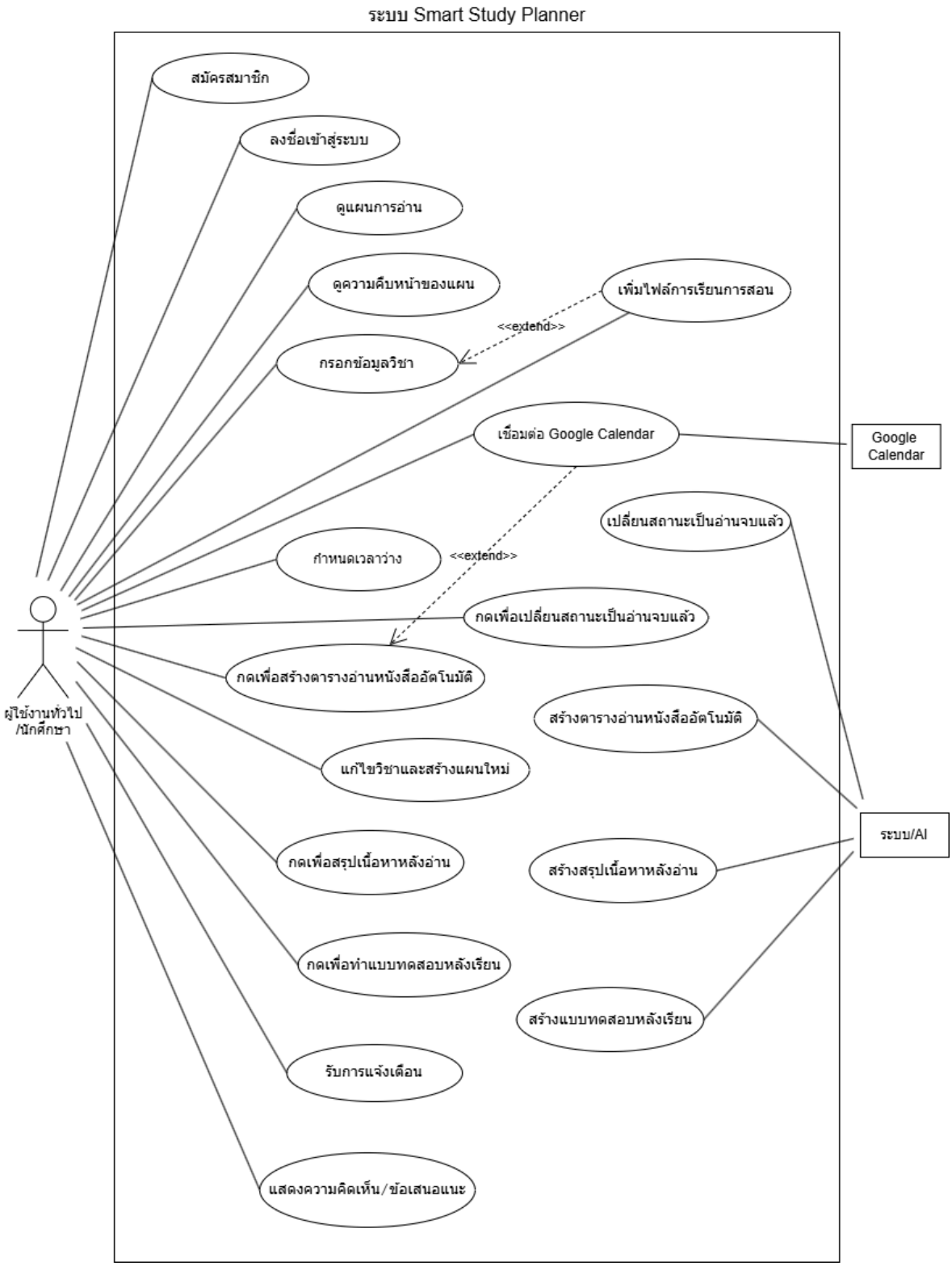
จากภาพประกอบที่ 3.21 แสดงหน้าการแจ้งปัญหา หรือให้ข้อเสนอแนะเกี่ยวกับระบบ โดยผู้ใช้สามารถแจ้งปัญาต่างๆ ได้

3.4 Use case diagram

Usecase Diagram เป็นแผนผังเพื่อแสดงฟังก์ชั่นการทำงานของระบบโดยรวมแสดงส่วนประกอบ ในระบบและกิจกรรม ที่เกิดขึ้นในระบบ สัญลักษณ์สำคัญต่างๆที่ใช้สำหรับการเขียน Usecase Diagram แสดงในตารางที่ 3.1

ตารางที่ 3.1 สัญลักษณ์ของ Use case Diagram

สัญลักษณ์	การใช้งาน
	Use case คือส่วนย่อยของระบบงาน แทนด้วยวงรีและชื่อของ Use case ภายในวงรี
	Actor คือบุคคลหรือระบบงานอื่นที่ใช้งานระบบหรือได้รับประโยชน์จากระบบซึ่งอยู่ภายนอกระบบ แทนด้วยรูปคนและมีชื่อบทบาทการใช้งานระบบ
	เส้นตรงที่แสดงถึงการใช้งาน Use case ของผู้กระทำ
	กรอบสี่เหลี่ยม แสดงถึงขอบเขตของระบบโดยแสดงชื่อระบบภายในหรือด้านบนกรอกสี่เหลี่ยม Use case อยู่ภายในกรอบสี่เหลี่ยม และ actor อยู่ภายนอกกรอบสี่เหลี่ยม
	ความสัมพันธ์แบบ «include» แสดงว่า Use case หนึ่งดำเนินการ ตามขั้นตอนของ Use case อื่น โดยแทนด้วยสัญลักษณ์ลูกศรเส้นประซึ่ง Use case ที่หางลูกศรเรียกใช้งาน Use case ที่หัวลูกศร ทุกครั้งที่มีการทำงาน
	ความสัมพันธ์แบบ «extends» แสดงว่า Use case หนึ่งดำเนินการ ตามขั้นตอนของ Use case อื่น โดยแทนด้วยสัญลักษณ์ลูกศรเส้นประซึ่ง Use case ที่หัวลูกศรเรียกใช้งาน Use case ที่หางลูกศร แต่การใช้งานไม่จำเป็นต้องเกิดขึ้นทุกครั้งขึ้นอยู่กับเงื่อนไขระหว่างการทำงาน



ภาพประกอบที่ 3.22 Usecase Diagram ของระบบจัดตารางอัตโนมัติ Smart Study Planner

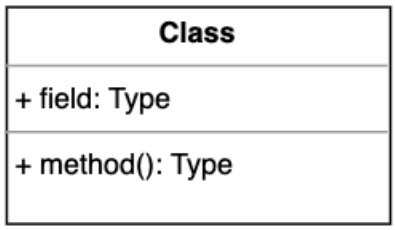
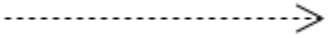
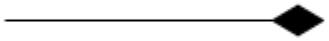
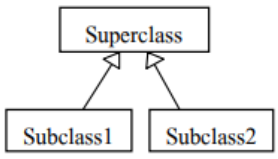
ตารางที่ 3.2 อธิบาย Use Case หน้าที่ของระบบ

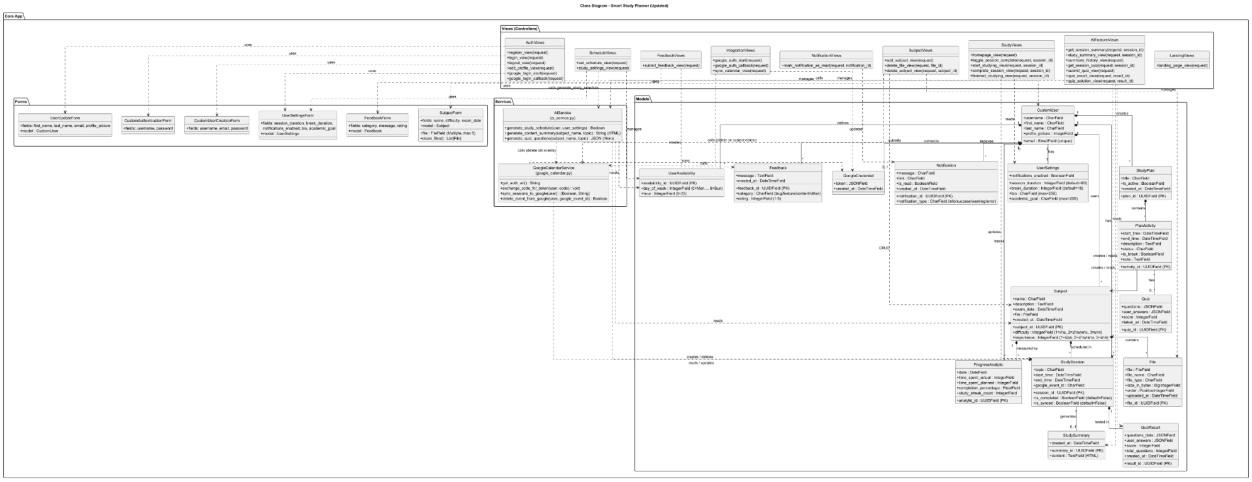
Use Case	คำอธิบาย
สมัครสมาชิก	ผู้ใช้งานกรอกข้อมูล เพื่อสร้างบัญชีสำหรับเข้าใช้งานระบบ
ลงชื่อเข้าสู่ระบบ	ผู้ใช้งานสามารถเข้าสู่ระบบได้โดยใช้ข้อมูลจากการสมัครสมาชิก
กรอกข้อมูลวิชา	ผู้ใช้งานเพิ่มข้อมูลรายวิชาที่ต้องการอ่าน โดยระบุชื่อวิชา,ความอยาก และวันที่สอบ เพื่อนำไปใช้ในการวางแผนการเรียน
เพิ่มไฟล์การเรียนการสอน	ผู้ใช้งานอัปโหลดไฟล์เอกสารที่เกี่ยวข้องกับรายวิชา เพื่อเป็นข้อมูลสำหรับการอ่านทบทวนหรือให้ AI ประมวลผล
กำหนดเวลาว่าง	ผู้ใช้งานระบุช่วงเวลาที่สะดวกในการอ่านหนังสือในแต่ละวัน เพื่อให้ระบบนำไปคำนวณและจัดตารางเรียนให้เหมาะสม
สร้างตารางอ่านหนังสืออัตโนมัติ	ระบบทำการประมวลผลข้อมูลรายวิชาและเวลาว่างของผู้ใช้ เพื่อจัดสรรตารางเรียนให้อัตโนมัติ
ดูแผนการอ่าน	ผู้ใช้งานสามารถดูตารางการอ่านที่ระบบจัดให้
แก้ไขวิชาและสร้างแผนใหม่	ผู้ใช้งานแก้ไขข้อมูลรายวิชาหรือการตั้งค่าการเรียน ซึ่งจะส่งผลให้ระบบทำการคำนวณและสร้างแผนการเรียนชุดใหม่ให้สอดคล้องกับการเปลี่ยนแปลง
เชื่อมต่อ Google Calendar	ผู้ใช้งานทำการเชื่อมต่อบัญชี Google เพื่อให้ระบบส่งข้อมูลตารางเรียนที่จัดไว้ไปแสดงผลบน Google Calendar ส่วนตัวได้
ดูความคืบหน้าของแผน	ผู้ใช้งานดูสถิติแสดงความก้าวหน้าในการเรียน เช่น จำนวนเปอร์เซ็นต์ของเนื้อหาที่อ่านจบแล้วเทียบกับแผนทั้งหมด
สรุปเนื้อหาหลังอ่าน	ผู้ใช้งานกดขอสรุปบทเรียน ระบบจะใช้ AI ในการประมวลผลเนื้อหาและแสดงบทสรุปสาระสำคัญให้อ่านทบทวน
ทำแบบทดสอบหลังเรียน	ผู้ใช้งานทำแบบทดสอบที่ระบบ AI สร้างขึ้นจากเนื้อหาที่เรียน เพื่อประเมินความเข้าใจของตนเอง และดูผลคะแนนพร้อมเฉลย
ติ๊กว่าอ่านจบแล้ว	ผู้ใช้งานบันทึกสถานะการเรียนของคาบเรียนนั้นๆ ว่า "สำเร็จ" เมื่ออ่านหนังสือจบ เพื่อให้ระบบอัปเดตความคืบหน้า
รับการแจ้งเตือน	ผู้ใช้งานรับการแจ้งเตือนจากระบบ เช่น แจ้งเตือนเมื่อใกล้ถึงเวลาอ่านหนังสือ, แจ้งเตือนวันสอบ
แสดงความคิดเห็น/ข้อเสนอแนะ	ผู้ใช้งานส่งข้อความเสนอแนะ รายงานปัญหาการใช้งาน หรือให้คำติชมเกี่ยวกับระบบไปยังผู้ดูแลระบบเพื่อการปรับปรุงแก้ไข

3.5 Class diagram

Class Diagram คือ แผนภาพที่ใช้แสดงคลาสและความสัมพันธ์ในแบบต่างๆ ระหว่างคลาส โดยมีสัญลักษณ์ที่ใช้ในการเขียน Class Diagram ดังนี้

ตารางที่ 3.3 สัญลักษณ์ของ Class Diagram

สัญลักษณ์	คำอธิบาย
	Class สัญลักษณ์ของ Class จะถูกแสดงในรูปสี่เหลี่ยม โดยมีส่วนบนสุดของสี่เหลี่ยมเป็นชื่อของ Class ส่วนกลางของสี่เหลี่ยมเป็น Attribute หรือ field ของ Class และส่วนล่างของสี่เหลี่ยมเป็น Method ของ Class โดยใช้รูปแบบเดียวกันกับภาษาโปรแกรมอื่น ๆ ที่ใช้สัญลักษณ์เหลี่ยมเพื่อแทน Class
Method Name()	Method Name()คือฟังก์ชันหรือโปรแกรมย่อยที่ถูกเขียนไว้ภายในคลาส โดยที่ชื่อของ Method จะใช้เรียกใช้งานฟังก์ชันเหล่านั้น เพื่อประมวลผลหรือดำเนินการต่างๆ กับข้อมูลในคลาสนั้นๆ
	Dependency Relationship หมายความว่า ความสัมพันธ์ระหว่างคลาสที่แสดงถึงการใช้งานหรือการอ้างอิงจากคลาสหนึ่งไปยังอีกคลาสหนึ่ง
	Composition ใช้ในกรณีที่วัตถุหนึ่งประกอบไปด้วยวัตถุอื่นและจะต้องถูกลบตามไปด้วยถ้าเจ้าของถูกลบ
	Aggregation Relationship เป็นความสัมพันธ์ระหว่างสองคลาสแบบมีส่วนหนึ่ง
	Generalization Relationship แสดงความสัมพันธ์ของ Class ที่มีลักษณะพิเศษเพิ่มเติมจาก Class แรก โดยเรียก Class ที่มีลักษณะพิเศษว่า Subclass และเรียก Class ที่เป็น Class หลักว่า Super class



ภาพประกอบที่ 3.23 Class Diagram ของของระบบจัดตารางอัตโนมัติz Smart Study Planner

ดูรายละเอียดภาพเพิ่มเติม : Class_Diagram__Smart_Study_Planner_1.png

ตารางที่ 3.4 อธิบาย Class Diagram ของคลาสระบบ

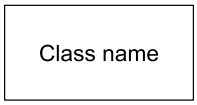
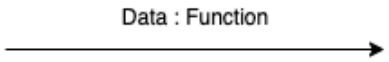
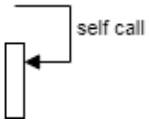
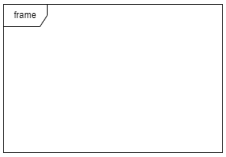
Class Diagram	คำอธิบาย
CustomUser	โมเดลสำหรับจัดการข้อมูลผู้ใช้งาน เช่น ชื่อผู้ใช้ (Username), อีเมล (Email), รหัสผ่าน (Password), รูปโปรไฟล์
Subject	โมเดลสำหรับเก็บข้อมูลรายวิชาที่ผู้ใช้สร้างขึ้น ประกอบด้วยชื่อวิชา, วันที่สอบ เพื่อใช้แสดงผลในตารางเรียน
File	โมเดลสำหรับจัดการไฟล์เอกสารประกอบการเรียนที่ผู้ใช้อัปโหลด เก็บรายละเอียด เช่น ชื่อไฟล์, ชนิดไฟล์, ขนาดไฟล์
UserAvailability	โมเดลสำหรับบันทึกข้อมูลช่วงเวลาว่างของผู้ใช้ในแต่ละวัน (ระบุเป็นวันในสัปดาห์และช่วงเวลา)
UserSettings	โมเดลสำหรับเก็บการตั้งค่าการเรียนของผู้ใช้แต่ละคน เช่น ระยะเวลาในการอ่าน หรือพักต่อครั้ง
StudySession	โมเดลสำหรับเก็บข้อมูลตารางเรียน ประกอบด้วยหัวข้อที่เรียน, เวลาเริ่ม-จบ, สถานะการเรียน (สำเร็จ/ยังไม่เริ่ม) และสถานะการซิงค์กับ Google Calendar
StudySummary	โมเดลสำหรับเก็บประวัติการสรุปเนื้อหา โดยจะเชื่อมโยงกับ StudySession เพื่อให้ผู้ใช้กลับมาอ่านทบทวนได้ภายหลัง
QuizResult	โมเดลสำหรับเก็บผลคะแนนการทำแบบทดสอบท้ายบทเรียน รวมถึงข้อมูลชุดคำถาม (JSON), คำตอบที่ผู้ใช้เลือก, คะแนนที่ได้
Notification	โมเดลสำหรับจัดการระบบแจ้งเตือนภายในแอปพลิเคชัน เก็บข้อความแจ้งเตือน
Feedback	โมเดลสำหรับเก็บข้อมูลข้อเสนอแนะ หรือการรายงานปัญหาการใช้งานที่ผู้ใช้ส่งเข้ามายังผู้ดูแลระบบ
GoogleCredential	โมเดลสำหรับเก็บข้อมูล Token และ Credential ที่ได้รับจากการยืนยันตัวตนผ่าน Google OAuth เพื่อใช้ในการเชื่อมต่อและจัดการข้อมูลบน Google Calendar
SubjectForm	ฟอร์มสำหรับจัดการข้อมูลของรายวิชาและไฟล์ประกอบการเรียน เช่น ตรวจสอบนามสกุลและขนาดไฟล์ก่อนบันทึกลงฐานข้อมูล
UserSettingsForm	ฟอร์มสำหรับจัดการข้อมูลการตั้งค่าการเรียน เพื่อให้ผู้ใช้สามารถบันทึกหรือแก้ไขช่วงเวลาและรูปแบบการเรียนที่ต้องการได้
CustomUserCreationForm	ฟอร์มสำหรับกระบวนการสมัครสมาชิก โดยปรับปรุงให้รองรับโครงสร้างข้อมูลผู้ใช้แบบ CustomUser ของระบบ

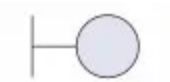
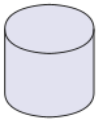
CustomAuthenticationForm	ฟอร์มสำหรับปรับแต่ง widget หน้า Login
UserUpdateForm	ฟอร์มสำหรับแก้ไขหรืออัปเดตข้อมูลส่วนตัว
FeedbackForm	ฟอร์มสำหรับส่งข้อมูล Feedback
AIService	คลาสให้บริการ ที่รวบรวมฟังก์ชันสำหรับเชื่อมต่อกับ Generative AI ทำหน้าที่ส่งข้อมูลหัวข้อเรียนไปประมวลผลเพื่อขอคำสรุป Summary และขอชุดคำถามแบบทดสอบ Quiz Generation
GoogleCalendarService	คลาสให้บริการ ที่จัดการตรรกะการเชื่อมต่อกับ Google Calendar API ทำหน้าที่ตรวจสอบความถูกต้องของ Token, และการแปลงข้อมูลตารางเรียน StudySession ให้เป็น Event เพื่อบันทึกลงปฏิทิน
AuthViews	กลุ่มฟังก์ชันสำหรับจัดการการยืนยันตัวตนของผู้ใช้ Authentication ทำหน้าที่ตรวจสอบข้อมูลการสมัครสมาชิก Register, การตรวจสอบชื่อผู้ใช้และรหัสผ่านเพื่อเข้าสู่ระบบ Login, การออกจากระบบ Logout และการแก้ไขข้อมูลส่วนตัว Edit Profile
SubjectViews	กลุ่มฟังก์ชันสำหรับจัดการข้อมูลรายวิชาและไฟล์เอกสาร ทำหน้าที่รับค่าจากฟอร์มเพื่อเพิ่มวิชาใหม่, ตรวจสอบไฟล์อัปโหลด, ลบไฟล์ที่ไม่ต้องการ และลบรายวิชาออกจากฐานข้อมูล
ScheduleViews	กลุ่มฟังก์ชันสำหรับจัดการตารางเวลาและการตั้งค่า ทำหน้าที่บันทึกช่วงเวลาว่าง Availability ที่ผู้ใช้ระบุ, รับค่าการตั้งค่าการเรียน UserSettings และเรียกใช้อัลกอริทึมเพื่อจัดตารางเรียนอัตโนมัติ
StudyViews	กลุ่มฟังก์ชันสำหรับจัดการหน้าหลัก Dashboard ทำหน้าที่แสดงผลตารางเรียนรายวัน, สถิติ, การจับเวลาเรียน, และเปลี่ยนสถานะการเรียนเมื่อผู้ใช้เรียนจบ Session
AI_FeatureViews	กลุ่มฟังก์ชันสำหรับฟีเจอร์อัจฉริยะ AI ทำหน้าที่รับส่งข้อมูลระหว่างหน้าเว็บกับ AIService เพื่อสร้างสรุปเนื้อหาบทเรียน Summary, แสดงประวัติการสรุป, สร้างโจทย์แบบทดสอบ Quiz, ตรวจคำตอบ และประมวลผลคะแนนสอบ
IntegrationViews	กลุ่มฟังก์ชันสำหรับเชื่อมต่อกับระบบภายนอก Google Calendar ทำหน้าที่เริ่มกระบวนการ OAuth 2.0 เพื่อขอสิทธิ์เข้าถึงปฏิทิน, รับค่า Callback จาก Google, และสั่งซิงค์ข้อมูลตารางเรียนลงในปฏิทิน
NotificationViews	กลุ่มฟังก์ชันสำหรับจัดการการกดอ่าน Notification
FeedbackViews	กลุ่มฟังก์ชันสำหรับจัดการการส่ง Feedback
LandingViews	กลุ่มฟังก์ชันสำหรับหน้า Landing Page

3.6 Sequence diagram

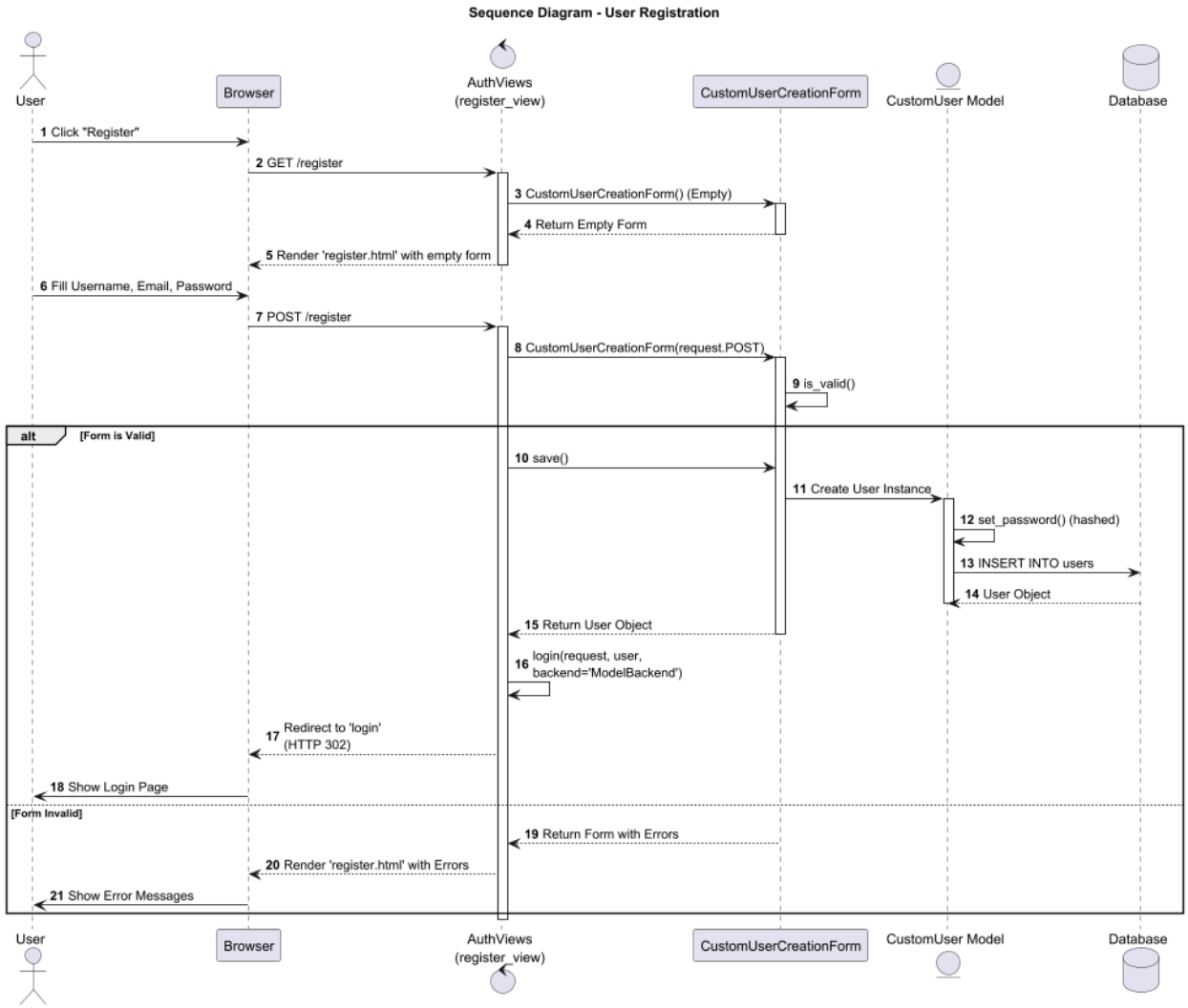
Sequence Diagram เป็น Diagram ที่แสดงขั้นตอนการทำงานของแต่ละ Use Case ระหว่าง Object ต่างๆ ที่ส่งข้อความถึงกันและกัน โดยจะช่วยให้มองเห็นการทำงานของระบบ ส่วนประกอบสัญลักษณ์ที่ใช้ในการเขียน Sequence Diagram

ตารางที่ 3.5 สัญลักษณ์ของ Sequence Diagram

สัญลักษณ์	การใช้งาน
	Class มีชื่อคลาสอยู่ภายในและแสดงคุณสมบัติของ Object ที่จะถูกสร้างจากคลาสนั้นๆ
	Lifeline เป็นการแสดงลำดับของการเรียก Method ระหว่าง Object ต่าง ๆ ในแต่ละเวลา และช่วยให้ผู้ใช้งานสามารถเข้าใจและวิเคราะห์การทำงานของระบบได้ง่ายขึ้น
	Focus of control หรือจุดควบคุมการใช้งานแสดงถึงการควบคุมของ Object ในระหว่างการทำงาน เป็นจุดที่ Object ควบคุมการส่ง Message ไปยัง Object อื่นๆ
	Message คือ ข้อความที่รับส่งระหว่าง Object สัญลักษณ์ แทนด้วยลูกศรและประกอบด้วย 2 ส่วนคือ ข้อมูล (Data) และฟังก์ชัน (Function)
	Return Message เป็นข้อมูลที่ส่งกลับหลังจากทำงานเสร็จ
	Self call เป็นการเรียกฟังชันก์การทำงานภายในตัวเอง
	สร้างกรอบการทำงานของโปรแกรม เพื่อให้รู้ขอบเขตของ การทำงานเช่น ลูป (loop) หรือ เงื่อนไขต่างๆ

	ใช้แทน Object ที่ทำหน้าที่ควบคุม Logic และ Flow การทำงานของระบบ
	แทนผู้ใช้งาน หรือระบบภายนอกที่เป็นผู้เริ่มต้นกระบวนการ โดยไม่ได้เป็นส่วนหนึ่งของระบบโดยตรง
	ใช้แทน Interface หรือจุดเชื่อมต่อระหว่างระบบกับสิ่งภายนอก เช่น หน้าจอ UI, Web Browser หรือ External API ที่ระบบต้องติดต่อด้วย
	แทนระบบจัดเก็บข้อมูล (Storage) ที่ทำหน้าที่รับและส่งข้อมูลกลับ

3.6.1 Sequence Diagram ของการสมัครสมาชิก

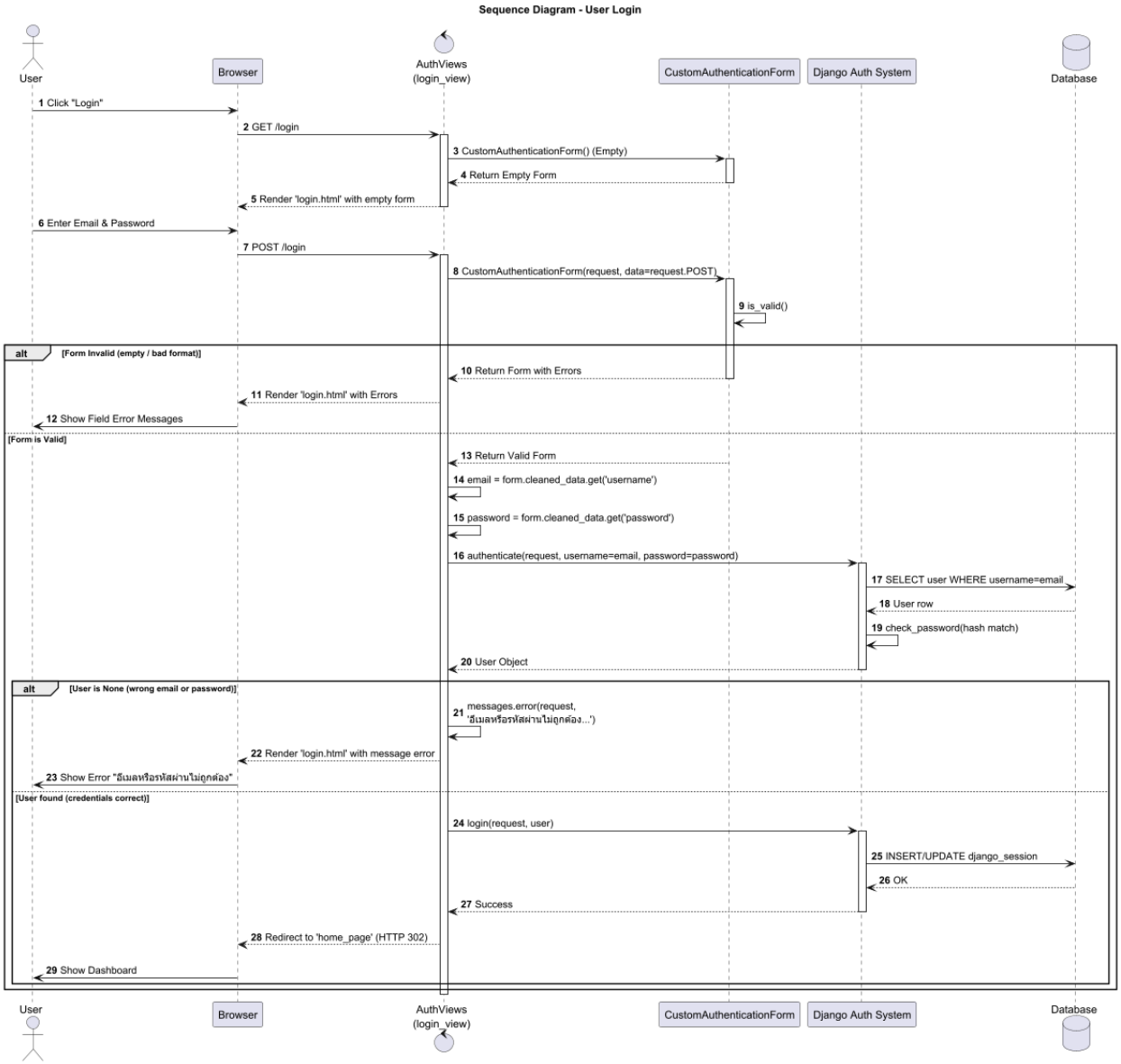


ภาพประกอบที่ 3.24 Sequence Diagram ของการสมัครสมาชิก

ดูรายละเอียดภาพเพิ่มเติม : Sequence_Diagram__User_Registration.png

จากภาพประกอบที่ 3.24 สามารถอธิบายภาพ Sequence Diagram ของการสมัครสมาชิกได้ดังต่อไปนี้ เมื่อผู้ใช้เข้าไปหน้าสมัครสมาชิก(Register) จากนั้นทำการกรอกฟอร์มสมัครสมาชิกจะมีการเรียกใช้คลาส register_view() จากไฟล์ view.py คลาสจะทำการตรวจสอบฟอร์มของผู้ใช้ว่าถูกต้องหรือไม่ ถ้าเกิดไม่ถูกต้อง จะให้ผู้ใช้กรอกฟอร์มใหม่ ถ้าเกิดฟอร์มถูกต้อง จะทำการบันทึกข้อมูลลงฐานข้อมูล และเปลี่ยนเส้นทางไปยังหน้าลงชื่อเข้าสู่ระบบ

3.6.2 Sequence Diagram ของการเข้าสู่ระบบ

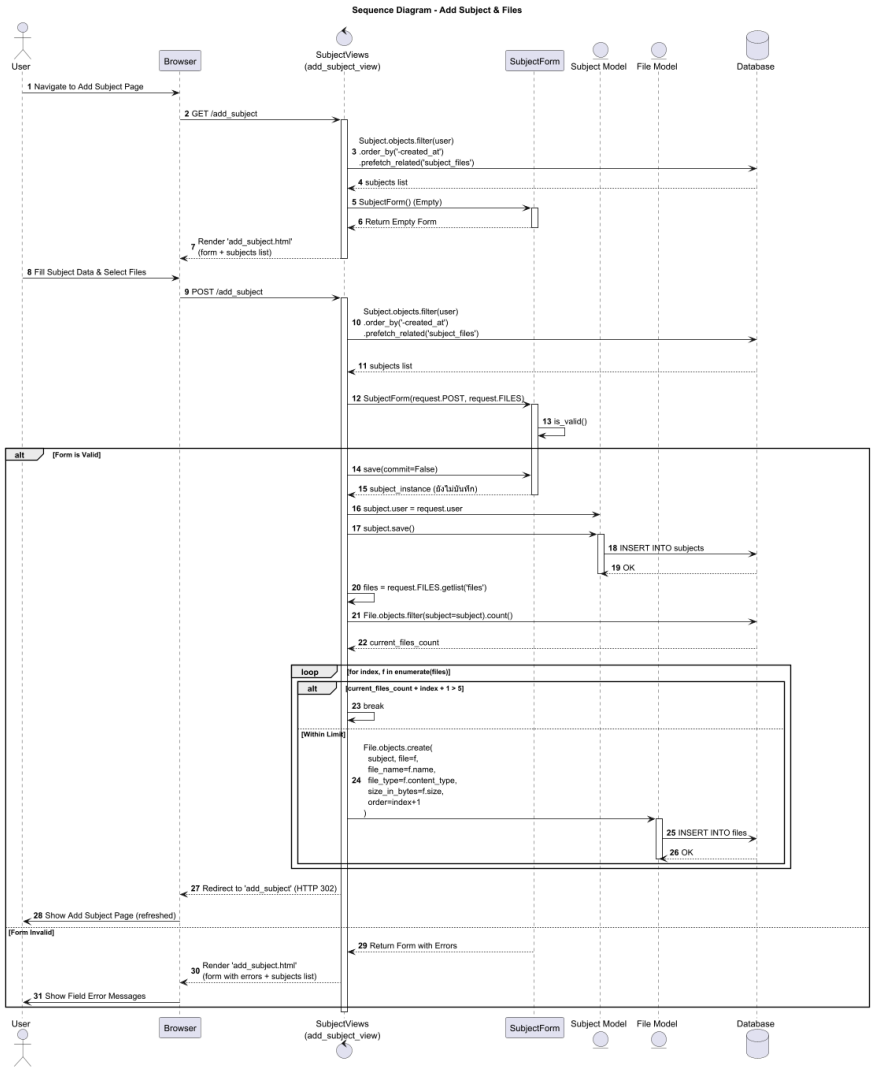


ภาพประกอบที่ 3.25 Sequence Diagram ของการเข้าสู่ระบบ

ดูรายละเอียดภาพเพิ่มเติม : Sequence_Diagram__User_Login.png

จากภาพประกอบที่ 3.25 สามารถอธิบายภาพ Sequence Diagram ของการเข้าสู่ระบบได้ดังต่อไปนี้ เมื่อผู้ใช้เข้าสู่หน้าลงชื่อเข้าสู่ระบบและทำการกรอกชื่อผู้ใช้ (Username) และรหัสผ่าน (Password) จะมีการเรียกใช้ฟังก์ชัน login_view() จากไฟล์ views.py ระบบจะทำการเรียกใช้ AuthenticationForm เพื่อตรวจสอบความถูกต้องของบัญชีผู้ใช้กับฐานข้อมูล หากข้อมูลไม่ถูกต้อง ระบบจะแสดงข้อความแจ้งเตือนให้ผู้ใช้กรอกใหม่ แต่หากข้อมูลถูกต้อง ระบบจะทำการสร้างเซสชัน สำหรับผู้ใช้นั้น และเปลี่ยนเส้นทางไปยังหน้าหลัก

3.6.3 Sequence Diagram ของการเพิ่มรายวิชาและไฟล์

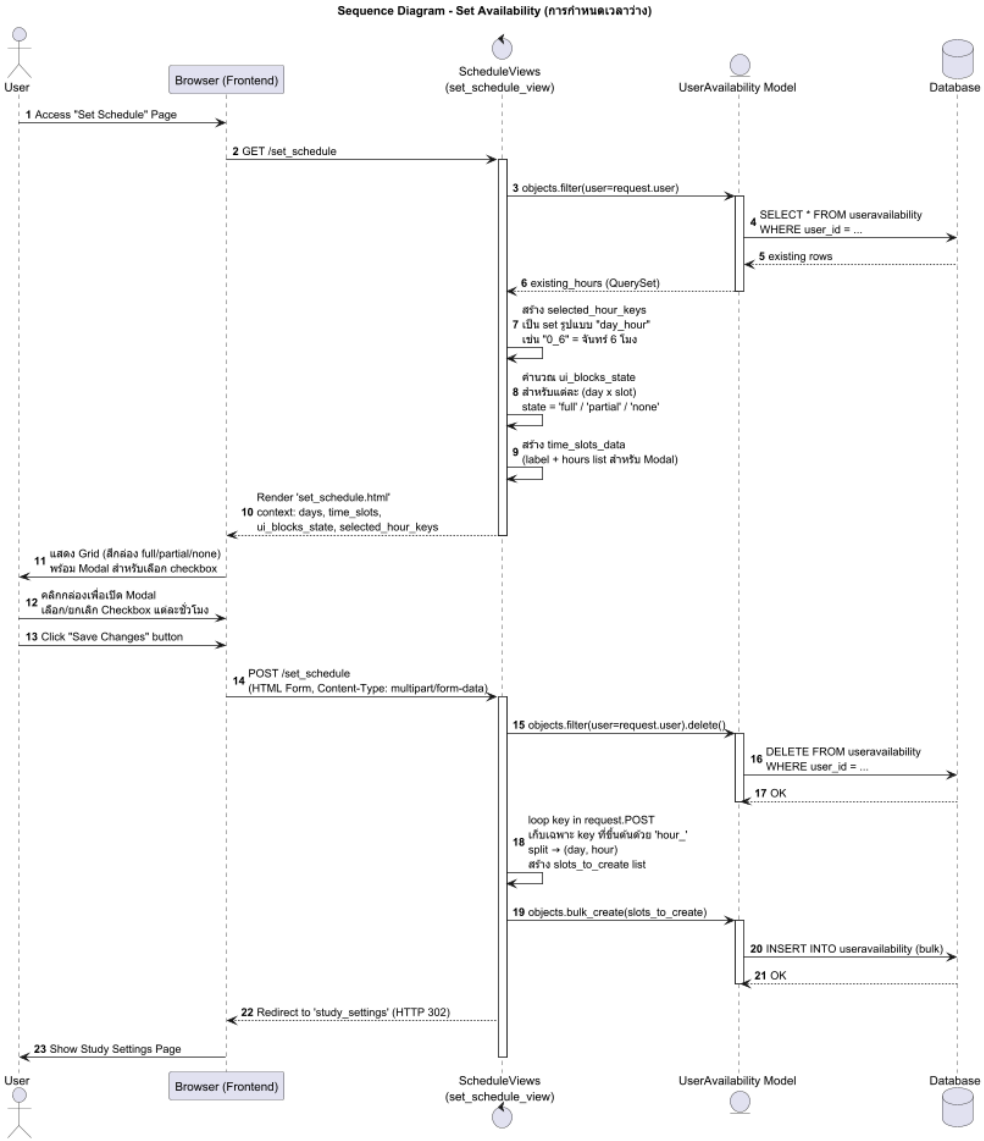


ภาพประกอบที่ 3.26 Sequence Diagram ของการเพิ่มรายวิชาและไฟล์

ดูรายละเอียดภาพเพิ่มเติม : Sequence_Diagram__Add_Subject__Files.png

จากภาพประกอบที่ 3.26 สามารถอธิบายภาพ Sequence Diagram ของการเพิ่มรายวิชาและไฟล์ได้ดังต่อไปนี้ เมื่อผู้ใช้เข้าสู่หน้าเพิ่มรายวิชา ทำการกรอกข้อมูลรายละเอียดวิชาและเลือกไฟล์เอกสารประกอบการเรียน จะมีการเรียกใช้ฟังก์ชัน add_subject_view() จากไฟล์ views.py โดยฟังก์ชันจะทำการตรวจสอบความถูกต้องผ่าน SubjectForm หากข้อมูลถูกต้อง ระบบจะบันทึกข้อมูลรายวิชาลงฐานข้อมูลตาราง Subject จากนั้นจะทำการวนซ้ำเพื่อบันทึกไฟล์ที่แนบมาลงในตาราง File และเมื่อเสร็จสิ้นจะเปลี่ยนเส้นทางกลับไปยังหน้ารายการวิชา

3.6.4 Sequence Diagram ของการกำหนดเวลาว่าง

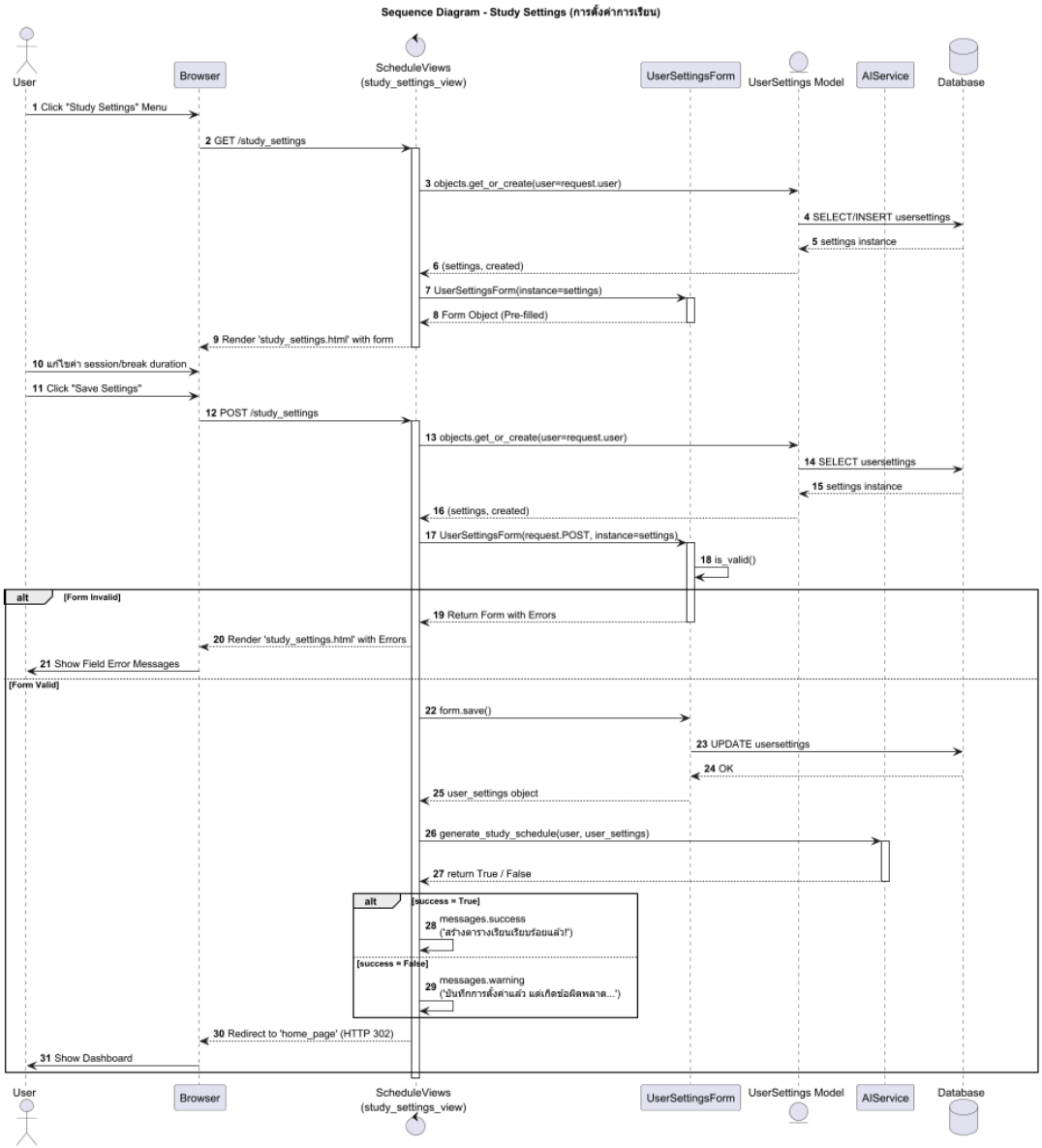


ภาพประกอบที่ 3.27 Sequence Diagram ของการกำหนดเวลาว่าง

ดูรายละเอียดภาพเพิ่มเติม : Sequence_Diagram__Set_Availability.png

จากภาพประกอบที่ 3.27 สามารถอธิบายภาพ Sequence Diagram ของการกำหนดเวลาว่างได้ดังต่อไปนี้ เมื่อผู้ใช้เลือกช่วงเวลาที่ว่างในตาราง Grid และกดปุ่มบันทึก ระบบจะส่งข้อมูลในรูปแบบ JSON ไปยังฟังก์ชัน set_schedule_view() จากไฟล์ views.py ระบบจะทำการลบข้อมูลเวลาว่างเดิมของผู้ใช้ทั้งหมดในตาราง UserAvailability ออกก่อน เพื่อป้องกันข้อมูลซ้ำซ้อน จากนั้นจะทำการสร้างและบันทึกข้อมูลช่วงเวลาใหม่ทั้งหมดลงในฐานข้อมูล และส่งสถานะความสำเร็จกลับมายังหน้าเว็บเพื่อแจ้งเตือนผู้ใช้

3.6.5 Sequence Diagram ของการตั้งค่าการเรียน

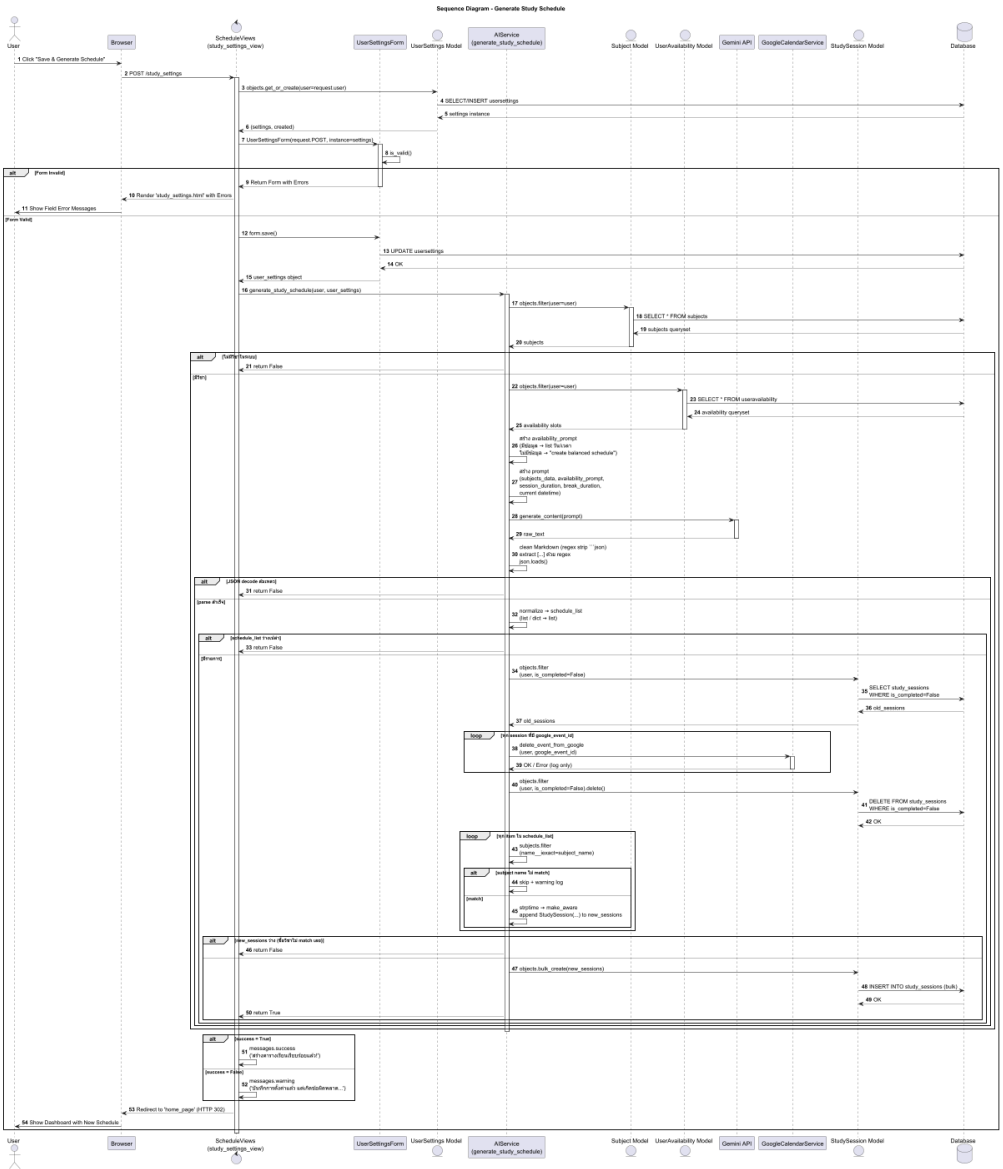


ภาพประกอบที่ 3.28 Sequence Diagram ของการตั้งค่าการเรียน

ดูรายละเอียดภาพเพิ่มเติม : Sequence_Diagram__Study_Settings__1.png

จากภาพประกอบที่ 3.28 สามารถอธิบายภาพ Sequence Diagram ของการตั้งค่าการเรียนได้ดังต่อไปนี้ เมื่อผู้ใช้ทำการเปลี่ยนแปลงการตั้งค่า และกดบันทึก จะมีการเรียกใช้ฟังก์ชัน study_settings_view() จากไฟล์ views.py เพื่อรับค่าผ่าน UserSettingsForm เมื่อตรวจสอบข้อมูลเรียบร้อยแล้ว ระบบจะบันทึกการตั้งค่าลงในตาราง UserSettings จากนั้นระบบจะเรียกฟังก์ชัน generate_study_schedule เพื่อประมวลผลจัดตารางเรียนใหม่ทันทีให้สอดคล้องกับค่าที่เปลี่ยนไป และเปลี่ยนเส้นทางไปยังหน้าหลัก

3.6.6 Sequence Diagram ของการจัดตารางเรียนอัตโนมัติ

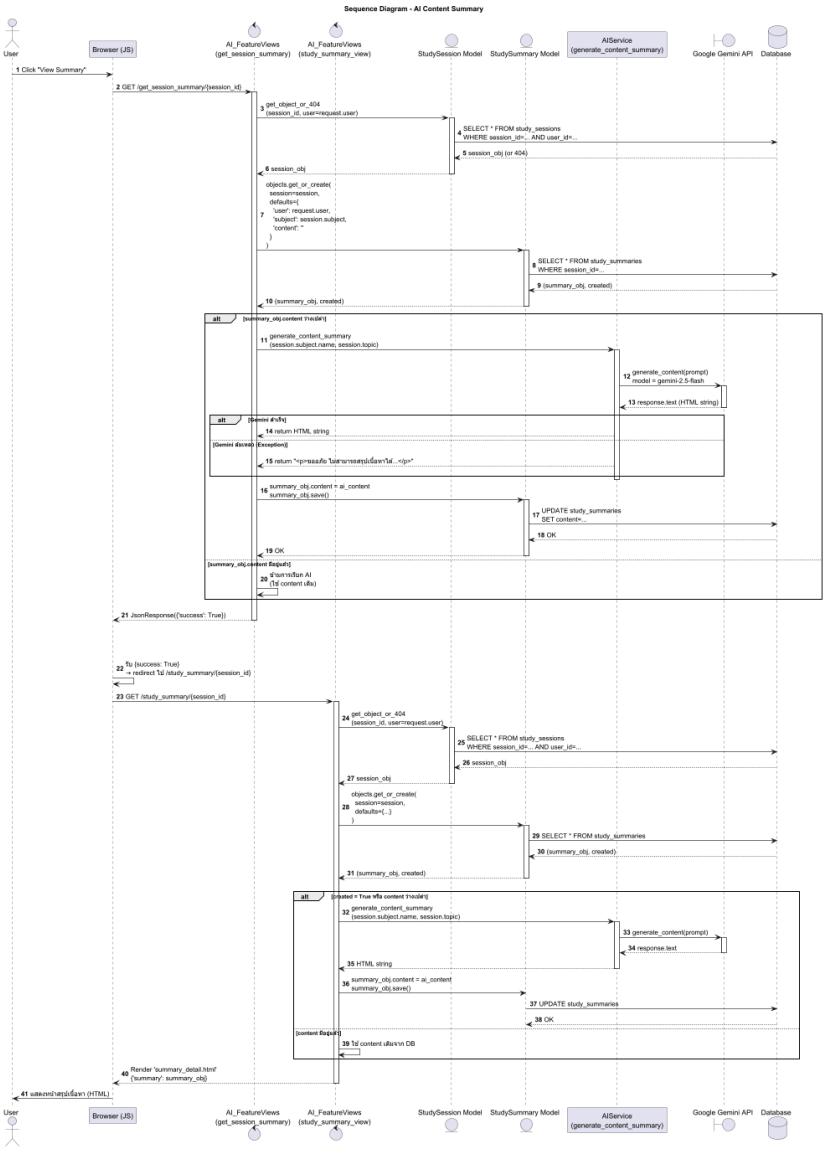


ภาพประกอบที่ 3.29 Sequence Diagram ของการจัดตารางเรียนอัตโนมัติ

ดูรายละเอียดภาพเพิ่มเติม : Sequence_Diagram__Generate_Study_Schedule_1.png

จากภาพประกอบที่ 3.29 สามารถอธิบายภาพ Sequence Diagram ของการจัดตารางเรียนอัตโนมัติได้ดังต่อไปนี้ เมื่อผู้ใช้ทำการบันทึกการตั้งค่าการเรียน ระบบจะเรียกใช้ฟังก์ชัน generate_study_schedule() ในไฟล์ views.py โดยขั้นตอนแรกระบบจะทำการดึงข้อมูลเวลาว่างของผู้ใช้จากตาราง UserAvailability และรายวิชาจากตาราง Subject มาประมวลผล จากนั้นระบบจะลบตารางเรียนเก่าที่ยังไม่เกิดขึ้นทิ้งเพื่อป้องกันความซ้ำซ้อน และเริ่มคำนวณหาช่วงเวลาที่เหมาะสมเพื่อสร้างตารางเรียนใหม่ บันทึกลงฐานข้อมูล เมื่อเสร็จสิ้นระบบจะเปลี่ยนเส้นทางไปยังหน้าหลักเพื่อแสดงตารางเรียนล่าสุด

3.6.7 Sequence Diagram ของการสรุปเนื้อหาด้วย AI

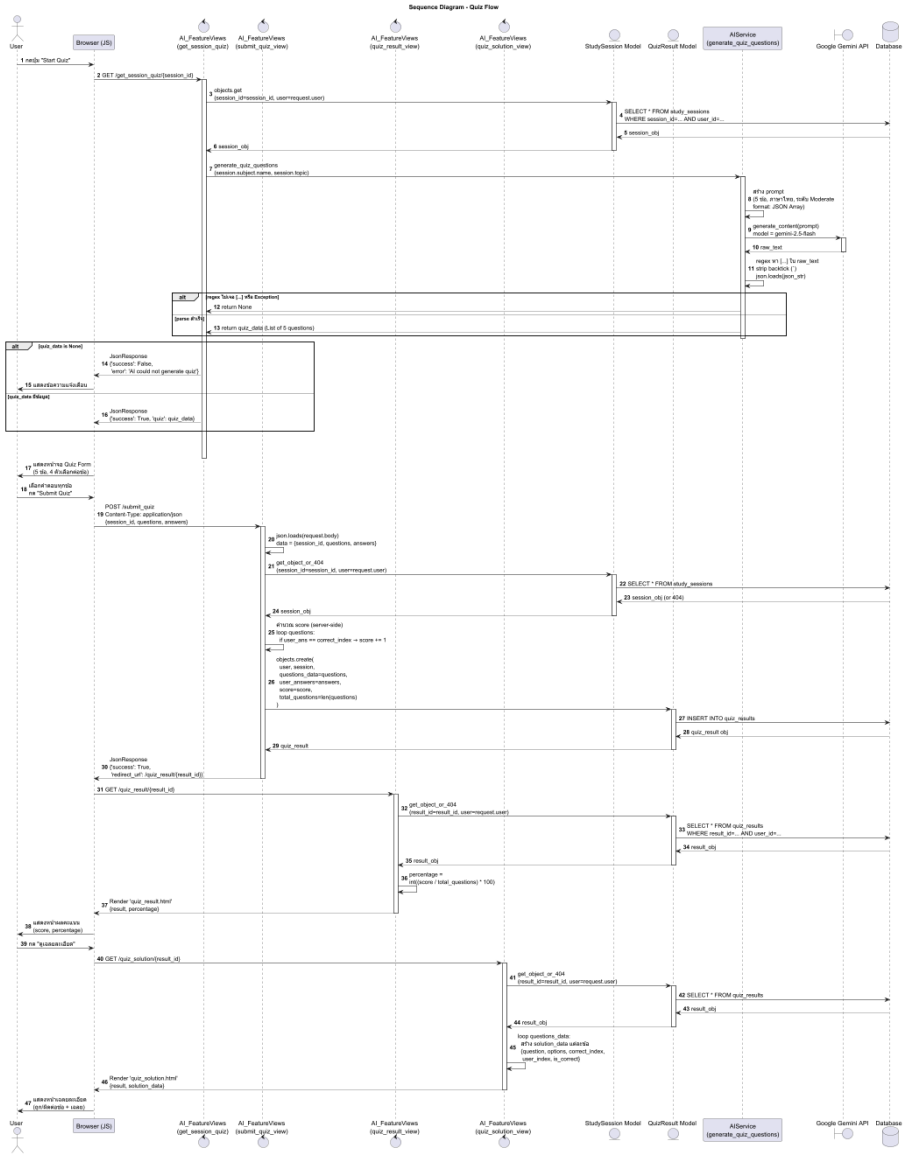


ภาพประกอบที่ 3.30 Sequence Diagram ของการสรุปเนื้อหาด้วย AI

ดูรายละเอียดภาพเพิ่มเติม : Sequence_Diagram__Content_Summary.png

จากภาพประกอบที่ 3.30 สามารถอธิบายภาพ Sequence Diagram ของการสรุปเนื้อหาด้วย AI ได้ดังต่อไปนี้ เมื่อผู้ใช้กดปุ่มเพื่อดูสรุปเนื้อหาของบทเรียน ระบบจะเรียกใช้งานฟังก์ชัน get_session_summary() หรือ study_summary_view() ในไฟล์ views.py ระบบจะตรวจสอบก่อนว่ามีสรุปของบทเรียนนี้อยู่ในฐานข้อมูล StudySummary แล้วหรือไม่ หากยังไม่มี ระบบจะเรียกใช้บริการ AIService เพื่อส่งหัวข้อและชื่อวิชาไปยัง Google Gemini API เมื่อ AI ประมวลผลและส่งข้อความสรุปกลับมา ระบบจะทำการบันทึกข้อมูลนั้นลงในฐานข้อมูลทันที และส่งข้อมูลกลับไปแสดงผลที่หน้าจอของผู้ใช้

3.6.8 Sequence Diagram ของการสร้างแบบทดสอบด้วย AI

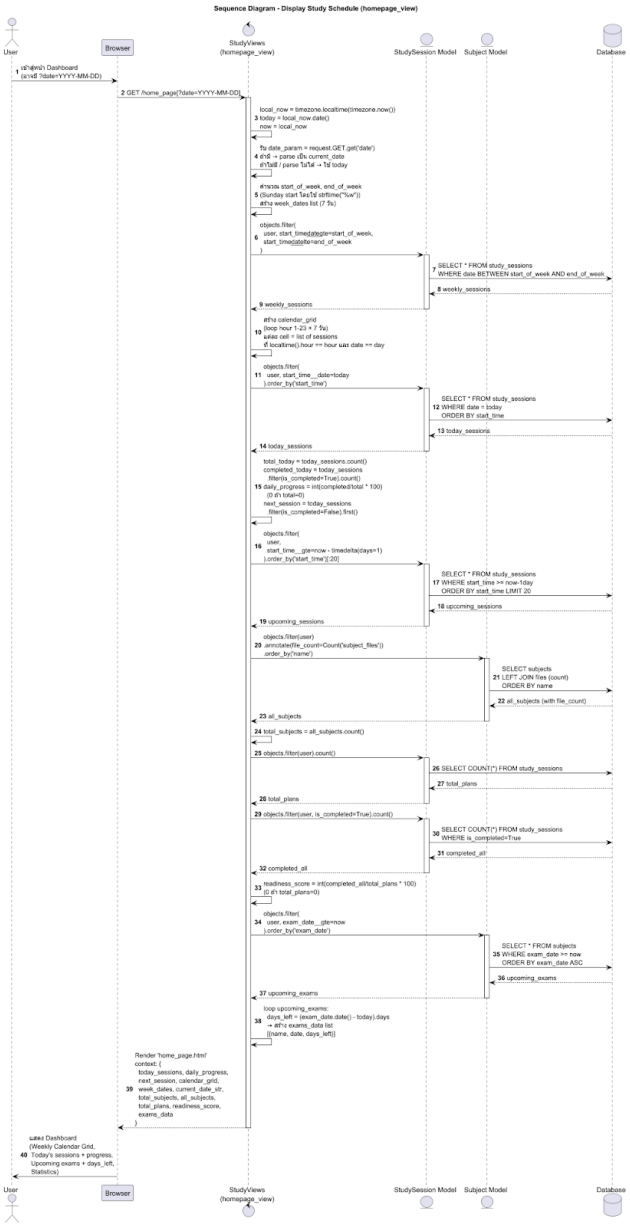


ภาพประกอบที่ 3.31 Sequence Diagram ของการสร้างแบบทดสอบด้วย AI

ดูรายละเอียดภาพเพิ่มเติม : Sequence_Diagram__Quiz_Flow.png

จากภาพประกอบที่ 3.31 สามารถอธิบายภาพ Sequence Diagram ของการสร้างแบบทดสอบด้วย AI ได้ดังต่อไปนี้ เมื่อผู้ใช้กดปุ่มเริ่มทำแบบทดสอบ จะมีการเรียกใช้ฟังก์ชัน get_session_quiz() จากไฟล์ views.py ระบบจะดึงข้อมูลหัวข้อการเรียนจากฐานข้อมูลและส่งต่อไปยัง AIService เพื่อเชื่อมต่อกับ Google Gemini API ให้ทำการสร้างโจทย์คำถาม เมื่อได้รับผลลัพธ์ ระบบจะส่งข้อมูลแบบทดสอบ (JSON) กลับไปยังหน้าเว็บเพื่อแสดงผลให้ผู้ใช้ทำแบบทดสอบ (โดยยังไม่บันทึกผลลงฐานข้อมูลจนกว่าจะส่งคำตอบ)

3.6.9 Sequence Diagram ของการแสดงตารางแผนการอ่าน (homepage)

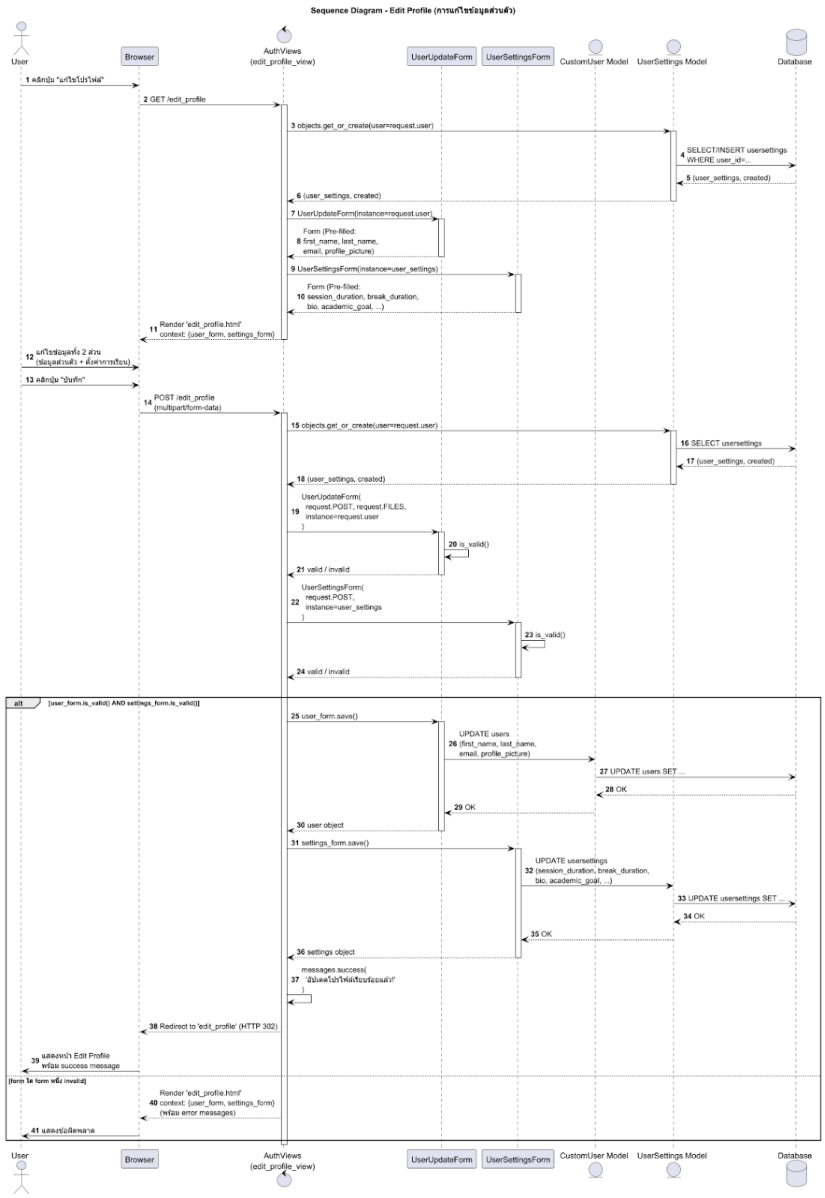


ภาพประกอบที่ 3.32 Sequence Diagram ของการแสดงตารางแผนการอ่าน (homepage)

ดูรายละเอียดภาพเพิ่มเติม : Sequence_Diagram__Display_Study_Schedule.png

จากภาพประกอบที่ 3.32 สามารถอธิบายภาพ Sequence Diagram ของการแสดงผลหน้าหลักได้ดังต่อไปนี้ เมื่อผู้ใช้เข้าสู่ระบบมายังหน้า Dashboard จะมีการเรียกใช้ฟังก์ชัน home_page() จากไฟล์ views.py ฟังก์ชันจะทำการดึงข้อมูลจากฐานข้อมูล 3 ส่วน ได้แก่ แผนการเรียนของวันที่ปัจจุบันจากตาราง StudySession, รายการสอบที่กำลังจะมาถึงจากตาราง Subject, และข้อมูลสถิติการเรียนทั้งหมด จากนั้นระบบจะนำข้อมูลทั้งหมดส่งไปแสดงผลที่หน้าจอเพื่อให้ผู้ใช้ทราบสถานะการเรียนปัจจุบัน

3.6.10 Sequence Diagram ของการแก้ไขข้อมูลส่วนตัว

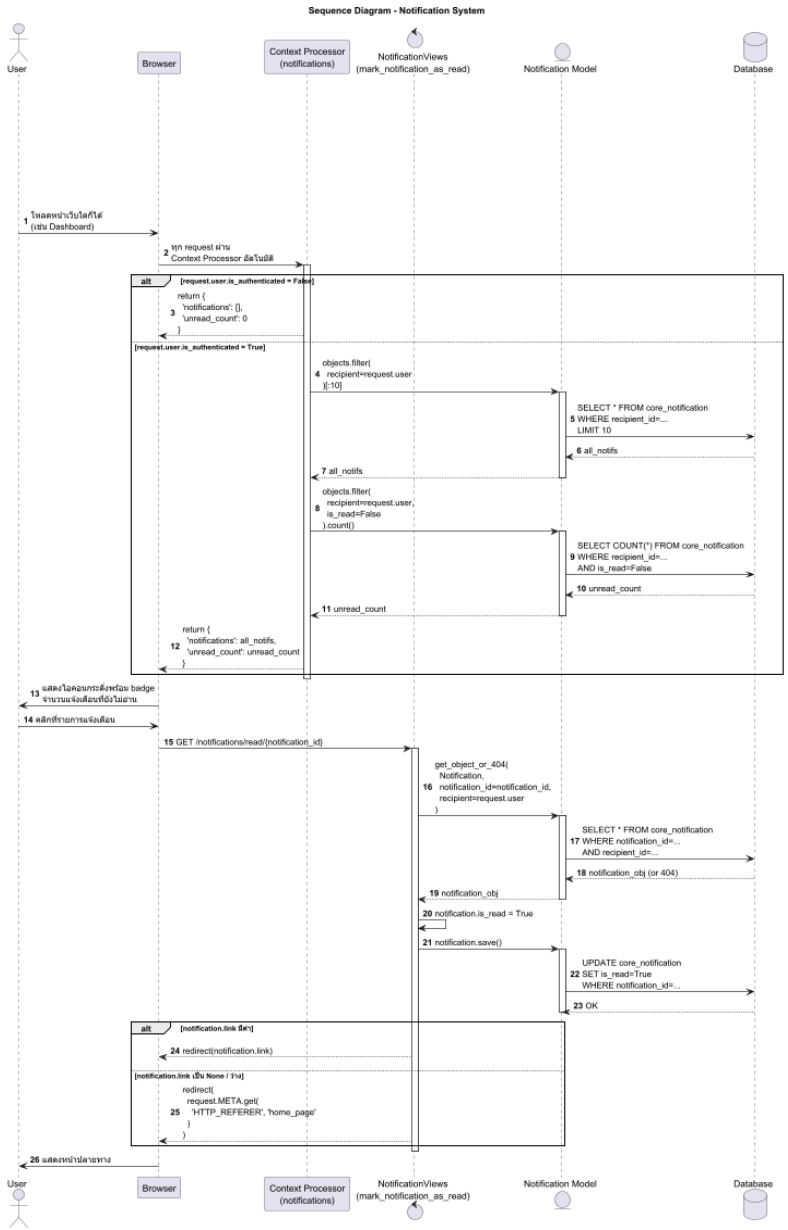


ภาพประกอบที่ 3.33 Sequence Diagram ของการแก้ไขข้อมูลส่วนตัว

ดูรายละเอียดภาพเพิ่มเติม : Sequence_Diagram__Edit_Profile.png

จากภาพประกอบที่ 3.33 สามารถอธิบายภาพ Sequence Diagram ของการแก้ไขข้อมูลส่วนตัวได้ดังต่อไปนี้ เมื่อผู้ใช้เข้าสู่หน้าแก้ไขโปรไฟล์ ระบบจะแสดงแบบฟอร์มพร้อมข้อมูลเดิม เมื่อผู้ใช้ทำการแก้ไขข้อมูลและกดบันทึก จะมีการเรียกใช้ฟังก์ชัน edit_profile_view() จากไฟล์ views.py โดยใช้ EditProfileForm ในการตรวจสอบความถูกต้องของข้อมูลและไฟล์รูปภาพ หากถูกต้อง ระบบจะทำการอัปเดตข้อมูลผู้ใช้ในตาราง CustomUser และเปลี่ยนเส้นทางกลับไปยังหน้าหลัก

3.6.11 Sequence Diagram ของระบบการแจ้งเตือน



ภาพประกอบที่ 3.34 Sequence Diagram ของระบบการแจ้งเตือน

ดูรายละเอียดภาพเพิ่มเติม : Sequence_Diagram__Notification_System.png

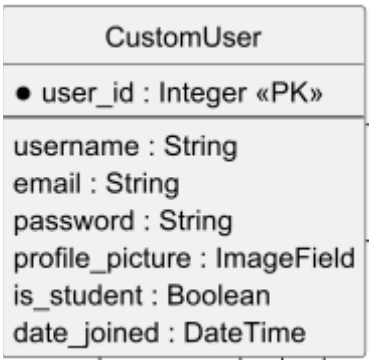
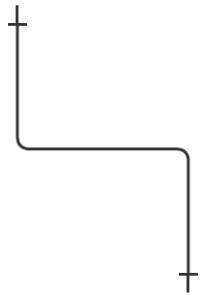
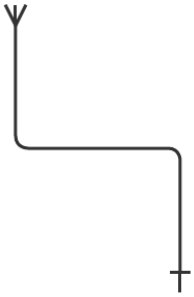
จากภาพประกอบที่ 3.34 สามารถอธิบายภาพ Sequence Diagram ของระบบแจ้งเตือนได้ดังต่อไปนี้ เมื่อระบบทำงานเสร็จสิ้น ระบบจะบันทึกข้อมูลลงตาราง Notification เมื่อผู้ใช้ใช้งานหน้าเว็บ Context Processor จะแสดงจำนวนการแจ้งเตือนที่ยังไม่อ่าน และเมื่อผู้ใช้คลิกที่การแจ้งเตือน จะมีการเรียกฟังก์ชัน

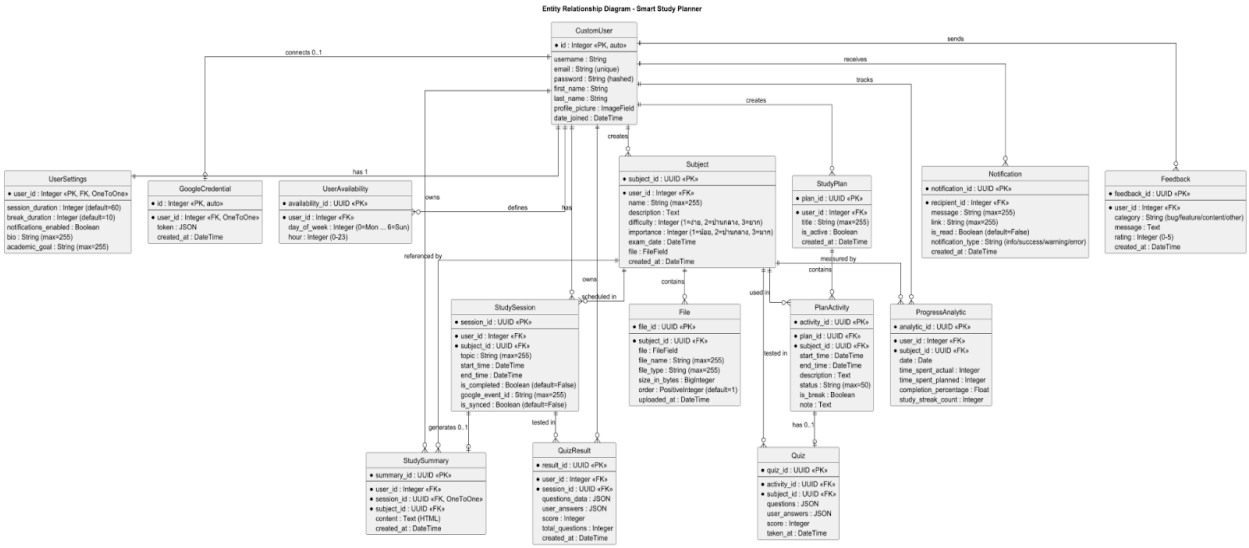
mark_notification_as_read() จากไฟล์ views.py เพื่อเปลี่ยนสถานะเป็น "อ่านแล้ว" ในฐานข้อมูล ก่อนจะเปลี่ยนเส้นทางไปยังหน้าเนื้อหาที่เกี่ยวข้อง

3.7 Entity Relationship Diagram

Entity Relationship Diagram หรือ ER Diagram เป็นแผนภาพที่ช่วยออกแบบฐานข้อมูล ทำหน้าที่เป็นแบบจำลองที่ใช่อธิบายโครงสร้างฐานข้อมูล ซึ่งเขียนออกมาในลักษณะของรูปภาพ การอธิบายโครงสร้างและความสัมพันธ์ของข้อมูล (Relationship) สัญลักษณ์ที่ใช้ในการเขียน ER Diagram แสดงในตารางที่ 3.

ตารางที่ 3.6 สัญลักษณ์ของ Entity Relationship Diagram

สัญลักษณ์	คำอธิบาย
	Entity เป็นชื่อของตารางฐานข้อมูล Entity อาจจะอยู่ในรูปธรรมที่จับต้องได้ เช่น อ๊อบเจกต์ บุคคล สถานที่ และอาจอยู่ในรูปแบบนามธรรมที่ไม่สามารถจับต้องได้
	เส้นแสดงความสัมพันธ์แบบ 1 to 1 คือ ข้อมูลหนึ่งข้อมูลของ Entity แรกมีความสัมพันธ์กับข้อมูลใน Entity ที่สองเพียงข้อมูลเดียว
	เส้นแสดงความสัมพันธ์แบบ 1 to many คือ ข้อมูลหนึ่งข้อมูลของ Entity แรกมีความสัมพันธ์กับข้อมูลใน Entity ที่สองหลายข้อมูล



ภาพประกอบที่ 3.24 Entity Relation Diagram แสดงความสัมพันธ์ของตารางในฐานข้อมูลของของระบบจัดตารางอัตโนมัติ Smart Study Planner

ดูรายละเอียดภาพเพิ่มเติม : Entity_Relationship_Diagram__Smart_Study_Planner.png

ตารางที่ 3.7 รายละเอียด Entity ที่เกี่ยวข้องกับระบบ

ชื่อตาราง	คำอธิบาย
CustomUser	ตารางเก็บข้อมูลผู้ใช้งานระบบ Users ทำหน้าที่เป็นศูนย์กลางเชื่อมโยงข้อมูลทั้งหมด โดยเก็บรายละเอียดส่วนตัว เช่น ชื่อผู้ใช้ (Username), รหัสผ่าน (Password), อีเมล (Email), รูปโปรไฟล์
Subject	ตารางเก็บข้อมูลรายวิชา Subjects โดยเก็บชื่อวิชา, ระดับความยาก, วันเวลาที่สอบ สำหรับแสดงในตารางเรียน
File	ตารางเก็บข้อมูลไฟล์เอกสารประกอบการเรียน Files ที่อัปโหลดเข้ามาในแต่ละรายวิชา เก็บชื่อไฟล์, ประเภทไฟล์, ขนาดไฟล์ และที่อยู่ไฟล์ (Path) บนเซิร์ฟเวอร์
UserAvailability	ตารางเก็บข้อมูลช่วงเวลาว่างของผู้ใช้ Availability ใช้สำหรับระบุวันและชั่วโมงที่ผู้ใช้สะดวกให้ระบบจัดตารางเรียนให้ เช่น วันจันทร์ เวลา 09:00-10:00 น.
UserSettings	ตารางเก็บข้อมูลการตั้งค่าส่วนบุคคล Settings เช่น ระยะเวลาอ่านต่อครั้ง และระยะเวลาพัก
StudySession	ตารางเก็บข้อมูลคาบเรียนหรือตารางอ่านหนังสือ Sessions ที่ระบบจัดให้อัตโนมัติหรือผู้ใช้สร้างเอง เก็บหัวข้อที่จะเรียน, เวลาเริ่มต้น-สิ้นสุด, สถานะการเรียน (เสร็จสิ้น/ยังไม่เริ่ม) และการเชื่อมต่อกับ Google Calendar
StudySummary	ตารางเก็บข้อมูลสรุปเนื้อหาบทเรียน Summaries ที่สร้างโดย AI เพื่อให้ผู้ใช้กลับมาทบทวนเนื้อหาสำคัญได้ภายหลัง
QuizResult	ตารางเก็บผลการทดสอบ Quiz Results เก็บข้อมูลชุดคำถามและตัวเลือกที่ AI สร้างขึ้นในรูปแบบ JSON, คำตอบที่ผู้ใช้เลือก, คะแนนที่ได้ และเปอร์เซ็นต์ความถูกต้อง
Notification	ตารางเก็บข้อมูลการแจ้งเตือน Notifications ภายในระบบ เช่น แจ้งเตือนเมื่อ AI สรุปเนื้อหาเสร็จ หรือแจ้งเตือนการสอบที่ใกล้จะถึง เก็บข้อความ, และสถานะการอ่าน
Feedback	ตารางเก็บข้อมูลข้อเสนอแนะ Feedbacks ที่ผู้ใช้ส่งถึงผู้ดูแลระบบ เก็บหัวข้อเรื่องและรายละเอียดข้อความ เพื่อนำไปปรับปรุงระบบต่อไป
GoogleCredential	เก็บ Token สำหรับเชื่อมต่อ Google Calendar OAuth2
QuizResult	เก็บเปอร์เซ็นต์ความถูกต้อง
Feedback	เก็บหัวข้อเรื่องและรายละเอียดข้อความ

UserSettings	ระยะเวลาอ่านต่อครั้ง และระยะเวลาพัก
StudySession	สถานะการเรียน (เสร็จสิ้น/ยังไม่เริ่ม)

บทที่ 4
การพัฒนาระบบ

ในบทนี้จะกล่าวถึงการสร้างระบบเว็บแอป ระบบจัดตารางอัตโนมัติ โดยนำผลที่ได้จากการวิเคราะห์และออกแบบระบบมาสร้างเป็นระบบงานซึ่งจะอธิบายถึงตัวอย่างและวิธีการเขียนโปรแกรมการทำงานของระบบในส่วนต่าง ๆ ดังต่อไปนี้

4.1 การพัฒนาระบบการลงทะเบียนและลงชื่อเข้าใช้

4.1.1 การพัฒนาระบบการบันทึกข้อมูลการลงทะเบียน(register_view)

```
1  class CustomUser(AbstractUser):
2      email = models.EmailField(unique=True)
3      profile_picture = models.ImageField(upload_to='profile_pics/', blank=True, null=True)
4      class Meta:
5          db_table = 'users
```

ภาพประกอบที่ 4.1 การพัฒนาระบบการบันทึกข้อมูลการลงทะเบียน (models.py)

จากภาพประกอบที่ 4.1 โครงสร้างของไฟล์ models.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: ประกาศฟังก์ชัน register_view รับค่า request เพื่อจัดการการลงทะเบียนสมาชิก
บรรทัดที่ 2: ตรวจสอบว่าการเรียกใช้งานเป็นแบบ POST (มีการส่งข้อมูลฟอร์มมา) หรือไม่
บรรทัดที่ 3: นำข้อมูลที่ส่งมาสร้างเป็น CustomUserCreationForm
บรรทัดที่ 4: ตรวจสอบความถูกต้องของข้อมูล (Validation) เช่น รหัสผ่านตรงกันหรือไม่
บรรทัดที่ 5: บันทึกข้อมูลผู้ใช้ใหม่ลงในฐานข้อมูล

4.1.2 การพัฒนาระบบฟอร์มสำหรับผู้ใช้

```
1  class CustomUserCreationForm(UserCreationForm):
2      class Meta(UserCreationForm.Meta):
3          model = CustomUser
4          fields = UserCreationForm.Meta.fields + ('email',)
```

ภาพประกอบที่ 4.2 การพัฒนาระบบฟอร์มสำหรับผู้ใช้ (forms.py)

จากภาพประกอบที่ 4.2 โครงสร้างของไฟล์ forms.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: ประกาศฟังก์ชัน register_view รับค่า request เพื่อจัดการการลงทะเบียนสมาชิก

บรรทัดที่ 2: ตรวจสอบว่าการเรียกใช้งานเป็นแบบ POST (มีการส่งข้อมูลฟอร์มมา) หรือไม่
บรรทัดที่ 3: นำข้อมูลที่ส่งมาสร้างเป็น CustomUserCreationForm
บรรทัดที่ 4: ตรวจสอบความถูกต้องของข้อมูล (Validation) เช่น รหัสผ่านตรงกันหรือไม่

4.1.3 การพัฒนาระบบการลงทะเบียน(register_view)

```
1   def register_view(request):
2       if request.method == 'POST':
3           form = CustomUserCreationForm(request.POST)
4           if form.is_valid():
5               user = form.save()
6               login(request, user, backend='django.contrib.auth.backends.ModelBackend')
7               return redirect('login')
8       else:
9           form = CustomUserCreationForm()
10      return render(request, 'core/register.html', {'form': form})
```

ภาพประกอบที่ 4.3 การพัฒนาระบบลงทะเบียน (views.py)

จากภาพประกอบที่ 4.3 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: ประกาศฟังก์ชัน register_view รับค่า request เพื่อจัดการการลงทะเบียนสมาชิก
บรรทัดที่ 2: ตรวจสอบว่าการเรียกใช้งานเป็นแบบ POST (มีการส่งข้อมูลฟอร์มมา) หรือไม่
บรรทัดที่ 3: นำข้อมูลที่ส่งมาสร้างเป็น CustomUserCreationForm
บรรทัดที่ 4: ตรวจสอบความถูกต้องของข้อมูล (Validation) เช่น รหัสผ่านตรงกันหรือไม่
บรรทัดที่ 5: บันทึกข้อมูลผู้ใช้ใหม่ลงในฐานข้อมูล
บรรทัดที่ 6: ทำการ Login ผู้ใช้เข้าสู่ระบบทันทีหลังจากสมัครเสร็จสิ้น
บรรทัดที่ 7: เปลี่ยนเส้นทาง (Redirect) ไปยังหน้า Login หรือหน้าแรก

4.1.4 การพัฒนาระบบการลงชื่อเข้าใช้

```
1   def login_view(request):
2       if request.method == 'POST':
3           form = CustomAuthenticationForm(request, data=request.POST)
4           if form.is_valid():
5               email = form.cleaned_data.get('username')
6               password = form.cleaned_data.get('password')
7
8               user = authenticate(request, username=email, password=password)
9
10              if user is not None:
11                  login(request, user)
12                  return redirect('home_page')
13              else:
14                  messages.error(request, 'อีเมลหรือรหัสผ่านไม่ถูกต้อง กรุณาลองใหม่อีกครั้ง')
15      else:
16          form = CustomAuthenticationForm()
17      return render(request, 'core/login.html', {'form': form})
```

ภาพประกอบที่ 4.4 การพัฒนาระบบการลงชื่อเข้าใช้ (views.py)

จากภาพประกอบที่ 4.4 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: ประกาศฟังก์ชัน login_view โดยรับค่า request เพื่อจัดการการเข้าสู่ระบบ
บรรทัดที่ 2: ตรวจสอบว่าเมธอดการเรียกใช้งานเป็น POST หรือไม่
บรรทัดที่ 3: สร้างอินสแตนซ์ CustomAuthenticationForm พร้อมข้อมูลที่ผู้ใช้กรอกมา
บรรทัดที่ 4: ตรวจสอบความถูกต้องของข้อมูล (Validation)
บรรทัดที่ 5-6: ดึงค่าอีเมลและรหัสผ่านจากฟอร์มที่ผ่านการตรวจสอบแล้ว
บรรทัดที่ 7: ใช้ฟังก์ชัน authenticate ตรวจสอบว่าอีเมลและรหัสผ่านถูกต้องหรือไม่
บรรทัดที่ 9-10: หากข้อมูลถูกต้อง ให้ทำการ login และเปลี่ยนเส้นทางไปยังหน้าแรก (home_page)
บรรทัดที่ 12: หากข้อมูลไม่ถูกต้อง ให้แสดงข้อความแจ้งเตือน Error
บรรทัดที่ 13-14: กรณีไม่ใช่ POST (เปิดหน้าเว็บครั้งแรก) ให้สร้างฟอร์มเปล่าเพื่อแสดงผล

4.2 การพัฒนาระบบตารางแผนการอ่านและกิจกรรมในแผนการอ่าน

4.2.1 การพัฒนาระบบการบันทึกข้อมูลตารางแผนการอ่าน

```
1  class StudyPlan(models.Model):
2      plan_id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
3      user = models.ForeignKey(CustomUser, on_delete=models.CASCADE)
4      title = models.CharField(max_length=255)
5      created_at = models.DateTimeField(auto_now_add=True)
6      is_active = models.BooleanField(default=True)
7
8      class Meta:
9          db_table = 'study_plans'
```

ภาพประกอบที่ 4.5 การพัฒนาระบบการบันทึกข้อมูลตารางแผนการอ่าน (models.py)

จากภาพประกอบที่ 4.5 โครงสร้างของไฟล์ models.py อธิบายการทำงานได้ดังนี้

บรรทัดที่ 1: ประกาศคลาส StudyPlan โดยสืบทอดคุณสมบัติจาก models.Model เพื่อสร้างเป็นตารางในฐานข้อมูล

บรรทัดที่ 2: กำหนดฟิลด์ plan_id ให้เป็น Primary Key ชนิด UUID ซึ่งจะถูกสร้างค่าสุ่มให้อัตโนมัติและไม่สามารถแก้ไขได้

บรรทัดที่ 3: กำหนดฟิลด์ user เป็น Foreign Key เชื่อมโยงกับตารางผู้ใช้งาน (CustomUser) โดยกำหนดให้ลบข้อมูลแผนการเรียนทิ้งหากผู้ใช้นั้นถูกลบ (CASCADE)

บรรทัดที่ 4: กำหนดฟิลด์ title สำหรับเก็บชื่อหัวข้อของแผนการเรียน โดยเก็บเป็นตัวอักษรความยาวไม่เกิน 255 ตัว

บรรทัดที่ 5: กำหนดฟิลด์ created_at สำหรับเก็บวันและเวลาที่สร้างข้อมูล โดยระบบจะบันทึกเวลาปัจจุบันให้อัตโนมัติเมื่อมีการสร้าง

บรรทัดที่ 6: กำหนดฟิลด์ is_active สำหรับเก็บสถานะของแผนการเรียน (เปิด/ปิด) โดยกำหนดค่าเริ่มต้นเป็น True (ใช้งานอยู่)

บรรทัดที่ 8: ประกาศคลาสย่อย Meta เพื่อกำหนดคุณสมบัติเพิ่มเติมให้กับโมเดลนี้

บรรทัดที่ 9: กำหนดชื่อตารางจริงที่จะถูกสร้างในฐานข้อมูลให้ชื่อว่า 'study_plans'

4.2.2 การพัฒนาระบบการบันทึกข้อมูลตารางกิจกรรมในแผนการอ่าน

```
1   class PlanActivity(models.Model):
2       activity_id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
3       plan = models.ForeignKey(StudyPlan, on_delete=models.CASCADE)
4       subject = models.ForeignKey('Subject', on_delete=models.CASCADE)
5       start_time = models.DateTimeField()
6       end_time = models.DateTimeField()
7       description = models.TextField()
8       status = models.CharField(max_length=50)
9       is_break = models.BooleanField(default=False)
10      note = models.TextField(blank=True, null=True)
11
12      class Meta:
13          db_table = 'plan_activities'
```

ภาพประกอบที่ 4.6 การพัฒนาระบบการบันทึกข้อมูลตารางกิจกรรมในแผนการอ่าน (models.py)

จากภาพประกอบที่ 4.6 โครงสร้างของไฟล์ models.py อธิบายการทำงานได้ดังนี้

บรรทัดที่ 1: ประกาศคลาส PlanActivity สืบทอดจาก models.Model เพื่อใช้สร้างตารางเก็บข้อมูลกิจกรรมย่อยในแผนการเรียน

บรรทัดที่ 2: กำหนดฟิลด์ activity_id เป็นคีย์หลัก (Primary Key) ชนิด UUID ที่สร้างค่าสุ่มให้อัตโนมัติ

บรรทัดที่ 3: กำหนดฟิลด์ plan เป็น Foreign Key เชื่อมโยงกับตาราง StudyPlan โดยหากแผนการเรียนหลักถูกลบ กิจกรรมย่อยนี้จะถูกลบไปด้วย (CASCADE)

บรรทัดที่ 4: กำหนดฟิลด์ subject เป็น Foreign Key เชื่อมโยงกับตารางรายวิชา (Subject) เพื่อระบุว่ากิจกรรมนี้เป็นของวิชาอะไร

บรรทัดที่ 5: กำหนดฟิลด์ start_time สำหรับเก็บวันและเวลาเริ่มต้นของกิจกรรม

บรรทัดที่ 6: กำหนดฟิลด์ end_time สำหรับเก็บวันและเวลาสิ้นสุดของกิจกรรม

บรรทัดที่ 7: กำหนดฟิลด์ description สำหรับเก็บรายละเอียดของกิจกรรม โดยใช้ TextField เพื่อรองรับข้อความยาว

บรรทัดที่ 8: กำหนดฟิลด์ status สำหรับเก็บสถานะของกิจกรรม (เช่น ยังไม่เริ่ม, กำลังทำ, เสร็จสิ้น) ความยาวไม่เกิน 50 ตัวอักษร

บรรทัดที่ 9: กำหนดฟิลด์ is_break เป็นชนิด Boolean เพื่อระบุว่าช่วงเวลานี้เป็นเวลาพักเบรกหรือไม่ (ค่าเริ่มต้นคือเท็จ)

บรรทัดที่ 10: กำหนดฟิลด์ note สำหรับเก็บหมายเหตุเพิ่มเติม โดยอนุญาตให้เป็นค่าว่างได้ (null=True)

บรรทัดที่ 12: ประกาศคลาสย่อย Meta เพื่อกำหนดคุณสมบัติเพิ่มเติมให้กับโมเดล

บรรทัดที่ 13: กำหนดชื่อตารางจริงที่จะถูกสร้างในฐานข้อมูลให้ชื่อว่า 'plan_activities'

4.2.3 การพัฒนาระบบหน้าแรก

```
@login_required
def homepage_view(request):
    user = request.user
    local_now = timezone.localtime(timezone.now())
    today = local_now.date()
    now = local_now

    date_param = request.GET.get('date')
    if date_param:
        try:
            current_date = datetime.strptime(date_param, '%Y-%m-%d').date()
        except ValueError:
            current_date = today
    else:
        current_date = today

    start_of_week = current_date - timedelta(days=int(current_date.strftime("%w")))
    end_of_week = start_of_week + timedelta(days=6)

    week_dates = []
    for i in range(7):
        day = start_of_week + timedelta(days=i)
        week_dates.append(day)

    weekly_sessions = StudySession.objects.filter(
        user=user,
        start_time__date__gte=start_of_week,
        start_time__date__lte=end_of_week
    )

    hours_range = range(1, 24)
    calendar_grid = []

    for hour in hours_range:
        row = {'hour': f"{hour:02d}:00", 'days': []}
        for day in week_dates:
            sessions_in_slot = []
            for s in weekly_sessions:
                local_start = timezone.localtime(s.start_time)
                if local_start.date() == day and local_start.hour == hour:
                    sessions_in_slot.append(s)

```

```
43              row['days'].append({
44                  'date': day,
45                  'sessions': sessions_in_slot
46              })
47          calendar_grid.append(row)
48
49      today_sessions = StudySession.objects.filter(user=user,
50  start_time__date=today).order_by('start_time')
51      total_today = today_sessions.count()
52      completed_today = today_sessions.filter(is_completed=True).count()
53
54      daily_progress = int((completed_today / total_today) * 100) if total_today > 0 else 0
55
56      next_session = today_sessions.filter(is_completed=False).first()
57
58      upcoming_sessions = StudySession.objects.filter(
59          user=user,
60          start_time__gte=now - timedelta(days=1)
61      ).order_by('start_time')[:20]
62
63      all_subjects = Subject.objects.filter(user=user).annotate(
64          file_count=Count('subject_files')
65      ).order_by('name')
66
67      total_subjects = all_subjects.count()
68      total_plans = StudySession.objects.filter(user=user).count()
69
70      completed_all = StudySession.objects.filter(user=user, is_completed=True).count()
71      readiness_score = int((completed_all / total_plans) * 100) if total_plans > 0 else 0
72
73      upcoming_exams = Subject.objects.filter(user=user,
74  exam_date__gte=now).order_by('exam_date')
75      exams_data = []
76      for subject in upcoming_exams:
77          days_left = (subject.exam_date.date() - today).days
78          exams_data.append({'name': subject.name, 'date': subject.exam_date, 'days_left':
79  days_left,})
80
81      context = {
82          'user': user,
83          'today_sessions': today_sessions,
84          'daily_progress': daily_progress,
85          'next_session': next_session,
86          'calendar_grid': calendar_grid,
```

```
87          'week_dates': week_dates,
88          'current_date_str': current_date.strftime('%Y-%m-%d'),
89          'total_subjects': total_subjects,
90          'all_subjects': all_subjects,
91          'total_plans': total_plans,
92          'readiness_score': readiness_score,
93          'exams_data': exams_data,
94      }
95      return render(request, 'core/home_page.html', context)
```

ภาพประกอบที่ 4.7 การพัฒนาระบบหน้าแรก (homepage_view)

จากภาพประกอบที่ 4.7 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้

บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อบังคับว่าผู้ใช้ต้องล็อกอินก่อนถึงจะเข้าถึงฟังก์ชันนี้ได้

บรรทัดที่ 2: ประกาศฟังก์ชัน homepage_view เพื่อประมวลผลข้อมูลสำหรับหน้าแรก (Dashboard)

บรรทัดที่ 3-6: ดึงข้อมูลผู้ใช้ปัจจุบัน และดึงวันเวลาปัจจุบัน (local_now) ตาม Timezone ของเครื่องเพื่อความแม่นยำ

บรรทัดที่ 8-15: ตรวจสอบว่ามีการส่งค่าวันที่ (date) มาทาง URL หรือไม่ หากมีให้ใช้วันที่นั้น แต่หากไม่มีหรือรูปแบบผิดให้ใช้วันที่ปัจจุบัน

บรรทัดที่ 17-22: คำนวณหาวันเริ่มต้นสัปดาห์ (start_of_week) และวนลูปสร้างรายการวันที่ทั้ง 7 วัน เพื่อเตรียมแสดงผลปฏิทินรายสัปดาห์

บรรทัดที่ 24-28: ดึงข้อมูลตารางเรียน (StudySession) ของผู้ใช้นั้นๆ เฉพาะช่วงเวลาในสัปดาห์ที่กำลังแสดงผล

บรรทัดที่ 30-41: สร้างโครงสร้างข้อมูลปฏิทิน (calendar_grid) โดยวนลูปตามช่วงเวลา (ชั่วโมง) และวัน เพื่อนำข้อมูลการเรียนไปหยอดลงในช่องตารางให้ถูกต้อง

บรรทัดที่ 43-46: ดึงข้อมูลการเรียนของ "วันนี้" และคำนวณเปอร์เซ็นต์ความคืบหน้า (daily_progress) ว่าเรียนไปแล้วกี่เปอร์เซ็นต์

บรรทัดที่ 53-55: ดึงข้อมูลรายวิชาทั้งหมด (Subject) และนับจำนวนไฟล์ประกอบการเรียนของแต่ละวิชา

บรรทัดที่ 57-60: คำนวณคะแนนความพร้อมรวม (readiness_score) จากกิจกรรมการเรียนทั้งหมดที่ทำสำเร็จหารด้วยจำนวนกิจกรรมทั้งหมด

บรรทัดที่ 62-65: ดึงข้อมูลวิชาที่มีสอบในอนาคต และคำนวณ "จำนวนวันที่เหลือ" (days_left) เพื่อใช้แจ้งเตือนผู้ใช้

บรรทัดที่ 67-79: จัดเตรียมข้อมูลทั้งหมด (User, ตารางเรียน, สถิติ, วันสอบ) ลงในตัวแปร context

บรรทัดที่ 80: ส่งข้อมูล context ไปแสดงผลที่หน้าจอ core/home_page.html

4.3 การพัฒนาระบบเพิ่มวิชา

4.3.1 การพัฒนาระบบการบันทึกข้อมูลรายวิชา

```
1   class Subject(models.Model):
2       DIFFICULTY_CHOICES = [
3           (1, 'ง่าย'), (2, 'ปานกลาง'), (3, 'ยาก'),
4       ]
5       IMPORTANCE_CHOICES = [
6           (1, 'น้อย'), (2, 'ปานกลาง'), (3, 'มาก'),
7       ]
8       subject_id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
9       user = models.ForeignKey(CustomUser, on_delete=models.CASCADE)
10      name = models.CharField(max_length=255)
11      description = models.TextField(blank=True, null=True)
12      difficulty = models.IntegerField(choices=DIFFICULTY_CHOICES, default=2)
13      importance = models.IntegerField(choices=IMPORTANCE_CHOICES, default=2)
14      exam_date = models.DateTimeField()
15      file = models.FileField(upload_to='subject_files/', blank=True, null=True)
16      created_at = models.DateTimeField(auto_now_add=True)
17
18      class Meta:
19          db_table = 'subjects'
20
21      def __str__(self):
22          return self.name
```

ภาพประกอบที่ 4.8 การพัฒนาระบบการบันทึกข้อมูลรายวิชา (models.py)

จากภาพประกอบที่ 4.8 โครงสร้างของไฟล์ models.py อธิบายการทำงานได้ดังนี้

บรรทัดที่ 1: ประกาศคลาส Subject สืบทอดจาก models.Model เพื่อสร้างตารางเก็บข้อมูลรายวิชา

บรรทัดที่ 2-4: กำหนดตัวแปร DIFFICULTY_CHOICES เพื่อเก็บตัวเลือกความยากง่ายของวิชา (1=ง่าย, 2=ปานกลาง, 3=ยาก) สำหรับนำไปใช้ใน Dropdown list

บรรทัดที่ 5-7: กำหนดตัวแปร IMPORTANCE_CHOICES เพื่อเก็บตัวเลือกความสำคัญของวิชา (1=น้อย, 2=ปานกลาง, 3=มาก)

บรรทัดที่ 8: กำหนดฟิลด์ subject_id เป็นคีย์หลัก (Primary Key) ชนิด UUID ซึ่งระบบจะสุ่มสร้างให้อัตโนมัติและแก้ไขไม่ได้

บรรทัดที่ 9: กำหนดฟิลด์ user เป็น Foreign Key เชื่อมโยงกับตารางผู้ใช้ (CustomUser) หากผู้ใช้ถูกลบ ข้อมูลวิชานี้จะถูกลบไปด้วย (CASCADE)

บรรทัดที่ 10: กำหนดฟิลด์ name สำหรับเก็บชื่อวิชา โดยรองรับตัวอักษรได้สูงสุด 255 ตัว

บรรทัดที่ 11: กำหนดฟิลด์ description สำหรับเก็บคำอธิบายรายวิชา โดยอนุญาตให้เป็นค่าว่างได้ (null=True)

บรรทัดที่ 12-13: กำหนดฟิลด์ difficulty และ importance เก็บค่าเป็นตัวเลข (Integer) โดยอ้างอิงจากตัวเลือกที่กำหนดไว้ข้างต้น และตั้งค่าเริ่มต้นเป็น "ปานกลาง"
บรรทัดที่ 14: กำหนดฟิลด์ exam_date สำหรับเก็บวันและเวลาที่จะสอบ
บรรทัดที่ 15: กำหนดฟิลด์ file สำหรับเก็บไฟล์เอกสารประกอบการเรียน โดยไฟล์จะถูกบันทึกไว้ในโฟลเดอร์ subject_files
บรรทัดที่ 16: กำหนดฟิลด์ created_at เก็บเวลาที่บันทึกข้อมูลนี้โดยอัตโนมัติ
บรรทัดที่ 18-19: ประกาศคลาสย่อย Meta และกำหนดชื่อตารางในฐานข้อมูลเป็น 'subjects'
บรรทัดที่ 21-22: กำหนดฟังก์ชัน _str_ ให้คืนค่าชื่อวิชา (self.name) เมื่อมีการอ้างอิงถึงออบเจกต์นี้

4.3.2 การพัฒนาระบบการบันทึกข้อมูลไฟล์เอกสาร

```
1   class File(models.Model):
2       file_id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
3       subject = models.ForeignKey('Subject', on_delete=models.CASCADE,
4   related_name='subject_files')
5
6       file = models.FileField(upload_to='subject_materials/')
7
8       file_name = models.CharField(max_length=255)
9       file_type = models.CharField(max_length=255)
10      size_in_bytes = models.BigIntegerField()
11      order = models.PositiveIntegerField(default=1)
12      uploaded_at = models.DateTimeField(auto_now_add=True)
13
14      class Meta:
15          db_table = 'files'
16          ordering = ['order']
```

ภาพประกอบที่ 4.9 การพัฒนาระบบการพัฒนาระบบการบันทึกข้อมูลไฟล์เอกสาร (models.py)

จากภาพประกอบที่ 4.9 โครงสร้างของไฟล์ models.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: ประกาศคลาส File สืบทอดจาก models.Model เพื่อสร้างตารางสำหรับเก็บข้อมูลไฟล์เอกสาร
บรรทัดที่ 2: กำหนดฟิลด์ file_id เป็นคีย์หลัก (Primary Key) ชนิด UUID ที่สร้างค่าสุ่มให้อัตโนมัติ
บรรทัดที่ 3: กำหนดฟิลด์ subject เป็น Foreign Key เชื่อมโยงกับตารางรายวิชา (Subject) โดยกำหนด related_name='subject_files' เพื่อให้สามารถดึงข้อมูลไฟล์ทั้งหมดของวิชานั้นๆ ได้ง่าย
บรรทัดที่ 5: กำหนดฟิลด์ file สำหรับเก็บไฟล์จริง โดยระบุตำแหน่งจัดเก็บไว้ที่โฟลเดอร์ subject_materials
บรรทัดที่ 7-9: กำหนดฟิลด์สำหรับเก็บรายละเอียดของไฟล์ ได้แก่ file_name (ชื่อไฟล์), file_type (ชนิดไฟล์), และ size_in_bytes (ขนาดไฟล์)
บรรทัดที่ 11: กำหนดฟิลด์ order สำหรับเก็บลำดับของไฟล์ เพื่อใช้ในการเรียงลำดับการแสดงผลหน้าเว็บ
บรรทัดที่ 12: กำหนดฟิลด์ uploaded_at เก็บวันและเวลาที่อัปโหลดไฟล์โดยอัตโนมัติ

บรรทัดที่ 14-16: ประกาศคลาสย่อย Meta เพื่อกำหนดชื่อตารางในฐานข้อมูลเป็น 'files' และกำหนดให้เรียงลำดับข้อมูลตามฟิลด์ order

4.3.3 การพัฒนาระบบฟอร์มสำหรับการเพิ่มไฟล์

```
1   class SubjectForm(forms.ModelForm):
2       file = forms.FileField(
3           widget=MultipleFileInput(attrs={'class': 'form-control', 'multiple': True}),
4           label='ไฟล์ประกอบการเรียน (สูงสุด 5 ไฟล์)',
5           required=False
6       )
7
8       class Meta:
9           model = Subject
10          fields = ['name', 'difficulty', 'exam_date']
11          widgets = {
12              'exam_date': forms.DateTimeInput(
13                  attrs={'type': 'datetime-local', 'class': 'form-control'}
14              ),
15              'name': forms.TextInput(attrs={'class': 'form-control'}),
16              'difficulty': forms.Select(attrs={'class': 'form-control'}),
17          }
18
19      def clean_files(self):
20          file = self.files.getlist('files')
21
22          if len(file) > 5:
23              raise forms.ValidationError("อัปโหลดได้สูงสุด 5 ไฟล์ต่อวิชา")
24
25          valid_extensions = ['.pdf', '.docx', '.pptx', '.txt']
26          max_size = 10 * 1024 * 1024 # 10 MB
27
28          for f in file:
29              ext = os.path.splitext(f.name)[1].lower()
30              if ext not in valid_extensions:
31                  raise forms.ValidationError(f"ไม่รองรับไฟล์ {f.name} (รองรับเฉพาะ .pdf, .docx, .pptx,
32  .txt)")
33              if f.size > max_size:
34                  raise forms.ValidationError(f"ไฟล์ {f.name} มีขนาดใหญ่เกิน 10MB")
35          return file
```

ภาพประกอบที่ 4.10 การพัฒนาระบบฟอร์มสำหรับการเพิ่มไฟล์ (forms.py)

จากภาพประกอบที่ 4.10 โครงสร้างของไฟล์ forms.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: ประกาศคลาส SubjectForm สืบทอดจาก forms.ModelForm เพื่อสร้างฟอร์มสำหรับจัดการข้อมูลรายวิชาที่เชื่อมโยงกับฐานข้อมูล
บรรทัดที่ 2-6: กำหนดฟิลด์ file สำหรับอัปโหลดไฟล์เอกสาร โดยใช้ Widget แบบ MultipleFileInput (ที่สร้างขึ้นเอง) เพื่อให้ผู้ใช้สามารถเลือกไฟล์ได้หลายไฟล์พร้อมกัน และกำหนดให้ฟิลด์นี้ไม่จำเป็นต้องกรอก (required=False)
บรรทัดที่ 8-10: ประกาศคลาส Meta เพื่อเชื่อมโยงฟอร์มเข้ากับโมเดล Subject และระบุฟิลด์ที่จะใช้งาน ได้แก่ ชื่อวิชา (name), ความยาก (difficulty), และวันสอบ (exam_date)
บรรทัดที่ 11-15: กำหนด widgets เพื่อปรับแต่งคุณสมบัติของ HTML Input เช่น กำหนดให้ช่อง exam_date เป็นปฏิทินเลือกวันเวลา (type: datetime-local) และใส่คลาส form-control เพื่อจัดรูปแบบ CSS
บรรทัดที่ 17-18: ประกาศฟังก์ชัน clean_files เพื่อเขียนลอจิกตรวจสอบความถูกต้องของไฟล์ (Validation) และดึงรายการไฟล์ทั้งหมดที่ผู้ใช้อัปโหลดมาเก็บไว้ในตัวแปร
บรรทัดที่ 20-21: ตรวจสอบจำนวนไฟล์ หากมีการอัปโหลดเกิน 5 ไฟล์ จะทำการแจ้งเตือนข้อผิดพลาด (ValidationError)
บรรทัดที่ 23-24: กำหนดรายการนามสกุลไฟล์ที่รองรับ (เช่น .pdf, .docx) และกำหนดขนาดไฟล์สูงสุดไม่เกิน 10 MB
บรรทัดที่ 26-30: วนลูปตรวจสอบไฟล์ทีละไฟล์ หากพบไฟล์ที่มีนามสกุลไม่ถูกต้องหรือมีขนาดใหญ่เกินกำหนด ระบบจะแจ้งเตือนข้อผิดพลาดและปฏิเสธการบันทึกข้อมูลทันที

4.3.4 การพัฒนาระบบบบเพิ่มวิชา

```
1   @login_required
2   def add_subject_view(request):
3       subjects =
4   Subject.objects.filter(user=request.user).order_by('-created_at').prefetch_related('subject_files')
5
6       if request.method == 'POST':
7           form = SubjectForm(request.POST, request.FILES)
8           if form.is_valid():
9               subject = form.save(commit=False)
10              subject.user = request.user
11              subject.save()
12
13              files = request.FILES.getlist('files')
14
15              current_files_count = File.objects.filter(subject=subject).count()
16
17              for index, f in enumerate(files):
18                  if current_files_count + index + 1 > 5:
19                      break
```

```
20
21              File.objects.create(
22                  subject=subject,
23                  file=f,
24                  file_name=f.name,
25                  file_type=f.content_type,
26                  size_in_bytes=f.size,
27                  order=index + 1
28              )
29
30          return redirect('add_subject')
31      else:
32          form = SubjectForm()
33
34      context = {
35          'form': form,
36          'subjects': subjects
37      }
38      return render(request, 'core/add_subject.html', context)
```

ภาพประกอบที่ 4.11 การพัฒนาระบบเพิ่มวิชา (views.py)

จากภาพประกอบที่ 4.11 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อตรวจสอบสิทธิ์การเข้าใช้งาน โดยผู้ใช้ต้องล็อกอินก่อนจึงจะเข้าถึงหน้านี้ได้
บรรทัดที่ 2-3: ประกาศฟังก์ชัน add_subject_view และดึงข้อมูลรายวิชา (subjects) ทั้งหมดของผู้ใช้นั้นๆ โดยเรียงลำดับจากล่าสุด และใช้ prefetch_related เพื่อดึงข้อมูลไฟล์ที่เกี่ยวข้องมารอไว้ (Optimization)
บรรทัดที่ 5-6: ตรวจสอบว่ามีการส่งข้อมูลแบบ POST (กดปุ่มบันทึก) หรือไม่ หากใช่ ให้นำข้อมูลและไฟล์ที่อัปโหลดเข้าสู่ SubjectForm
บรรทัดที่ 7-10: ตรวจสอบความถูกต้องของข้อมูล (is_valid) หากถูกต้อง จะทำการบันทึกข้อมูลวิชาลงในตัวแปร แต่ยังไม่บันทึกลงฐานข้อมูล (commit=False) เพื่อระบุผู้ใช้ (user) ก่อน แล้วจึงสั่ง save()
บรรทัดที่ 12-16: ดึงรายการไฟล์ทั้งหมดที่อัปโหลดเข้ามา และวนลูปตรวจสอบจำนวนไฟล์ หากรวมกับไฟล์ที่มีอยู่แล้วเกิน 5 ไฟล์ จะหยุดการทำงานในส่วนนี้
บรรทัดที่ 18-24: สร้างข้อมูลลงในตาราง File โดยเชื่อมโยงกับวิชาที่เพิ่งสร้าง (subject) และบันทึกรายละเอียดของไฟล์ เช่น ชื่อไฟล์ (file_name), ชนิดไฟล์ (file_type), และขนาดไฟล์ (size_in_bytes)
บรรทัดที่ 26: เมื่อบันทึกข้อมูลเสร็จสิ้น ให้ทำการรีโหลดหน้าเว็บ (redirect) กลับมาที่หน้าเพิ่มวิชาอีกครั้ง
บรรทัดที่ 27-28: กรณีที่เป็นการเข้าชมหน้าเว็บครั้งแรก (GET request) ให้สร้างฟอร์มเปล่าเตรียมไว้สำหรับรับข้อมูล
บรรทัดที่ 30-34: ส่งข้อมูลฟอร์ม (form) และรายวิชาที่มีอยู่ (subjects) ไปแสดงผลที่หน้าจอ core/add_subject.html

4.3.5 การพัฒนาระบบการลบข้อมูลไฟล์

```
1   @login_required
2   def delete_file_view(request, file_id):
3       file_obj = get_object_or_404(File, file_id=file_id, subject__user=request.user)
4       subject = file_obj.subject
5
6       file_obj.delete()
7       remaining_files = File.objects.filter(subject=subject).order_by('uploaded_at')
8
9       for index, f in enumerate(remaining_files):
10          f.order = index + 1
11          f.save()
12
13      messages.success(request, 'ลบไฟล์เรียบร้อยแล้ว')
14      return redirect('add_subject')
```

ภาพประกอบที่ 4.12 การพัฒนาระบบการลบข้อมูลไฟล์ (views.py)

จากภาพประกอบที่ 4.12 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อจำกัดสิทธิ์การเข้าถึง โดยผู้ใช้จะต้องล็อกอินเข้าสู่ระบบก่อนจึงจะสามารถใช้งานฟังก์ชันนี้ได้
บรรทัดที่ 2: ประกาศฟังก์ชัน delete_file_view โดยรับพารามิเตอร์ file_id เพื่อระบุไฟล์ที่ต้องการลบ
บรรทัดที่ 3-4: ตรวจสอบและดึงข้อมูลไฟล์จากฐานข้อมูล (get_object_or_404) โดยมีเงื่อนไขสำคัญคือไฟล์นั้นต้องเป็นของวิชาที่ผู้ใช้นั้นเป็นเจ้าของเท่านั้น (subject__user=request.user) เพื่อป้องกันการลบไฟล์ของคนอื่น
บรรทัดที่ 6: สั่งลบข้อมูลไฟล์ออกจากฐานข้อมูลด้วยคำสั่ง .delete()
บรรทัดที่ 7-11: ดึงรายการไฟล์ที่เหลืออยู่ในวิชานั้นมาเรียงตามเวลาที่อัปโหลด แล้วทำการวนลูปเพื่อจัดลำดับ (order) ใหม่ (เช่น 1, 2, 3...) เพื่อให้ลำดับไฟล์เรียงกันถูกต้องไม่กระโดดข้ามหลังจากมีไฟล์ถูกลบไป
บรรทัดที่ 13-14: แสดงข้อความแจ้งเตือนความสำเร็จ (messages.success) และเปลี่ยนเส้นทางกลับไปยังหน้าเพิ่มรายวิชา (add_subject)

4.4 การพัฒนาระบบกำหนดเวลาว่าง

4.4.1 การพัฒนาการบันทึกข้อมูลเวลาว่าง

```
1   class UserAvailability(models.Model):
2       DAY_CHOICES = [
3           (0, 'Monday'), (1, 'Tuesday'), (2, 'Wednesday'),
4           (3, 'Thursday'), (4, 'Friday'), (5, 'Saturday'), (6, 'Sunday')
5       ]
6
7       availability_id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
8       user = models.ForeignKey(CustomUser, on_delete=models.CASCADE)
9       day_of_week = models.IntegerField(choices=DAY_CHOICES)
10      hour = models.IntegerField()
11
12      class Meta:
13          unique_together = ('user', 'day_of_week', 'hour')
14
15      def __str__(self):
16          return f"{self.user.username} - {self.get_day_of_week_display()} @ {self.hour}:00"
```

ภาพประกอบที่ 4.13 การพัฒนาระบบการบันทึกข้อมูลเวลาว่าง (models.py)

จากภาพประกอบที่ 4.13 โครงสร้างของไฟล์ models.py อธิบายการทำงานได้ดังนี้

บรรทัดที่ 1: ประกาศคลาส UserAvailability สืบทอดจาก models.Model เพื่อสร้างตารางสำหรับเก็บข้อมูลช่วงเวลาที่ผู้ใช้ว่าง

บรรทัดที่ 2-5: กำหนดตัวแปร DAY_CHOICES เพื่อเก็บตัวเลือกวันในสัปดาห์ โดยจับคู่ค่าตัวเลข 0-6 กับชื่อวัน (จันทร์ - อาทิตย์) เพื่อใช้ในการบันทึกและแสดงผล

บรรทัดที่ 7: กำหนดฟิลด์ availability_id เป็นคีย์หลัก (Primary Key) ชนิด UUID ที่สร้างค่าสุ่มให้อัตโนมัติและไม่สามารถแก้ไขได้

บรรทัดที่ 8: กำหนดฟิลด์ user เป็น Foreign Key เชื่อมโยงกับตารางผู้ใช้ (CustomUser) หากผู้ใช้ถูกลบ ข้อมูลเวลาว่างนี้จะถูกลบไปด้วย (CASCADE)

บรรทัดที่ 9: กำหนดฟิลด์ day_of_week สำหรับเก็บวันที่ว่าง เป็นชนิดตัวเลขจำนวนเต็ม โดยอ้างอิงค่าจาก DAY_CHOICES

บรรทัดที่ 10: กำหนดฟิลด์ hour สำหรับเก็บช่วงเวลาที่ว่าง เป็นชนิดตัวเลขจำนวนเต็ม (เช่น 9 หมายถึง 09:00 - 10:00)

บรรทัดที่ 12-13: ประกาศคลาสย่อย Meta และกำหนด unique_together เพื่อสร้างกฎบังคับว่า ห้ามมีข้อมูลซ้ำกันในชุดของ (ผู้ใช้, วัน, ชั่วโมง) เพื่อป้องกันการบันทึกเวลาเดิมซ้ำซ้อน

บรรทัดที่ 15-16: กำหนดฟังก์ชัน __str__ ให้คืนค่าเป็นชื่อผู้ใช้และวันเวลา (เช่น "UserA - Monday @ 9:00") เพื่อให้ง่ายต่อการตรวจสอบข้อมูลในระบบหลังบ้าน (Admin)

4.4.2 การพัฒนาระบบกำหนดเวลาว่าง

```
@login_required
def set_schedule_view(request):
    days_of_week = {
        0: 'จันทร์', 1: 'อังคาร', 2: 'พุธ',
        3: 'พฤหัสบดี', 4: 'ศุกร์', 5: 'เสาร์', 6: 'อาทิตย์'
    }

    time_definitions = {
        'morning': list(range(6, 12)),
        'afternoon': list(range(12, 17)),
        'evening': list(range(17, 21)),
        'night': list(range(21, 24)) + list(range(0, 6))
    }

    if request.method == 'POST':
        UserAvailability.objects.filter(user=request.user).delete()

        slots_to_create = []
        for key in request.POST:
            if key.startswith('hour_'):
                _, day, hour = key.split('_')
                slots_to_create.append(
                    UserAvailability(user=request.user, day_of_week=int(day), hour=int(hour))
                )
        UserAvailability.objects.bulk_create(slots_to_create)
        return redirect('study_settings')

    existing_hours = UserAvailability.objects.filter(user=request.user)
    selected_hour_keys = set(f"{slot.day_of_week}_{slot.hour}" for slot in existing_hours)

    ui_blocks_state = {}

    for day_num in days_of_week.keys():
        for slot_name, hours in time_definitions.items():
            selected_count = sum(1 for h in hours if f"{day_num}_{h}" in selected_hour_keys)
            total_count = len(hours)

            state = 'none'
            if selected_count == total_count:
                state = 'full'
            elif selected_count > 0:
                state = 'partial'
```

```
43
44              ui_blocks_state[f"{day_num}_{slot_name}"] = {
45                  'state': state,
46                  'selected_hours': [h for h in hours if f"{day_num}_{h}" in selected_hour_keys]
47              }
48
49      time_slots_data = [
50          {'id': 'morning', 'class': 'morning', 'label': 'ช่วงเช้า', 'hours': time_definitions['morning']},
51          {'id': 'afternoon', 'class': 'afternoon', 'label': 'ช่วงบ่าย', 'hours':
52  time_definitions['afternoon']},
53          {'id': 'evening', 'class': 'evening', 'label': 'ช่วงเย็น', 'hours': time_definitions['evening']},
54          {'id': 'night', 'class': 'night', 'label': 'ช่วงดึก', 'hours': time_definitions['night']},
55      ]
565
7       context = {
58          'days': days_of_week,
59          'time_slots': time_slots_data,
60          'ui_blocks_state': ui_blocks_state,
61          'selected_hour_keys': list(selected_hour_keys), # เอาไว้ check ใน modal
62      }
63      return render(request, 'core/set_schedule.html', context)
```

ภาพประกอบที่ 4.14 การพัฒนาระบบกำหนดเวลาว่าง (views.py)

จากภาพประกอบที่ 4.14 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้

บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อจำกัดสิทธิ์ให้เฉพาะผู้ใช้ที่ล็อกอินแล้วเท่านั้นที่สามารถเข้าถึงหน้านี้ได้

บรรทัดที่ 3-6: กำหนดตัวแปร days_of_week แบบ Dictionary เพื่อจับคู่ค่าตัวเลข 0-6 ให้เป็นชื่อวันภาษาไทย (จันทร์ - อาทิตย์)

บรรทัดที่ 9-14: กำหนดช่วงเวลามาตรฐาน (time_definitions) โดยแบ่งเป็น เช้า (06-12), บ่าย (12-17), เย็น (17-21) และดึก (21-24, 0-6) เพื่อใช้ในการแสดงผลหน้าเว็บ

บรรทัดที่ 17-18: ตรวจสอบว่ามีการส่งข้อมูลแบบ POST (กดบันทึก) หรือไม่ หากใช่ ให้ทำการลบข้อมูลเวลาว่างเดิมของผู้ใช้ทั้งหมดทิ้งก่อน (delete()) เพื่อป้องกันข้อมูลซ้ำซ้อน

บรรทัดที่ 20-25: วนลูปรับค่าจากฟอร์มที่ชื่อขึ้นต้นด้วย hour_ (ซึ่งเก็บค่า วัน_ชั่วโมง) แล้วสร้างออบเจกต์ UserAvailability เตรียมไว้ในลิสต์

บรรทัดที่ 26-27: ใช้คำสั่ง bulk_create เพื่อบันทึกข้อมูลทั้งหมดลงฐานข้อมูลในครั้งเดียว แล้วเปลี่ยนเส้นทางไปยังหน้าตั้งค่าการเรียน (study_settings)

บรรทัดที่ 30-31: กรณีเป็น GET Request ให้ดึงข้อมูลเวลาว่างเดิมที่มีอยู่ในฐานข้อมูลมาเก็บไว้ในตัวแปร selected_hour_keys เพื่อนำไปแสดงผลว่าเคยเลือกเวลาไหนไว้บ้าง

บรรทัดที่ 35-47: คำนวณสถานะของปุ่มช่วงเวลา (ui_blocks_state) โดยตรวจสอบว่าในแต่ละช่วง (เช่น เช้า) ผู้ใช้เลือกเวลาครบทุกชั่วโมงหรือไม่ (full), เลือกบางชั่วโมง (partial), หรือไม่เลือกเลย (none) เพื่อให้หน้าเว็บแสดงสีปุ่มได้ถูกต้อง
บรรทัดที่ 57-62: รวบรวมข้อมูลทั้งหมด (วัน, ช่วงเวลา, สถานะปุ่ม) ใส่ในตัวแปร context และส่งไปแสดงผล

4.5 การพัฒนาระบบการตั้งค่าการอ่าน

```
1   @login_required
2   def study_settings_view(request):
3       settings, created = UserSettings.objects.get_or_create(user=request.user)
4
5       if request.method == 'POST':
6           form = UserSettingsForm(request.POST, instance=settings)
7           if form.is_valid():
8               user_settings = form.save() # บันทึกค่า settings ก่อน
9               success = generate_study_schedule(request.user, user_settings)
10
11              if success:
12                  messages.success(request, 'สร้างตารางเรียนเรียบร้อยแล้ว!')
13              else:
14                  messages.warning(request, 'บันทึกการตั้งค่าแล้ว แต่เกิดข้อผิดพลาดในการสร้างตาราง (
15  อาจไม่มีข้อมูลวิชา หรือ AI มีปัญหา)')
16
17              return redirect('home_page')
18      else:
19          form = UserSettingsForm(instance=settings)
20
21      context = {
22          'form': form
23      }
24      return render(request, 'core/study_settings.html', context)
```

ภาพประกอบที่ 4.15 การพัฒนาระบบการตั้งค่าการอ่าน (views.py)

จากภาพประกอบที่ 4.15 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อตรวจสอบสิทธิ์ โดยอนุญาตให้เฉพาะผู้ใช้ที่ล็อกอินแล้วเท่านั้นเข้าถึงหน้านี้ได้
บรรทัดที่ 2: ประกาศฟังก์ชัน study_settings_view และใช้คำสั่ง get_or_create เพื่อดึงค่าการตั้งค่าเดิมของผู้ใช้มาแสดง หากยังไม่มีข้อมูล ระบบจะสร้างค่าเริ่มต้นให้ทันที (ป้องกัน Error)

บรรทัดที่ 4-6: ตรวจสอบว่ามีการส่งข้อมูลแบบ POST (กดบันทึก) หรือไม่ หากใช่ ให้นำข้อมูลจากฟอร์มมาผูกกับข้อมูลการตั้งค่าเดิม (instance=settings) และตรวจสอบความถูกต้อง (is_valid)
บรรทัดที่ 7: สั่งบันทึกการตั้งค่าใหม่ลงในฐานข้อมูล (form.save())
บรรทัดที่ 8: จุดสำคัญ: เรียกใช้ฟังก์ชัน generate_study_schedule เพื่อสั่งให้ AI ประมวลผลและสร้างตารางเรียนใหม่ทันที โดยอิงจากการตั้งค่าล่าสุดที่เพิ่งบันทึกไป
บรรทัดที่ 10-14: ตรวจสอบผลลัพธ์จาก AI (success) หากสำเร็จจะแสดงข้อความแจ้งเตือนสีเขียว แต่หากล้มเหลว (เช่น ไม่ได้เพิ่มวิชา) จะแสดงข้อความเตือนสีเหลือง
บรรทัดที่ 16: เมื่อกระบวนการเสร็จสิ้น ให้เปลี่ยนเส้นทางกลับไปยังหน้าแรก (home_page)

4.6 การพัฒนาระบบ AI

4.6.1 การพัฒนาฟังก์ชันการจัดตารางด้วย AI

```
1  def generate_study_schedule(user, user_settings):
2      print("--- เริ่มต้นกระบวนการ AI (Robust Version) ---")
3
4      subjects = Subject.objects.filter(user=user)
5      if not subjects.exists():
6          print("Error: ไม่พบวิชาเรียนในระบบ (กรุณาเพิ่มวิชาก่อน)")
7          return False
8      subjects_data = []
9      for sub in subjects:
10         subjects_data.append({
11             "name": sub.name,
12             "difficulty": sub.get_difficulty_display(),
13             "exam_date": sub.exam_date.strftime("%Y-%m-%d %H:%M") if sub.exam_date else
14 "No exam date"
15         })
16     print(f"ส่งข้อมูลวิชาไป: {[s['name'] for s in subjects_data]}")
17     availability = UserAvailability.objects.filter(user=user)
18     availability_data = []
19     if availability.exists():
20         for slot in availability:
21             # availability_data.append(f"{slot.get_day_of_week_display()} -
22 {slot.get_time_slot_display()}")
23             start_time = f"{slot.hour:02d}:00"
24             end_time = f"{(slot.hour + 1) % 24:02d}:00"
25             availability_data.append(f"{slot.get_day_of_week_display()}: {start_time} - {end_time}")
26         availability_prompt = f"User's available slots: {availability_data}"
27     else:
28         availability_prompt = "The user has NOT provided specific availability. Please create a
29 balanced schedule."
30
```

```
31      now = timezone.localtime(timezone.now())
32      current_time_str = now.strftime("%Y-%m-%d %H:%M")
33
34      prompt = f"""
35      You are an expert study planner. Create a study schedule for a student.
36
37      Current Date/Time: {current_time_str} (Do NOT schedule anything before this time).
38
39      Configuration:
40      - Session Duration: {user_settings.session_duration} minutes per session.
41      - Break Duration: {user_settings.break_duration} minutes between sessions.
42
43      Subjects to study (Source of Truth):
44      {json.dumps(subjects_data, ensure_ascii=False)}
45
46      Availability Constraints:
47      {availability_prompt}
48
49      Instructions:
50      1. Plan for the next 5 days only.
51      2. Return the output STRICTLY as a JSON Array.
52      3. Date format MUST be "YYYY-MM-DD HH:MM".
53      4. CRITICAL: You MUST use the EXACT subject name provided in the 'Subjects to study'
54  list.
55          - Do NOT paraphrase (e.g., do not change "History of Art" to "Art History").
565         - Do NOT abbreviate.
7           - Copy the name string exactly character-by-character.
58
59      JSON Format required:
60      [
61          {{
62              "subject_name": "Subject Name Here (EXACT MATCH ONLY)",
63              "start_time": "YYYY-MM-DD HH:MM",
64              "end_time": "YYYY-MM-DD HH:MM",
65              "topic": "Topic to read"
66          }}
67      ]
68      """
69      model = genai.GenerativeModel('models/gemini-2.5-flash')
70      try:
71          response = model.generate_content(prompt)
72          raw_text = response.text
73          print(f"AI Response Raw (First 100 chars): {raw_text[:100]}...")
74          cleaned_text = re.sub(r'```json\s*', '', raw_text)
```

```
75          cleaned_text = re.sub(r'```\s*', '', cleaned_text)
76          cleaned_text = cleaned_text.strip()
77          json_match = re.search(r'\[.*\]', cleaned_text, re.DOTALL)
78          if json_match:
79              cleaned_text = json_match.group(0)
80          try:
81              data = json.loads(cleaned_text)
82          except json.JSONDecodeError as e:
83              print(f"JSON Decode Error: {e}")
84              print(f"Text that failed: {cleaned_text}")
85              return False
86          schedule_list = []
87          if isinstance(data, list):
88              schedule_list = data
89          elif isinstance(data, dict):
90              for key, value in data.items():
91                  if isinstance(value, list):
92                      schedule_list = value
93                      break
94
95          if not schedule_list:
96               print("Error: แปลง JSON ได้แต่ไม่พบรายการตารางเรียน (Empty List)")
97               return False
98
99          print(f"ได้รายการตารางเรียนมาทั้งหมด: {len(schedule_list)} รายการ")
100         old_sessions = StudySession.objects.filter(user=user, is_completed=False)
101
102         for session in old_sessions:
103             if session.google_event_id:
104                 delete_event_from_google(user, session.google_event_id)
105
106         StudySession.objects.filter(user=user, is_completed=False).delete()
107
108         new_sessions = []
109
110         for item in schedule_list:
111             subject_name = item.get('subject_name', '').strip()
112
113             subject_obj = subjects.filter(name__iexact=subject_name).first()
114
115             if subject_obj:
116                 try:
117                     naive_start = datetime.datetime.strptime(item['start_time'], "%Y-%m-%d
119 %H:%M")
```

```
119                 naive_end = datetime.datetime.strptime(item['end_time'], "%Y-%m-%d %H:%M")
120
121                 start_t = timezone.make_aware(naive_start)
122                 end_t = timezone.make_aware(naive_end)
123
124                 new_sessions.append(StudySession(
125                     user=user,
126                     subject=subject_obj,
127                     start_time=start_t,
128                     end_time=end_t,
129                     topic=item.get('topic', 'Review')
130                 ))
131             except ValueError as ve:
132                 print(f"Date format error: {ve} in item: {item}")
133         else:
134             print(f"Warning: วิชา '{subject_name}' ที่ AI บอกมา ไม่มีในฐานข้อมูล")
135
136     if new_sessions:
137         StudySession.objects.bulk_create(new_sessions)
139         print(f"SUCCESS: บันทึกตารางเรียนลง DB สำเร็จ {len(new_sessions)} รายการ")
140         return True
141     else:
142         print("Error: ไม่สามารถสร้าง Session ได้เลย (อาจเพราะชื่อวิชาไม่ตรง)")
143         return False
144
145 except Exception as e:
146     print(f"CRITICAL ERROR: {e}")
147     return False
```

ภาพประกอบที่ 4.16 การพัฒนาระบบ

จากภาพประกอบที่ 4.16 โครงสร้างของไฟล์ ai_service.py อธิบายการทำงานได้ดังนี้

บรรทัดที่ 1-3: ประกาศฟังก์ชัน generate_study_schedule รับค่า user และ user_settings เพื่อเริ่มกระบวนการจัดตาราง

บรรทัดที่ 5-10: ดึงข้อมูลรายวิชา (Subject) ของผู้ใช้จากฐานข้อมูล หากไม่พบวิชาจะจบการทำงานทันที

บรรทัดที่ 12-18: เตรียมข้อมูลวิชา (ชื่อ, ความยาก, วันสอบ) ให้อยู่ในรูปแบบ List เพื่อส่งให้ AI

บรรทัดที่ 21-28: ดึงข้อมูลเวลาว่าง (UserAvailability) ของผู้ใช้ และแปลงรูปแบบเวลาให้เป็นข้อความที่ AI เข้าใจง่าย (เช่น "Monday: 09:00 - 12:00")

บรรทัดที่ 30-45: สร้างชุดคำสั่ง (Prompt) เพื่อส่งไปยัง Google Gemini API โดยระบุเงื่อนไขการจัดตาราง เช่น ระยะเวลาเรียน, เวลาพัก, และรูปแบบ JSON ที่ต้องการ

บรรทัดที่ 47-49: เรียกใช้โมเดล gemini-2.5-flash และส่ง Prompt เพื่อขอผลลัพธ์

บรรทัดที่ 51-58: รับค่าที่ได้จาก AI และทำการตัดส่วนที่ไม่ใช่ JSON ออก (Clean Data) เพื่อป้องกัน Error

บรรทัดที่ 60-72: แปลงข้อความ JSON เป็น Object และวนลูปบันทึกข้อมูล StudySession ลงในฐานข้อมูล พร้อมกำหนดวันที่และเวลาเรียนให้ถูกต้อง

4.6.2 การพัฒนาฟังก์ชันการสรุปเนื้อหาหลังอ่านด้วย AI

```
1   def generate_content_summary(subject_name, topic):
2       """
3       ฟังก์ชันสำหรับให้ AI สรุปเนื้อหาการเรียน
4       """
5       try:
6           # ถ้าไม่มีหัวข้อ ให้สรุปภาพรวมวิชา
7           topic_text = topic if topic else "General concepts"
8
9           prompt = f"""
10          You are a helpful tutor. Summarize the key takeaways for the topic: "{topic_text}"
11          in the subject: "{subject_name}".
12
13          Instructions:
14          1. Summarize in Thai language.
15          2. Keep it concise (around 3-5 bullet points).
16          3. Make it encouraging.
17          4. Use HTML tags for formatting (e.g., <ul>, <li>, <strong>).
18
19          Example Output format:
20          <ul>
21              <li><strong>Point 1:</strong> Detail...</li>
22              <li><strong>Point 2:</strong> Detail...</li>
23          </ul>
24          <p>Keep up the good work!</p>
25          """
26
27          model = genai.GenerativeModel('models/gemini-2.5-flash') # หรือรุ่นที่คุณใช้ได้
28          response = model.generate_content(prompt)
29          return response.text
30
31      except Exception as e:
32          print(f"AI Summary Error: {e}")
33          return "<p>ขออภัย ไม่สามารถสรุปเนื้อหาได้ในขณะนี้ (AI Error)</p>"
```

ภาพประกอบที่ 4.17 การพัฒนาฟังก์ชันการสรุปเนื้อหาหลังอ่านด้วย AI (ai_service.py)

จากภาพประกอบที่ 4.17 โครงสร้างของไฟล์ ai_service.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: ประกาศฟังก์ชัน generate_content_summary รับค่า subject_name (ชื่อวิชา) และ topic (หัวข้อที่เรียน) เพื่อใช้เป็นข้อมูลนำเข้าให้ AI
บรรทัดที่ 5-7: เริ่มต้นกระบวนการในบล็อก try และตรวจสอบตัวแปร topic หากไม่มีข้อมูลส่งมา ให้กำหนดค่าเริ่มต้นเป็น "General concepts"
บรรทัดที่ 9-23: สร้างชุดคำสั่ง (Prompt) เพื่อส่งให้ AI โดยกำหนดบทบาทให้เป็นติวเตอร์ สั่งให้สรุปเนื้อหาเป็นภาษาไทย สั้นกระชับ และระบุให้ตอบกลับในรูปแบบ HTML (เช่น <ul>, <li>) เพื่อนำไปแสดงผลบนหน้าเว็บได้ทันที
บรรทัดที่ 25-27: เรียกใช้โมเดล gemini-2.5-flash ส่งคำสั่งไปประมวลผล (generate_content) และส่งคืนข้อความ (response.text) กลับไปยังส่วนที่เรียกใช้
บรรทัดที่ 29-31: ส่วนดักจับข้อผิดพลาด (except) หากเกิดปัญหากับ API ของ AI จะทำการพิมพ์ Error Log และส่งคืนข้อความแจ้งเตือน HTML กลับไปแทน เพื่อไม่ให้โปรแกรมหยุดทำงาน

4.6.3 การพัฒนาฟังก์ชันการสร้างแบบทดสอบหลังอ่านด้วย AI

```
1   def generate_quiz_questions(subject_name, topic):
2       print(f"--- AI Quiz Start: {subject_name} ---")
3
4       try:
5           topic_text = topic if topic else "General concepts"
6
7           prompt = f"""
8           Create a multiple-choice quiz for the subject "{subject_name}", topic: "{topic_text}".
9           Instructions:
10          1. Create exactly 5 questions.
11          2. Language: Thai (ภาษาไทย).
12          3. Difficulty: Moderate.
13          4. Return ONLY a JSON Array. No Markdown. No Intro text.
14
15          JSON Format Example:
16          [
17              {{
18                  "question": "คำถาม?",
19                  "options": ["ก", "ข", "ค", "ง"],
20                  "correct_index": 0
21              }}
22          ]
23          """
24          model = genai.GenerativeModel('models/gemini-2.5-flash')
25          response = model.generate_content(prompt)
26          raw_text = response.text
27
```

```
28          print(f"DEBUG RAW AI: {raw_text[:50]}...")
29          match = re.search(r'\[.*\]', raw_text, re.DOTALL)
30
31          if match:
32              json_str = match.group(0)
33              json_str = json_str.replace("`", "")
34              return json.loads(json_str)
35          else:
36              print("Error: AI ไม่ได้ส่ง JSON Array มา")
37              return None
38
39      except Exception as e:
40          print(f"AI Quiz Error: {e}")
41          return None
```

ภาพประกอบที่ 4.18 การพัฒนาระบบ

จากภาพประกอบที่ 4.18 โครงสร้างของไฟล์ ai_service.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: ประกาศฟังก์ชัน generate_quiz_questions รับชื่อวิชาและหัวข้อที่จะสอบ
บรรทัดที่ 4-15: สร้าง Prompt สั่งให้ AI สร้างโจทย์ข้อสอบปรนัย 5 ข้อ เป็นภาษาไทย ระดับความยากปานกลาง พร้อมตัวเลือกและเฉลย ในรูปแบบ JSON
บรรทัดที่ 17-18: ส่งคำสั่งไปยัง Gemini API
บรรทัดที่ 20-25: ใช้ Regular Expression (re) ค้นหาและดึงเฉพาะส่วนที่เป็น JSON Array ออกมาจากข้อความที่ AI ตอบกลับ
บรรทัดที่ 27-30: แปลงข้อมูล JSON เป็น Python List และส่งกลับไปยังส่วนแสดงผล (View)

4.7 การพัฒนาระบบเชื่อมต่อ Google Calendar

4.7.1 การพัฒนาระบบเชื่อมต่อ Google Calendar (google_calendar.py)

```
1   def sync_sessions_to_google(user):
2       try:
3           g_cred = GoogleCredential.objects.get(user=user)
4           creds = Credentials(**g_cred.token)
5
6           try:
7               if not creds.valid:
8                   if creds.expired and creds.refresh_token:
9                       creds.refresh(Request())
10                      g_cred.token.update({
11                          'token': creds.token,
12                          'expiry': creds.expiry.isoformat() if creds.expiry else None
13                      })
14                      g_cred.save()
15          except (RefreshError, Exception) as e:
16              print(f"Token expired/invalid: {e}")
17              g_cred.delete()
18              return False, "ยังไม่ได้เชื่อมต่อ (Session หมดอายุ กรุณา Login ใหม่)"
19
20          service = build('calendar', 'v3', credentials=creds)
21
22          sessions = StudySession.objects.filter(user=user, is_synced=False)
23
24          synced_count = 0
25          for session in sessions:
26              start_time = session.start_time.isoformat()
27              end_time = session.end_time.isoformat()
28
29              event = {
30                  'summary': f"อ่าน: {session.subject.name}",
31                  'description': f"Topic: {session.topic}\n(Created by Smart Study Planner)",
32                  'start': {
33                      'dateTime': start_time,
34                      'timeZone': 'Asia/Bangkok', # หรือ 'UTC' ตาม setting
35                  },
36                  'end': {
37                      'dateTime': end_time,
38                      'timeZone': 'Asia/Bangkok',
39                  },
40                  'reminders': {
```

```
41                  'useDefault': False,
42                  'overrides': [
43                      {'method': 'popup', 'minutes': 10},
44                  ],
45              },
46          }
47
48          try:
49              event_result = service.events().insert(calendarId='primary', body=event).execute()
50
51              session.google_event_id = event_result['id']
52              session.is_synced = True
53              session.save()
54              synced_count += 1
55
56          except Exception as e:
57              error_str = str(e)
58              print(f"Error syncing session {session.session_id}: {e}")
59
60              if 'invalid_grant' in error_str:
61                  g_cred.delete() # ลบ Token ทิ้ง
62                  return False, "ยังไม่ได้เชื่อมต่อ (Token หลุดระหว่างทำงาน กรุณา Login ใหม่)"
63
64      return True, f"ซิงค์เรียบร้อย {synced_count} รายการ"
65
66  except GoogleCredential.DoesNotExist:
67      return False, "ยังไม่ได้เชื่อมต่อบัญชี Google"
68  except Exception as e:
69      return False, str(e)
70
```

ภาพประกอบที่ 4.19 การพัฒนาระบบ

จากภาพประกอบที่ 4.19 โครงสร้างของไฟล์ google_calendar.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 3-4: ดึงข้อมูล Credential (Token) ของผู้ใช้จากฐานข้อมูล GoogleCredential และแปลงให้อยู่ในรูปแบบออบเจกต์ที่ Google API รองรับ
บรรทัดที่ 7-14: ตรวจสอบสถานะของ Token ว่าหมดอายุหรือไม่ หากหมดอายุ (expired) ระบบจะทำการขอ Token ใหม่ (Refresh Token) และบันทึกค่าใหม่ลงฐานข้อมูลทันทีเพื่อให้ใช้งานต่อได้โดยผู้ใช้ไม่ต้องล็อกอินใหม่
บรรทัดที่ 21: สร้างออบเจกต์ service สำหรับเรียกใช้งาน Google Calendar API เวอร์ชัน 3
บรรทัดที่ 23: ค้นหาข้อมูลตารางเรียน (StudySession) ของผู้ใช้นั้น เฉพาะรายการที่ยังไม่เคยถูกซิงค์ (is_synced=False)

บรรทัดที่ 26-45: วนลูปข้อมูลตารางเรียน และจัดเตรียมข้อมูลให้อยู่ในรูปแบบ JSON (Payload) ตามมาตรฐานของ Google ประกอบด้วย ชื่อวิชา (summary), รายละเอียด (description), เวลาเริ่ม-จบ (start, end) โดยระบุ Timezone เป็น Asia/Bangkok และตั้งค่าการแจ้งเตือนล่วงหน้า 10 นาที
บรรทัดที่ 48: เรียกใช้คำสั่ง service.events().insert เพื่อส่งข้อมูลไปสร้างนัดหมายบนปฏิทิน Google ของผู้ใช้จริง
บรรทัดที่ 50-52: เมื่อสร้างสำเร็จ ระบบจะได้รับ id ของ Event จาก Google กลับมา ให้ทำการบันทึกลงในฐานข้อมูลและเปลี่ยนสถานะ is_synced เป็น True เพื่อป้องกันการซิงค์ซ้ำ
บรรทัดที่ 58-60: กรณีเกิดข้อผิดพลาดร้ายแรงเช่น invalid_grant (ผู้ใช้ยกเลิกสิทธิ์) ระบบจะลบ Token ออกจากฐานข้อมูลเพื่อให้ผู้ใช้ทำการเชื่อมต่อใหม่ในครั้งถัดไป

4.7.2 การพัฒนาระบบเชื่อมต่อ Google Calendar (views.py)

```
1   @login_required
2   def sync_calendar_view(request):
3       """กดปุ่มซิงค์"""
4       success, msg = sync_sessions_to_google(request.user)
5
6       if not success and "ยังไม่ได้เชื่อมต่อ" in msg:
7           # ถ้ายังไม่เคย Login ให้ส่งไปหน้า Login Google
8           return redirect('google_auth_start')
9
10      if success:
11          messages.success(request, f"Sync Success: {msg}")
12      else:
13          messages.error(request, f"Sync Error: {msg}")
14
15      return redirect('home_page')
```

ภาพประกอบที่ 4.20 การพัฒนาระบบเชื่อมต่อ Google Calendar (views.py)

จากภาพประกอบที่ 4.20 โครงสร้างของไฟล์ อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อป้องกันการเข้าถึงจากผู้ที่ยังไม่ได้ล็อกอิน
บรรทัดที่ 2: ประกาศฟังก์ชัน sync_calendar_view สำหรับรองรับการกดปุ่ม "Sync Google Calendar" จากหน้าเว็บ
บรรทัดที่ 4: เรียกใช้ฟังก์ชัน sync_sessions_to_google (ที่สร้างไว้ในหัวข้อก่อนหน้า) เพื่อเริ่มกระบวนการซิงค์ข้อมูล โดยรับค่าสถานะความสำเร็จ (success) และข้อความตอบกลับ (msg)
บรรทัดที่ 6-8: ตรวจสอบกรณีที่ซิงค์ไม่สำเร็จเนื่องจากยังไม่เคยเชื่อมต่อบัญชี Google มาก่อน ระบบจะทำการเปลี่ยนเส้นทาง (redirect) ไปยังฟังก์ชันเริ่มการยืนยันตัวตน (google_auth_start)
บรรทัดที่ 10-11: กรณีซิงค์สำเร็จ ให้แสดงข้อความแจ้งเตือนสีเขียว (messages.success) เพื่อบอกผู้ใช้ว่าข้อมูลถูกส่งไปปฏิทินเรียบร้อยแล้ว

บรรทัดที่ 12-13: กรณีเกิดข้อผิดพลาดอื่นๆ (เช่น Token หมดอายุ หรือเน็ตหลุด) ให้แสดงข้อความแจ้งเตือนสีแดง (messages.error) พร้อมระบุสาเหตุ
บรรทัดที่ 15: เมื่อทำงานเสร็จสิ้น ให้เปลี่ยนเส้นทางกลับไปยังหน้าแรก (home_page)

4.8 การพัฒนาระบบแจ้งเตือน

4.8.1 การพัฒนาระบบแจ้งเตือน (context_processors.py)

```
1  def notifications(request):
2      if request.user.is_authenticated:
3          # ดึงแจ้งเตือนทั้งหมดของผู้ใช้ (เอาแค่ 10 อันล่าสุด)
4          all_notifs = Notification.objects.filter(recipient=request.user)[:10]
5          # นับจำนวนที่ยังไม่อ่าน
6          unread_count = Notification.objects.filter(recipient=request.user, is_read=False).count()
7          return {
8              'notifications': all_notifs,
9              'unread_count': unread_count
10         }
11     return {
12         'notifications': [],
13         'unread_count': 0
14     }
```

ภาพประกอบที่ 4.21 การพัฒนาระบบการตั้งค่าการแจ้งเตือน

จากภาพประกอบที่ 4.21 โครงสร้างของไฟล์ context_processors.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: ประกาศฟังก์ชัน notifications ซึ่งทำหน้าที่เป็น Context Processor สำหรับส่งข้อมูลแจ้งเตือนไปยังทุกหน้าเว็บโดยอัตโนมัติ
บรรทัดที่ 2: ตรวจสอบสถานะของผู้ใช้ว่าได้ทำการล็อกอินเข้าสู่ระบบแล้วหรือไม่ (is_authenticated)
บรรทัดที่ 4: ดึงข้อมูลการแจ้งเตือน (Notification) ของผู้ใช้นั้นๆ โดยกรองเฉพาะของผู้ใช้คนปัจจุบัน และตัดมาแสดงผลเพียง 10 รายการล่าสุด
บรรทัดที่ 6: นับจำนวนการแจ้งเตือนที่ยังไม่ได้เปิดอ่าน (is_read=False) เพื่อนำไปแสดงเป็นตัวเลขแจ้งเตือน (Badge) บนแถบเมนู
บรรทัดที่ 7-10: ส่งค่ากลับในรูปแบบ Dictionary ประกอบด้วยรายการแจ้งเตือน (notifications) และจำนวนที่ยังไม่ได้อ่าน (unread_count) เพื่อให้ Template นำไปแสดงผล
บรรทัดที่ 11-14: กรณีที่ผู้ใช้ยังไม่ได้ล็อกอิน ให้ส่งค่ารายการแจ้งเตือนว่างเปล่าและจำนวนเป็น 0 กลับไป เพื่อป้องกันข้อผิดพลาดในการแสดงผล

4.8.2 การพัฒนาระบบการบันทึกข้อมูลการแจ้งเตือน

```
1   class Notification(models.Model):
2       TYPE_CHOICES = [
3           ('info', 'Information'),
4           ('success', 'Success'),
5           ('warning', 'Warning'),
6           ('error', 'Error'),
7       ]
8
9       notification_id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
10      recipient = models.ForeignKey(CustomUser, on_delete=models.CASCADE,
11  related_name='notifications')
12      message = models.CharField(max_length=255)
13      link = models.CharField(max_length=255, blank=True, null=True)
14      is_read = models.BooleanField(default=False)
15      notification_type = models.CharField(max_length=20, choices=TYPE_CHOICES, default='info')
16      created_at = models.DateTimeField(auto_now_add=True)
17
18      class Meta:
19          ordering = ['-created_at']
20
21      def __str__(self):
22          return f"Notification for {self.recipient.username}: {self.message}"
```

ภาพประกอบที่ 4.22 การพัฒนาระบบการบันทึกข้อมูลการแจ้งเตือน (models.py)

จากภาพประกอบที่ 4.22 โครงสร้างของไฟล์ models.py อธิบายการทำงานได้ดังนี้

บรรทัดที่ 1: ประกาศคลาส Notification สืบทอดจาก models.Model เพื่อสร้างตารางสำหรับเก็บข้อมูลการแจ้งเตือนในระบบ

บรรทัดที่ 2-7: กำหนดตัวแปร TYPE_CHOICES เพื่อระบุประเภทของการแจ้งเตือนที่มีได้ (เช่น แจ้งเพื่อทราบ, แจ้งความสำเร็จ, แจ้งเตือนระวัง, แจ้งข้อผิดพลาด)

บรรทัดที่ 9: กำหนดฟิลด์ notification_id เป็นคีย์หลัก (Primary Key) ชนิด UUID ซึ่งจะถูกสร้างค่าสุ่มให้อัตโนมัติและแก้ไขไม่ได้

บรรทัดที่ 10: กำหนดฟิลด์ recipient เป็น Foreign Key เชื่อมโยงกับตารางผู้ใช้ (CustomUser) เพื่อระบุว่าการแจ้งเตือนนี้เป็นของใคร โดยหากผู้ใช้ถูกลบ การแจ้งเตือนจะถูกลบตาม (CASCADE)

บรรทัดที่ 11: กำหนดฟิลด์ message สำหรับเก็บข้อความแจ้งเตือน ความยาวไม่เกิน 255 ตัวอักษร

บรรทัดที่ 12: กำหนดฟิลด์ link สำหรับเก็บลิงก์ URL ปลายทาง (ถ้ามี) เพื่อให้ผู้ใช้กดที่การแจ้งเตือนแล้วไปยังหน้านั้นๆ ได้

บรรทัดที่ 13: กำหนดฟิลด์ is_read เป็นชนิด Boolean เพื่อเก็บสถานะว่าผู้ใช้เปิดอ่านการแจ้งเตือนนี้หรือยัง (ค่าเริ่มต้นเป็น False)
บรรทัดที่ 14: กำหนดฟิลด์ notification_type สำหรับเก็บประเภทของการแจ้งเตือน โดยอ้างอิงค่าจาก TYPE_CHOICES
บรรทัดที่ 17-18: ประกาศคลาสย่อย Meta และกำหนด ordering = ['-created_at'] เพื่อให้ระบบดึงข้อมูลการแจ้งเตือนโดยเรียงจากเวลาล่าสุดขึ้นก่อนเสมอ

4.8.3 การพัฒนาระบบการจัดการสถานะการแจ้งเตือน

```
1  @login_required
2  def mark_notification_as_read(request, notification_id):
3      notification = get_object_or_404(Notification, notification_id=notification_id,
4  recipient=request.user)
5      notification.is_read = True
6      notification.save()
7
8      if notification.link:
9          return redirect(notification.link)
10     return redirect(request.META.get('HTTP_REFERER', 'home_page'))
```

ภาพประกอบที่ 4.23 การพัฒนาระบบการจัดการสถานะการแจ้งเตือน (views.py)

จากภาพประกอบที่ 4.23 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อจำกัดสิทธิ์การเข้าถึงฟังก์ชันนี้เฉพาะผู้ใช้ที่ล็อกอินแล้วเท่านั้น
บรรทัดที่ 2: ประกาศฟังก์ชัน mark_notification_as_read โดยรับค่า notification_id เพื่อระบุรายการแจ้งเตือนที่ต้องการเปิดอ่าน
บรรทัดที่ 3: ค้นหาข้อมูลการแจ้งเตือนจากฐานข้อมูล (get_object_or_404) โดยต้องตรงกับ ID ที่ระบุ และผู้รับ (recipient) ต้องเป็นผู้ใช้คนปัจจุบันเท่านั้น เพื่อความปลอดภัยของข้อมูลส่วนตัว
บรรทัดที่ 4-5: เปลี่ยนสถานะ is_read ให้เป็น True และบันทึกข้อมูลลงฐานข้อมูล เพื่อระบุว่าผู้ใช้ได้เปิดอ่านข้อความนี้แล้ว
บรรทัดที่ 7-8: ตรวจสอบว่าการแจ้งเตือนนี้มีลิงก์ปลายทาง (link) หรือไม่ หากมีให้เปลี่ยนเส้นทาง (redirect) ไปยังหน้านั้นทันที
บรรทัดที่ 9: กรณีไม่มีลิงก์ปลายทาง ให้เปลี่ยนเส้นทางกลับไปยังหน้าเดิมที่ผู้ใช้กดเข้ามา (HTTP_REFERER) หรือหากไม่พบให้กลับไปที่หน้าแรก (home_page)

4.9 การพัฒนาระบบสรุปเนื้อหา

4.9.1 การพัฒนาระบบการบันทึกข้อมูลสรุปเนื้อหา

```
1   class StudySummary(models.Model):
2       summary_id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
3       user = models.ForeignKey(CustomUser, on_delete=models.CASCADE)
4       session = models.OneToOneField(StudySession, on_delete=models.CASCADE,
5   related_name='summary') # 1 Session มี 1 สรุป
6       subject = models.ForeignKey(Subject, on_delete=models.CASCADE)
7       content = models.TextField() # เก็บ HTML ที่ AI ส่งมา
8       created_at = models.DateTimeField(auto_now_add=True)
9
10      class Meta:
11          db_table = 'study_summaries'
12          ordering = ['-created_at']
13
14      def __str__(self):
15          return f"Summary: {self.subject.name} - {self.created_at}"
```

ภาพประกอบที่ 4.24 การพัฒนาระบบการบันทึกข้อมูลสรุปเนื้อหา (models.py)

จากภาพประกอบที่ 4.24 โครงสร้างของไฟล์ models.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: ประกาศคลาส StudySummary สืบทอดจาก models.Model เพื่อสร้างตารางสำหรับเก็บข้อมูลสรุปบทเรียนที่ได้จากการประมวลผลของ AI
บรรทัดที่ 2: กำหนดฟิลด์ summary_id เป็นคีย์หลัก (Primary Key) ชนิด UUID ที่สร้างค่าสุ่มให้อัตโนมัติ
บรรทัดที่ 3: กำหนดฟิลด์ user เป็น Foreign Key เชื่อมโยงกับตารางผู้ใช้ (CustomUser) หากผู้ใช้ถูกลบ ข้อมูลสรุปนี้จะถูกลบไปด้วย
บรรทัดที่ 4: กำหนดฟิลด์ session เชื่อมโยงกับตาราง StudySession แบบ One-to-One (OneToOneField) ซึ่งเป็นการกำหนดเงื่อนไขว่า การเรียน 1 ครั้ง จะสามารถมีสรุปผลได้เพียง 1 ฉบับเท่านั้น
บรรทัดที่ 5: กำหนดฟิลด์ subject เป็น Foreign Key เชื่อมโยงกับตารางรายวิชา (Subject) เพื่อระบุว่าสรุปนี้เป็นของวิชาอะไร
บรรทัดที่ 6: กำหนดฟิลด์ content เป็นชนิด TextField เพื่อรองรับการเก็บข้อความสรุปเนื้อหาที่มีความยาวและอาจมีรูปแบบ HTML Tag ผสมอยู่
บรรทัดที่ 7: กำหนดฟิลด์ created_at เก็บวันและเวลาที่สร้างสรุปนี้โดยอัตโนมัติ
บรรทัดที่ 9-11: ประกาศคลาสย่อย Meta เพื่อกำหนดชื่อตารางเป็น 'study_summaries' และเรียงลำดับข้อมูลจากล่าสุดไปเก่าสุด (-created_at)

4.9.2 การพัฒนาระบบ API สำหรับเรียก AI สรุปเนื้อหา

```
1   @login_required
2   def get_session_summary(request, session_id):
3       """
4       API สำหรับเรียก AI สรุปเนื้อหาและบันทึกลง DB (ใช้คู่กับ Popup Loading)
5       """
6       if request.method == 'GET':
7           try:
8               session = get_object_or_404(StudySession, session_id=session_id, user=request.user)
9
10              summary_obj, created = StudySummary.objects.get_or_create(
11                  session=session,
12                  defaults={
13                      'user': request.user,
14                      'subject': session.subject,
15                      'content': ''
16                  }
17              )
18
19              if not summary_obj.content:
20                  ai_content = generate_content_summary(session.subject.name, session.topic)
21                  summary_obj.content = ai_content
22                  summary_obj.save()
23
24              return JsonResponse({'success': True})
25
26          except Exception as e:
27              return JsonResponse({'success': False, 'error': str(e)})
28
29      return JsonResponse({'success': False, 'error': 'Invalid request'})
```

ภาพประกอบที่ 4.25 การพัฒนาระบบ API สำหรับเรียก AI สรุปเนื้อหา (views.py)

จากภาพประกอบที่ 4.25 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อตรวจสอบสิทธิ์การเข้าใช้งาน โดยผู้ใช้ต้องล็อกอินก่อนจึงจะเรียกใช้งาน API นี้ได้
บรรทัดที่ 2: ประกาศฟังก์ชัน get_session_summary ซึ่งทำหน้าที่เป็น API สำหรับรับคำขอจากหน้าเว็บ (Frontend) เพื่อให้ AI ทำการสรุปเนื้อหา
บรรทัดที่ 5: ตรวจสอบรูปแบบการร้องขอข้อมูลว่าเป็นแบบ GET หรือไม่

บรรทัดที่ 7: ดึงข้อมูลการเรียน (StudySession) จากฐานข้อมูล โดยตรวจสอบว่า session_id ตรงกันและต้องเป็นของผู้ใช้คนนั้นจริงๆ (user=request.user) เพื่อความปลอดภัย
บรรทัดที่ 9-16: ใช้คำสั่ง get_or_create เพื่อตรวจสอบว่ามีสรุปของ Session นี้อยู่แล้วหรือไม่ หากยังไม่มีให้สร้างข้อมูลเปล่าเตรียมไว้ (ป้องกันการสร้างข้อมูลซ้ำซ้อน)
บรรทัดที่ 18-21: ตรวจสอบว่าเนื้อหาสรุปยังว่างอยู่หรือไม่ หากว่าง ให้เรียกใช้ฟังก์ชัน generate_content_summary (จากไฟล์ ai_service.py) เพื่อให้ AI สร้างเนื้อหา แล้วจึงบันทึกลงฐานข้อมูล
บรรทัดที่ 23: ส่งค่ากลับในรูปแบบ JSON (JsonResponse) เพื่อแจ้งให้ระบบหน้าบ้าน (JavaScript) ทราบว่าการทำงานเสร็จสมบูรณ์ (success: True)

4.9.3 การพัฒนาระบบหน้าแสดงสรุปเนื้อหา

```
1   @login_required
2   def study_summary_view(request, session_id):
3       """
4       หน้าแสดงสรุปเนื้อหา (แยกออกมาเป็นหน้าใหม่)
5       """
6       session = get_object_or_404(StudySession, session_id=session_id, user=request.user)
7
8       summary_obj, created = StudySummary.objects.get_or_create(
9           session=session,
10          defaults={
11              'user': request.user,
12              'subject': session.subject,
13              'content': ''
14          }
15      )
16
17      if created or not summary_obj.content:
18          ai_content = generate_content_summary(session.subject.name, session.topic)
19
20          summary_obj.content = ai_content
21          summary_obj.save()
22
23      return render(request, 'core/summary_detail.html', {'summary': summary_obj})
```

ภาพประกอบที่ 4.26 การพัฒนาระบบหน้าแสดงสรุปเนื้อหา (views.py)

จากภาพประกอบที่ 4.26 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อจำกัดสิทธิ์การเข้าถึงหน้านี้เฉพาะผู้ใช้ที่ล็อกอินแล้วเท่านั้น
บรรทัดที่ 2: ประกาศฟังก์ชัน study_summary_view โดยรับค่า session_id เพื่อใช้ระบุว่าจะดูสรุปของการเรียนครั้งไหน

บรรทัดที่ 6: ค้นหาและดึงข้อมูลการเรียน (StudySession) จากฐานข้อมูล โดยต้องตรงกับ ID ที่ระบุและเป็นของผู้ใช้คนปัจจุบัน (request.user)
บรรทัดที่ 8-14: ใช้คำสั่ง get_or_create เพื่อค้นหาข้อมูลสรุป (StudySummary) ในระบบ หากพบจะดึงมาใช้ แต่หากไม่พบจะสร้างข้อมูลใหม่ให้ทันทีโดยกำหนดค่าเริ่มต้นให้ว่างไว้ก่อน
บรรทัดที่ 17: ตรวจสอบเงื่อนไขว่า หากเป็นข้อมูลที่เพิ่งถูกสร้างขึ้นใหม่ (created) หรือข้อมูลเดิมยังไม่มีเนื้อหา (not summary_obj.content) ระบบจึงจะเริ่มกระบวนการเรียก AI (เพื่อประหยัด Resource ไม่ให้เรียกซ้ำถ้ามีอยู่แล้ว)
บรรทัดที่ 18-21: เรียกใช้ฟังก์ชัน generate_content_summary ส่งชื่อวิชาและหัวข้อไปให้ AI ประมวลผล เมื่อได้ผลลัพธ์กลับมาจะบันทึกลงฟิลด์ content และสั่ง save() ลงฐานข้อมูล
บรรทัดที่ 24: ส่งข้อมูลออบเจกต์ summary ไปแสดงผลรายละเอียดที่หน้า core/summary_detail.html

4.9.4 การพัฒนาระบบหน้าประวัติการสรุปเนื้อหา

```
1  @login_required
2  def summary_history_view(request):
3      """
4      หน้ารวมรายการสรุปทั้งหมดที่เคยทำ
5      """
6      summaries = StudySummary.objects.filter(user=request.user).select_related('subject')
7      return render(request, 'core/summary_list.html', {'summaries': summaries})
```

ภาพประกอบที่ 4.27 การพัฒนาระบบหน้าประวัติการสรุปเนื้อหา (views.py)

จากภาพประกอบที่ 4.27 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อตรวจสอบสิทธิ์ โดยอนุญาตให้เฉพาะผู้ใช้ที่ล็อกอินแล้วเท่านั้นที่สามารถเข้าถึงหน้านี้ได้
บรรทัดที่ 2: ประกาศฟังก์ชัน summary_history_view สำหรับแสดงรายการสรุปผลการเรียนย้อนหลังทั้งหมดที่ผู้ใช้เคยทำไว้
บรรทัดที่ 6: ดึงข้อมูลสรุป (StudySummary) ของผู้ใช้คนปัจจุบัน โดยใช้คำสั่ง select_related('subject') เพื่อดึงข้อมูลรายวิชาที่เกี่ยวข้องมาพร้อมกันในคำสั่งเดียว (Database Optimization) ช่วยให้การแสดงผลชื่อวิชาในหน้าเว็บรวดเร็วขึ้น
บรรทัดที่ 7: ส่งข้อมูลรายการสรุปทั้งหมด (summaries) ไปแสดงผลที่หน้าจอ core/summary_list.html

4.10 การพัฒนาระบบสร้างแบบทดสอบ

4.10.1 การพัฒนาระบบการบันทึกข้อมูลแบบทดสอบ

```
1   class Quiz(models.Model):
2       quiz_id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
3       activity = models.ForeignKey(PlanActivity, on_delete=models.CASCADE)
4       subject = models.ForeignKey('Subject', on_delete=models.CASCADE)
5       questions = models.JSONField()
6       user_answers = models.JSONField(null=True, blank=True)
7       score = models.IntegerField(null=True, blank=True)
8       taken_at = models.DateTimeField(auto_now_add=True)
9
10      class Meta:
11          db_table = 'quizzes'
```

ภาพประกอบที่ 4.28 การพัฒนาระบบการบันทึกข้อมูลแบบทดสอบ (models.py)

จากภาพประกอบที่ 4.28 โครงสร้างของไฟล์ models.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: ประกาศคลาส Quiz สืบทอดจาก models.Model เพื่อสร้างตารางสำหรับเก็บข้อมูลแบบทดสอบ
บรรทัดที่ 2: กำหนดฟิลด์ quiz_id เป็นคีย์หลัก (Primary Key) ชนิด UUID ที่สร้างค่าสุ่มให้อัตโนมัติและไม่สามารถแก้ไขได้
บรรทัดที่ 3-4: กำหนดฟิลด์ activity และ subject เป็น Foreign Key เชื่อมโยงกับตารางกิจกรรม (PlanActivity) และรายวิชา (Subject) เพื่อระบุว่าแบบทดสอบนี้เป็นของวิชาใดและกิจกรรมช่วงไหน
บรรทัดที่ 5-6: กำหนดฟิลด์ questions และ user_answers เป็นชนิด JSONField เพื่อเก็บชุดคำถามและคำตอบของผู้ใช้ในรูปแบบโครงสร้าง JSON ซึ่งช่วยให้จัดเก็บข้อมูลที่มีความซับซ้อนได้ง่าย
บรรทัดที่ 7: กำหนดฟิลด์ score เป็นตัวเลขจำนวนเต็มสำหรับเก็บคะแนนสอบ โดยอนุญาตให้เป็นค่าว่างได้ (null=True)
บรรทัดที่ 8: กำหนดฟิลด์ taken_at เก็บวันและเวลาที่เริ่มทำแบบทดสอบโดยอัตโนมัติ
บรรทัดที่ 10-11: ประกาศคลาสย่อย Meta เพื่อกำหนดชื่อตารางจริงในฐานข้อมูลให้ชื่อว่า 'quizzes'

4.10.2 การพัฒนาระบบการบันทึกข้อมูลผลการทดสอบ

```
1   class QuizResult(models.Model):
2       result_id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
3       user = models.ForeignKey(CustomUser, on_delete=models.CASCADE)
4       session = models.ForeignKey(StudySession, on_delete=models.CASCADE)
5
6       questions_data = models.JSONField()
7       user_answers = models.JSONField()
8
9       score = models.IntegerField()
10      total_questions = models.IntegerField()
11      created_at = models.DateTimeField(auto_now_add=True)
12
13      class Meta:
14          db_table = 'quiz_results'
15          ordering = ['-created_at']
```

ภาพประกอบที่ 4.29 การพัฒนาระบบการบันทึกข้อมูลผลการทดสอบ (models.py)

จากภาพประกอบที่ 4.29 โครงสร้างของไฟล์ models.py อธิบายการทำงานได้ดังนี้

บรรทัดที่ 1: ประกาศคลาส QuizResult สืบทอดจาก models.Model เพื่อสร้างตารางสำหรับเก็บประวัติและผลคะแนนการทำแบบทดสอบ

บรรทัดที่ 2: กำหนดฟิลด์ result_id เป็นคีย์หลัก (Primary Key) ชนิด UUID ที่สร้างค่าสุ่มให้อัตโนมัติ

บรรทัดที่ 3-4: กำหนดฟิลด์ user และ session เป็น Foreign Key เชื่อมโยงกับตารางผู้ใช้ (CustomUser) และตารางการเรียน (StudySession) เพื่อระบุว่าใครเป็นคนสอบและสอบในหัวข้อการเรียนใด

บรรทัดที่ 6-7: กำหนดฟิลด์ questions_data และ user_answers เป็นชนิด JSONField เพื่อบันทึกโจทย์คำถามที่ AI สุ่มมาให้และคำตอบที่ผู้ใช้เลือกตอบเก็บไว้ในรูปแบบ JSON

บรรทัดที่ 9-10: กำหนดฟิลด์ score และ total_questions เป็นตัวเลขจำนวนเต็ม เพื่อเก็บคะแนนที่ได้และจำนวนข้อสอบทั้งหมด

บรรทัดที่ 11: กำหนดฟิลด์ created_at เก็บวันและเวลาที่ส่งคำตอบแบบทดสอบโดยอัตโนมัติ

บรรทัดที่ 13-15: ประกาศคลาสย่อย Meta เพื่อกำหนดชื่อตารางในฐานข้อมูลเป็น 'quiz_results' และให้เรียงลำดับจากรายการล่าสุดไปเก่าสุด

4.10.3 การพัฒนาระบบ API สำหรับเรียก AI สร้างแบบทดสอบ

```
1   @login_required
2   def get_session_quiz(request, session_id):
3       if request.method == 'GET':
4           try:
5               session = StudySession.objects.get(session_id=session_id, user=request.user)
6               quiz_data = generate_quiz_questions(session.subject.name, session.topic)
7
8               if quiz_data:
9                   return JsonResponse({'success': True, 'quiz': quiz_data})
10              else:
11                  return JsonResponse({'success': False, 'error': 'AI could not generate quiz'})
12
13          except Exception as e:
14              return JsonResponse({'success': False, 'error': str(e)})
15
16      return JsonResponse({'success': False, 'error': 'Invalid request'})
```

ภาพประกอบที่ 4.30 การพัฒนาระบบ API สำหรับเรียก AI สร้างแบบทดสอบ (views.py)

จากภาพประกอบที่ 4.30 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อตรวจสอบสิทธิ์การเข้าใช้งาน โดยผู้ใช้ต้องล็อกอินก่อนจึงจะเรียกใช้งานฟังก์ชันนี้ได้
บรรทัดที่ 2: ประกาศฟังก์ชัน get_session_quiz ที่รับค่า session_id เพื่อใช้ระบุหัวข้อการเรียนที่จะนำมาสร้างแบบทดสอบ
บรรทัดที่ 3: ตรวจสอบรูปแบบการร้องขอข้อมูลจากหน้าเว็บว่าเป็นแบบ GET request หรือไม่
บรรทัดที่ 5: ดึงข้อมูลการเรียน (StudySession) จากฐานข้อมูล โดยตรวจสอบความถูกต้องของ session_id และต้องเป็นของผู้ใช้คนปัจจุบัน (request.user) เท่านั้น
บรรทัดที่ 6: เรียกใช้ฟังก์ชัน generate_quiz_questions (จากไฟล์ ai_service.py) โดยส่งชื่อวิชาและหัวข้อเรียนไปให้ AI ประมวลผลและสร้างโจทย์คำถาม
บรรทัดที่ 8-9: ตรวจสอบผลลัพธ์จาก AI หากได้ข้อมูลกลับมา (quiz_data) จะส่งข้อมูลแบบทดสอบกลับไปให้หน้าเว็บในรูปแบบ JSON (JsonResponse) พร้อมสถานะ success: True
บรรทัดที่ 10-11: กรณี AI ไม่สามารถสร้างโจทย์ได้ จะส่งค่าความผิดพลาดกลับไปแจ้งเตือนผู้ใช้
บรรทัดที่ 13-14: ใช้คำสั่ง try-except เพื่อดักจับข้อผิดพลาดอื่นๆ (เช่น หา Session ไม่เจอ) และส่ง Error Message กลับไปในรูปแบบ JSON เพื่อป้องกันระบบล่ม

4.10.4 การพัฒนาระบบการบันทึกผลการสอบ

```
1   @login_required
2   @require_POST
3   def submit_quiz_view(request):
4       """
5       API รับข้อมูลการสอบ บันทึก และส่งคืน URL สำหรับ redirect
6       """
7       try:
8           data = json.loads(request.body)
9           session_id = data.get('session_id')
10          questions = data.get('questions')
11          user_answers = data.get('answers')
12
13          session = get_object_or_404(StudySession, session_id=session_id, user=request.user)
14
15          score = 0
16          for i, q in enumerate(questions):
17              user_ans = user_answers[i]
18              if user_ans is not None and int(user_ans) == int(q['correct_index']):
19                  score += 1
20
21          quiz_result = QuizResult.objects.create(
22              user=request.user,
23              session=session,
24              questions_data=questions,
25              user_answers=user_answers,
26              score=score,
27              total_questions=len(questions)
28          )
29
30          return JsonResponse({
31              'success': True,
32              'redirect_url': reverse('quiz_result', args=[quiz_result.result_id])
33          })
34
35      except Exception as e:
35          return JsonResponse({'success': False, 'error': str(e)})
```

ภาพประกอบที่ 4.31 การพัฒนาระบบการบันทึกผลการสอบ (views.py)

จากภาพประกอบที่ 4.31 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1-2: เรียกใช้ Decorator @login_required เพื่อบังคับให้ผู้ใช้ต้องล็อกอินก่อน และ @require_POST เพื่อจำกัดให้ฟังก์ชันนี้รับเฉพาะการส่งข้อมูลแบบ POST เท่านั้น (เพื่อความปลอดภัยในการส่งข้อมูลคำตอบ)

บรรทัดที่ 3: ประกาศฟังก์ชัน submit_quiz_view เพื่อทำหน้าที่เป็น API สำหรับรับข้อมูลการสอบจากหน้าเว็บมาประมวลผล
บรรทัดที่ 6-9: แปลงข้อมูลที่ส่งมาในรูปแบบ JSON (request.body) ให้เป็น Python Dictionary และดึงค่าตัวแปรสำคัญออกมา ได้แก่ รหัสเซสชัน (session_id), ข้อมูลคำถาม (questions), และคำตอบที่ผู้ใช้เลือก (answers)
บรรทัดที่ 10: ตรวจสอบความถูกต้องของเซสชัน (get_object_or_404) ว่ามีอยู่จริงและเป็นของผู้ใช้คนปัจจุบันหรือไม่
บรรทัดที่ 11-15: วนลูปเพื่อตรวจคำตอบทีละข้อ โดยเปรียบเทียบคำตอบของผู้ใช้ (user_ans) กับดัชนีเฉลย (correct_index) หากตรงกันให้เพิ่มคะแนน (score) ขึ้นทีละ 1
บรรทัดที่ 16-23: บันทึกผลการสอบลงในตาราง QuizResult โดยเก็บข้อมูลครบถ้วนทั้งคะแนน, คำถาม, และคำตอบของผู้ใช้ เพื่อใช้แสดงผลย้อนหลัง
บรรทัดที่ 25-28: ส่งข้อมูลตอบกลับแบบ JSON (JsonResponse) แจ้งสถานะความสำเร็จ (success: True) พร้อมส่ง URL ปลายทาง (redirect_url) เพื่อให้ระบบหน้าบ้านพาผู้ใช้เปลี่ยนหน้าไปดูผลคะแนน (quiz_result)

4.10.5 การพัฒนาระบบหน้าแสดงผลคะแนนสอบ

```
1   @login_required
2   def quiz_result_view(request, result_id):
3       """ หน้าแสดงผลคะแนน """
4       result = get_object_or_404(QuizResult, result_id=result_id, user=request.user)
5       percentage = int((result.score / result.total_questions) * 100)
6
7       return render(request, 'core/quiz_result.html', {
8           'result': result,
9           'percentage': percentage
10      })
```

ภาพประกอบที่ 4.32 การพัฒนาระบบหน้าแสดงผลคะแนนสอบ (views.py)

จากภาพประกอบที่ 4.32 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อจำกัดสิทธิ์การเข้าถึงหน้านี้เฉพาะผู้ใช้ที่ล็อกอินแล้วเท่านั้น
บรรทัดที่ 2: ประกาศฟังก์ชัน quiz_result_view โดยรับพารามิเตอร์ result_id เพื่อระบุรายการผลสอบที่ต้องการนำมาแสดง
บรรทัดที่ 4: ค้นหาข้อมูลผลสอบ (QuizResult) จากฐานข้อมูลที่ตรงกับ ID ที่ระบุ และตรวจสอบความปลอดภัย โดยต้องเป็นข้อมูลของผู้ใช้คนปัจจุบันเท่านั้น (request.user) หากไม่พบจะแสดงหน้าแจ้งเตือน (404 Not Found)
บรรทัดที่ 5: คำนวณคะแนนที่ได้ออกมาเป็นเปอร์เซ็นต์ (ร้อยละ) และแปลงค่าเป็นจำนวนเต็ม (int) เพื่อความสวยงามในการแสดงผล
บรรทัดที่ 7-10: ส่งข้อมูลผลสอบ (result) และค่าเปอร์เซ็นต์ที่คำนวณได้ (percentage) ไปยังไฟล์ Template core/quiz_result.html เพื่อแสดงผลหน้าจอสรุปคะแนน

4.10.6 การพัฒนาระบบหน้าแสดงเฉลยแบบทดสอบ

```
1   @login_required
2   def quiz_solution_view(request, result_id):
3       """ หน้าดูเฉลยละเอียด """
4       result = get_object_or_404(QuizResult, result_id=result_id, user=request.user)
5
6       solution_data = []
7       for i, q in enumerate(result.questions_data):
8           user_ans_index = result.user_answers[i]
9           solution_data.append({
10              'question': q['question'],
11              'options': q['options'],
12              'correct_index': q['correct_index'],
13              'user_index': user_ans_index,
14              'is_correct': (user_ans_index is not None) and (int(user_ans_index) == int(q['correct_index']))
15          })
16
17      return render(request, 'core/quiz_solution.html', {
18          'result': result,
19          'solution_data': solution_data
20      })
```

ภาพประกอบที่ 4.33 การพัฒนาระบบหน้าแสดงเฉลยแบบทดสอบ (views.py)

จากภาพประกอบที่ 4.33 โครงสร้างของไฟล์ views.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1: เรียกใช้ Decorator @login_required เพื่อจำกัดสิทธิ์การเข้าถึงหน้านี้เฉพาะผู้ใช้ที่ล็อกอินแล้วเท่านั้น
บรรทัดที่ 2: ประกาศฟังก์ชัน quiz_solution_view โดยรับพารามิเตอร์ result_id เพื่อระบุผลสอบที่ต้องการดูเฉลย
บรรทัดที่ 4: ค้นหาข้อมูลผลสอบ (QuizResult) จากฐานข้อมูล โดยต้องตรงกับ ID ที่ระบุและเป็นของผู้ใช้คนปัจจุบัน (request.user) เพื่อป้องกันการแอบดูเฉลยของผู้อื่น
บรรทัดที่ 6-8: สร้างตัวแปรลิสต์ว่าง (solution_data) และเริ่มวนลูปรายการคำถามที่บันทึกไว้ (questions_data) พร้อมกับดึงคำตอบที่ผู้ใช้เลือก (user_answers) ในลำดับเดียวกันออกมา
บรรทัดที่ 9-15: จัดเตรียมข้อมูลของแต่ละข้อเพื่อส่งไปแสดงผล ประกอบด้วย โจทย์, ตัวเลือก, เฉลย, คำตอบของผู้ใช้ และตัวแปร is_correct ที่คำนวณว่าผู้ใช้ตอบถูกหรือไม่ (True/False)
บรรทัดที่ 17-20: ส่งข้อมูลทั้งหมด (solution_data) ไปยังไฟล์ Template core/quiz_solution.html เพื่อแสดงรายละเอียดเฉลยทีละข้อ

4.10 การพัฒนาระบบยันตัวตนด้วยอีเมล (backends.py)

```
1   UserModel = get_user_model()
2
3   class EmailBackend(ModelBackend):
4       def authenticate(self, request, username=None, password=None, **kwargs):
5           try:
6               user = UserModel.objects.get(email=username)
7           except UserModel.DoesNotExist:
8               return None
9
10          if user.check_password(password):
11              return user
12          return None
13
14      def get_user(self, user_id):
15          try:
16              return UserModel.objects.get(pk=user_id)
17          except UserModel.DoesNotExist:
18              return None
```

ภาพประกอบที่ 4.34 การพัฒนาระบบ

จากภาพประกอบที่ 4.34 โครงสร้างของไฟล์ backends.py อธิบายการทำงานได้ดังนี้
บรรทัดที่ 1-3: สร้าง Class EmailBackend สืบทอดมาจาก ModelBackend เพื่อปรับเปลี่ยนวิธีการตรวจสอบสิทธิ์
บรรทัดที่ 4-6: ในฟังก์ชัน authenticate เปลี่ยนจากการค้นหาด้วย Username เป็นการค้นหาด้วย email ในฐานข้อมูลผู้ใช้
บรรทัดที่ 8-10: หากพบอีเมลผู้ใช้ ให้ทำการตรวจสอบรหัสผ่านด้วย check_password
บรรทัดที่ 11: หากรหัสผ่านถูกต้อง ให้คืนค่า Object user กลับไปเพื่อให้ระบบทำการ Login
บรรทัดที่ 13-17: ฟังก์ชัน get_user ใช้สำหรับดึงข้อมูลผู้ใช้จาก User ID เพื่อคงสถานะการล็อกอิน

บทที่ 5

การทดสอบระบบ

ขั้นตอนการทดสอบระบบเพื่อตรวจสอบการทำงานของแต่ละฟังก์ชันกันรวมถึงตรวจสอบความถูกต้องของการทำงาน ทั้งระบบเพื่อให้การพัฒนาโปรแกรมเป็นไปอย่างมีประสิทธิภาพ และทดสอบระบบเพื่อหาข้อผิดพลาดของระบบ โดยในการทดสอบระบบจะแบ่งเป็น 9 ส่วน ดังนี้

5.1 การทดสอบฟังก์ชันสมัครสมาชิกและลงชื่อเข้าสู่ระบบ

5.2 การทดสอบฟังก์ชันการเพิ่มรายวิชาและไฟล์การเรียนการสอบ

5.3 การทดสอบฟังก์ชันการกำหนดเวลาว่าง

5.4 การทดสอบฟังก์ชันตั้งค่าการอ่าน

5.5 การทดสอบฟังก์ชันการจัดตารางอัตโนมัติด้วย AI

5.6 การทดสอบตารางแผนการอ่าน, แผนการอ่านวันนี้, สถิติภาพรวม, การสอบที่ใกล้จะถึง

5.7 การทดสอบฟังก์ชันการจับเวลาในการอ่าน

5.8 การทดสอบฟังก์ชันการสรุปเนื้อหาและสร้างแบบทดสอบ

5.9 การทดสอบฟังก์ชันข้อเสนอแนะและแจ้งปัญหา

5.1 การทดสอบฟังก์ชันสมัครสมาชิกและลงชื่อเข้าสู่ระบบ

5.1.1 การทดสอบฟังก์ชันลงทะเบียน

ตารางที่ 5.1 ผลการทดสอบการลงทะเบียน

การทดสอบ	เงื่อนไขการทดสอบ	ผลการทดสอบ
สมัครสมาชิกด้วยข้อมูลถูกต้อง	กรอก Username, Email, Password ครบถ้วน	ระบบบันทึกข้อมูลและเข้าสู่ระบบอัตโนมัติ
สมัครสมาชิกด้วยอีเมลซ้ำ	ใช้อีเมลที่มีอยู่ในระบบแล้ว	ระบบแจ้งเตือนว่าอีเมลนี้ถูกใช้งานแล้ว
รหัสผ่านไม่ตรงกัน	กรอกรหัสผ่านยืนยันไม่ตรงกับช่องแรก	ระบบแจ้งเตือนให้กรอกรหัสผ่านใหม่

5.1.2 การทดสอบฟังก์ชันลงชื่อเข้าสู่ระบบ

ตารางที่ 5.2 ผลการทดสอบการลงชื่อเข้าสู่ระบบ

การทดสอบ	เงื่อนไขการทดสอบ	ผลการทดสอบ
เข้าสู่ระบบถูกต้อง	กรอก Email และ Password ถูกต้อง	เข้าสู่หน้าแรกได้สำเร็จ
เข้าสู่ระบบผิดพลาด	กรอกรหัสผ่านผิด	ระบบแสดงข้อความแจ้งเตือน

5.2 การทดสอบฟังก์ชันการเพิ่มรายวิชาและไฟล์การเรียนการสอบ

ตารางที่ 5.3 ผลการทดสอบการเพิ่มรายวิชาและไฟล์การเรียนการสอบ

การทดสอบ	เงื่อนไขการทดสอบ	ผลการทดสอบ
การเพิ่มรายวิชาใหม่	กรอกชื่อวิชา ระดับความยาก และวันสอบ ครบถ้วน	ระบบบันทึกรายวิชาลงฐานข้อมูลและแสดงในรายการวิชาได้ถูกต้อง
การอัปโหลดไฟล์ประกอบการเรียน	เลือกไฟล์นามสกุลที่รองรับ (.pdf, .docx, .pptx, .txt) และขนาดไม่เกิน 10MB	ระบบบันทึกไฟล์ และเชื่อมโยงกับรายวิชาได้ถูกต้อง
การป้องกันไฟล์ผิดประเภท	เลือกไฟล์นามสกุลที่ไม่รองรับ (เช่น .exe, .bat)	ระบบปฏิเสธการอัปโหลดและแจ้งเตือนข้อผิดพลาด
การลบไฟล์ประกอบการเรียน	กดปุ่มลบไฟล์ที่ต้องการในหน้ารายละเอียดวิชา	ระบบลบไฟล์ออกจากฐานข้อมูลได้สำเร็จ

5.3 การทดสอบฟังก์ชันการกำหนดเวลาว่าง

ตารางที่ 5.4 ผลการทดสอบการกำหนดเวลาว่าง

การทดสอบ	เงื่อนไขการทดสอบ	ผลการทดสอบ
การเลือกช่วงเวลาว่าง	คลิกเลือกช่วงเวลา (Slot) ในตาราง 7 วัน (จันทร์-อาทิตย์)	ระบบแสดงสถานะ สีตามช่วงเวลาบนช่องที่เลือกได้ถูกต้อง
การยกเลิกช่วงเวลาว่าง	คลิกซ้ำที่ช่วงเวลาเดิมที่เลือกไว้	ระบบเปลี่ยนสถานะกลับเป็น สีขาว ได้ถูกต้อง
การบันทึกเวลาว่าง	กดปุ่ม "บันทึกและถัดไป" หลังจากเลือกเวลา	ระบบบันทึกช่วงเวลา (Day, Hour) ลงในตาราง UserAvailability ได้ถูกต้อง

5.4 การทดสอบฟังก์ชันตั้งค่าการอ่าน

ตารางที่ 5.5 ผลการทดสอบการตั้งค่าการอ่าน

การทดสอบ	เงื่อนไขการทดสอบ	ผลการทดสอบ
การกำหนดระยะเวลาอ่านต่อครั้ง	ระบุจำนวนนาทีขั้นต่ำและสูงสุดต่อหนึ่ง Session	ระบบบันทึกค่า Duration และนำไปใช้ในการคำนวณตารางเรียนได้ถูกต้อง
การกำหนดระยะเวลาพักต่อครั้ง	ระบุจำนวนนาทีขั้นต่ำและสูงสุดต่อหนึ่ง Session	ระบบบันทึกค่า Duration และนำไปใช้ในการคำนวณตารางเรียนได้ถูกต้อง

5.5 การทดสอบฟังก์ชันการจัดตารางอัตโนมัติด้วย AI

ตารางที่ 5.6 ผลการทดสอบการจัดตารางอัตโนมัติด้วย AI

การทดสอบ	เงื่อนไขการทดสอบ	ผลการทดสอบ
การสร้างตารางเรียน (Generate)	มีข้อมูลรายวิชาและเวลาว่างในระบบครบถ้วน	ระบบประมวลผลและสร้างตารางเรียน (Study Session) ลงในปฏิทินได้โดยไม่ทับซ้อนกัน
การตรวจสอบความขัดแย้ง	ตรวจสอบช่วงเวลาเรียนที่จัดขึ้น	ระบบไม่จัดตารางเรียนในเวลาที่ผู้ใช้ไม่ได้ระบุว่า "ว่าง"
การเชื่อมต่อ Google Calendar	กดปุ่ม Sync Google Calendar (เชื่อมต่อบัญชีแล้ว)	ระบบส่งข้อมูลตารางเรียนไปยังปฏิทิน Google ของผู้ใช้ได้สำเร็จ

5.6 การทดสอบตารางแผนการอ่าน, แผนการอ่านวันนี้, สถิติภาพรวม, การสอบที่ใกล้จะถึง

ตารางที่ 5.7 ผลการทดสอบแผนการอ่าน, แผนการอ่านวันนี้, สถิติภาพรวม, การสอบที่ใกล้จะถึง

การทดสอบ	เงื่อนไขการทดสอบ	ผลการทดสอบ
การแสดงแผนการอ่านวันนี้	เมีตารางเรียนที่ตรงกับวันที่ปัจจุบัน	ระบบแสดงรายการวิชาที่ต้องอ่านวันนี้ พร้อมเวลาเริ่ม-จบ ได้ถูกต้อง
การแสดงการสอบที่ใกล้จะถึง	มีข้อมูลวันที่สอบในรายวิชา	ระบบแสดงรายวิชาที่กำลังจะสอบเรียงลำดับจากเร็วที่สุดได้ถูกต้อง
การแสดงสถิติ	มีประวัติการอ่านที่ผ่านมา	ระบบแสดงการสรุปเวลาเรียนแยกตามรายวิชาได้ถูกต้อง

5.7 การทดสอบฟังก์ชันการจับเวลาในการอ่าน

ตารางที่ 5.8 ผลการทดสอบการการจับเวลาในการอ่าน

การทดสอบ	เงื่อนไขการทดสอบ	ผลการทดสอบ
การเริ่มจับเวลา	กดปุ่ม เริ่ม ในหน้าจับเวลา	ตัวเลขเวลานับถอยหลังทำงานและสถานะเปลี่ยนเป็น "กำลังโฟกัส"
การพักชั่วคราว (Pause/Resume)	กดปุ่ม หยุด และ เริ่มต่อ	เวลาหยุดเดินชั่วคราว และสามารถนับต่อจากเดิมได้เมื่อกด เริ่มต่อ
การบันทึกผลเมื่อครบเวลา	เวลาจับเวลาเดินจนครบกำหนด	ระบบแจ้งเตือนหมดเวลา และเปลี่ยนสถานะ Session เป็น "สำเร็จ"

5.8 การทดสอบฟังก์ชันการสรุปเนื้อหาและสร้างแบบทดสอบ

ตารางที่ 5.9 ผลการทดสอบการการสรุปเนื้อหาและสร้างแบบทดสอบ

การทดสอบ	เงื่อนไขการทดสอบ	ผลการทดสอบ
การสรุปเนื้อหา (AI Summary)	กดปุ่ม "สรุปเนื้อหา" ในวิชาที่มีหัวข้อเรียน	ระบบแสดง Popup และแสดงข้อความสรุปที่ได้จาก AI (Gemini) ได้ถูกต้อง
การสร้างแบบทดสอบ (Generate Quiz)	กดปุ่ม "ทำแบบทดสอบ" ในวิชาที่เรียนจบแล้ว	ระบบสร้างโจทย์คำถามแบบปรนัย 4 ตัวเลือก จำนวน 5 ข้อ ได้สำเร็จ
การตรวจคำตอบและคำนวณคะแนน	เลือกคำตอบครบทุกข้อและกดส่งคำตอบ	ระบบแสดงผลคะแนนที่ได้ และเฉลยคำตอบที่ถูกต้องได้แม่นยำ

5.9 การทดสอบฟังก์ชันข้อเสนอแนะและแจ้งปัญหา

ตารางที่ 5.15 ผลการทดสอบข้อเสนอแนะและแจ้งปัญหา

การทดสอบ	เงื่อนไขการทดสอบ	ผลการทดสอบ
การส่งข้อเสนอแนะ	กรอกหัวข้อและรายละเอียดข้อเสนอแนะ	ระบบบันทึกข้อมูลลงฐานข้อมูล Feedback และแสดงข้อความขอบคุณ
การตรวจสอบความครบถ้วน	กดส่งโดยไม่กรอกรายละเอียด	ระบบแจ้งเตือนให้กรอกข้อมูลให้ครบถ้วนก่อนส่ง

บทที่ 6
สรุปและข้อเสนอแนะ

การดำเนินโครงงานเพื่อพัฒนาระบบเว็บแอป ระบบจัดตารางอ่านหนังสืออัตโนมัติ Smart Study Planner พบว่าระบบสามารถทำงานได้ตามที่วิเคราะห์และออกแบบไว้ พบปัญหาและอุปสรรคระหว่างการพัฒนา ในบทนี้ผู้พัฒนาจึงขอสรุปความสามารถของระบบชี้แจงปัญหาและอุปสรรค พร้อมเสนอแนวทางในการพัฒนาระบบเว็บแอป ระบบจัดตารางอ่านหนังสืออัตโนมัติ Smart Study Planner

6.1 สรุปความสามารถของระบบ

3.2.1 ความต้องการของโปรแกรม (Functional Requirement)

3.2.1.1 ผู้ใช้งานทั่วไป/นักศึกษา(User/Student)

- สมัครสมาชิก
- ลงชื่อเข้าสู่ระบบ
- กรอกข้อมูลวิชา (ชื่อวิชา, ความยาก, วันที่สอบ)
- เพิ่มไฟล์การเรียนการสอน(.pdf .txt)
- กำหนดเวลาว่างในแต่ละวัน/สัปดาห์
- สร้างตารางอ่านหนังสืออัตโนมัติ
- ดูแผนการอ่าน (รายวัน / ปฏิทิน)
- ติ๊กว่าอ่านจบแล้วในแต่ละวิชา
- ทำแบบทดสอบหลังเรียน
- สรุปเนื้อหาหลังอ่าน
- ดูความคืบหน้าของแผน
- แก้ไขวิชา และ สร้างแผนใหม่
- รับการแจ้งเตือน (เวลาอ่านใกล้ถึง,มีเวลาเพิ่ม หรือใกล้ถึงวันสอบ)
- เชื่อมต่อ Google Calendar
- แสดงความคิดเห็นหรือให้คำแนะนำเพิ่มเติมเกี่ยวกับระบบ

3.2.1.2 ระบบ (System)

- ประมวลผลข้อมูลวิชา + เวลาว่าง
- คำนวณและจัดตารางอ่านหนังสือให้อัตโนมัติ
- ปรับแผนอ่านใหม่เมื่อมีการเปลี่ยนแปลง
- คำนวณเปอร์เซ็นต์ความคืบหน้าตามวิชา ที่ติ๊กแล้ว
- สร้างแบบทดสอบหลังเรียน
- สรุปเนื้อหาหลังผู้ใช้ติ๊กว่าอ่านแล้ว
- แจ้งเตือนความคืบหน้าในการอ่านแต่ละวิชา
- แจ้งเตือนเมื่อใกล้ถึงวันสอบ

3.2.2 ความต้องการด้านคุณภาพ (Non-functional Requirements)

- ระบบต้องสามารถสร้างตารางการอ่านได้
- ระบบต้องสามารถสรุปเนื้อหาได้
- ระบบต้องเชื่อมต่ออินเทอร์เน็ตก่อนใช้งานทุกครั้ง
- ระบบต้องมีการออกแบบหน้าจอที่ใช้งานง่ายและเข้าถึงฟีเจอร์ต่าง ๆ ได้อย่างรวดเร็ว
- ระบบมีการรักษาความปลอดภัยของรหัสผ่านของผู้ใช้งานrd

3.2.3 ความสามารถของระบบ (System Requirements)

- ระบบต้องสามารถสร้างตารางการอ่านได้อย่างถูกต้อง
- ระบบต้องสามารถ
- ระบบต้องสามารถส่งการแจ้งเตือนเพื่อเตือนให้ผู้ใช้รู้ได้ว่าต้องอ่านวิชาไหนบ้าง
- ระบบต้องสามารถให้ผู้ใช้ จัดการข้อมูลส่วนตัว ของตนเองได้ เช่น การอัปเดตข้อมูลส่วนตัว, การลบข้อมูล หรือ การเพิ่มข้อมูลส่วนตัว
- ระบบต้องสามารถสรุปเนื้อหาสำคัญให้ผู้ใช้เข้าใจง่าย
- ระบบต้องสามารถสร้างแบบทดสอบที่ถูกต้องได้

6.2 ปัญหาและอุปสรรคในการพัฒนา

1. ความซับซ้อนของ Algorithm: การจัดตารางเรียนให้ลงล็อกพอดีกับเวลาว่างที่มีจำกัด มีความซับซ้อนสูงในกรณีที่ผู้ใช้มีเวลาว่างน้อยมาก
2. การเชื่อมต่อ API: การเชื่อมต่อ Google Calendar API และ Gemini AI มีข้อจำกัดเรื่องโควตาการใช้งานและ Latency ในการตอบกลับบางช่วงเวลา
3. การจัดการ Timezone: การแปลงเวลาให้ตรงกันระหว่าง Server, Database และ Browser ของผู้ใช้มีความซับซ้อน

6.3 แนวทางในการพัฒนาต่อ

1. เพิ่มฟีเจอร์ Social: ให้ผู้ใช้สามารถแชร์ตารางเรียนหรือจับกลุ่มติวหนังสือกับเพื่อนได้
2. ระบบ Gamification: เพิ่มระบบเก็บแต้ม หรือปลดล็อก Badge เมื่ออ่านหนังสือครบตามเป้าหมาย
3. รองรับ Mobile Application: พัฒนาเป็นแอปพลิเคชันบนมือถือ (iOS/Android) เพื่อการแจ้งเตือนที่แม่นยำและเข้าถึงง่ายขึ้น

บรรณานุกรม

[1] Django Software Foundation. (2024). *Django Documentation*. Retrieved from https://docs.djangoproject.com/en/5.0/

[2] Google AI. (2024). *Gemini API Overview*. Retrieved from

https://ai.google.dev/docs

[3] Google Developers. (2024). *Google Calendar API*. Retrieved from https://developers.google.com/calendar

[4] Google Developers. (2024). *Google Identity: OAuth 2.0*. Retrieved from https://developers.google.com/identity/protocols/oauth2

[5] Mozilla Developer Network. (2024). *MDN Web Docs: HTML & CSS*. Retrieved from https://developer.mozilla.org/

[6] Python Software Foundation. (2025). *Python 3.12.1 documentation*. Retrieved from https://docs.python.org/3/

[7] W3Schools. (2024). *Python Tutorial*. Retrieved from

https://www.w3schools.com/python/

ภาคผนวก

ภาคผนวก ก
การติดตั้งเครื่องมือที่ใช้พัฒนาโปรแกรม

การติดตั้งเครื่องมือที่ใช้ในการพัฒนาเว็บแอพพลิเคชันระบบจัดการลำดับเข้าสอบสัมภาษณ์แบบออนไลน์มีโปรแกรมที่จำเป็นในการพัฒนาระบบดังต่อไปนี้

การติดตั้ง Visual Studio Code

การติดตั้ง Python

การติดตั้ง Git

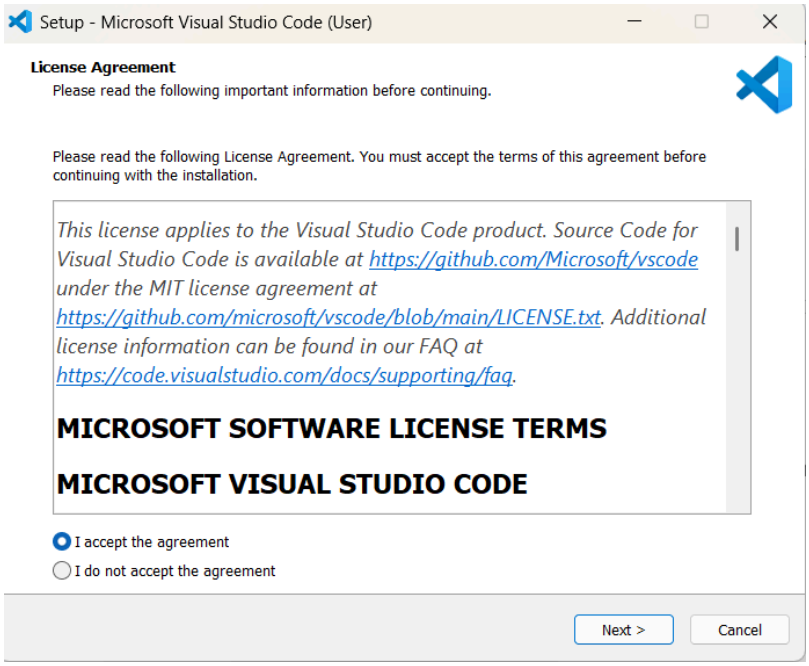
ก.1 ขั้นตอนการติดตั้งโปรแกรม Visual Studio Code หรือ VS Code

1. สามารถดาวน์โหลด Visual Studio Code ได้ที่ https://code.visualstudio.com/download โปรแกรม VSCode โดยเลือกให้ตรงกับ OS ของเครื่องคอมพิวเตอร์



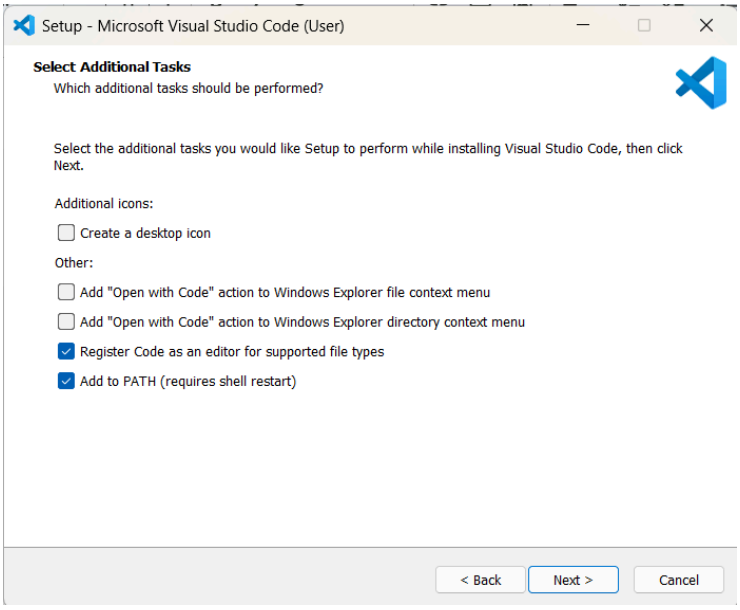
ภาพที่ ก.1 หน้าเว็บดาวน์โหลด Visual Studio Code

1. หลังดาวน์โหลดโปรแกรมเสร็จดับเบิ้ลคลิก หรือคลิกขวาและกด “Open” โปรแกรมที่ดาวน์โหลดมา ระบบจะแสดงหน้าต่างดังรูปที่ ก.2



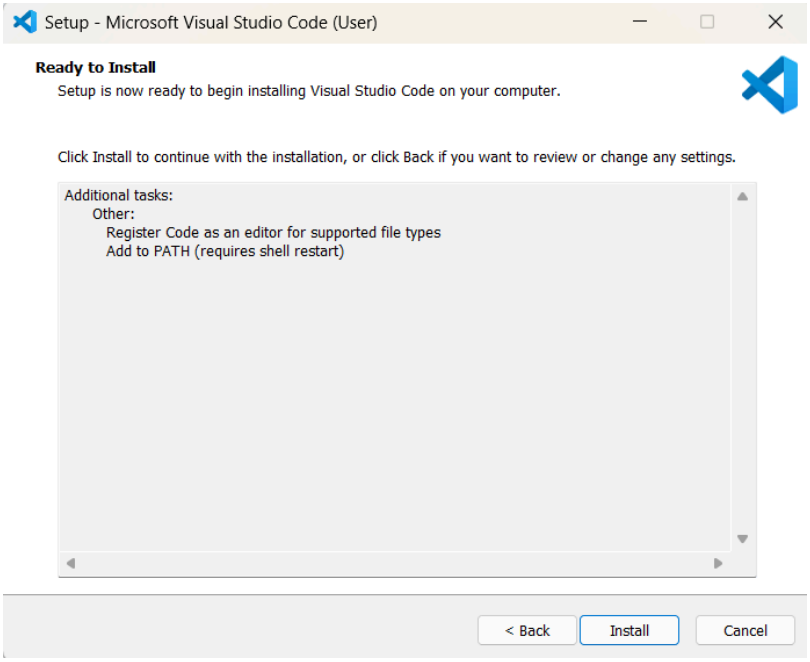
ภาพที่ ก.2 หน้าต่าง License Agreement

2. เลือก I accept the agreement แล้วกดปุ่ม Next > ระบบจะแสดงดังรูปที่ ก.3



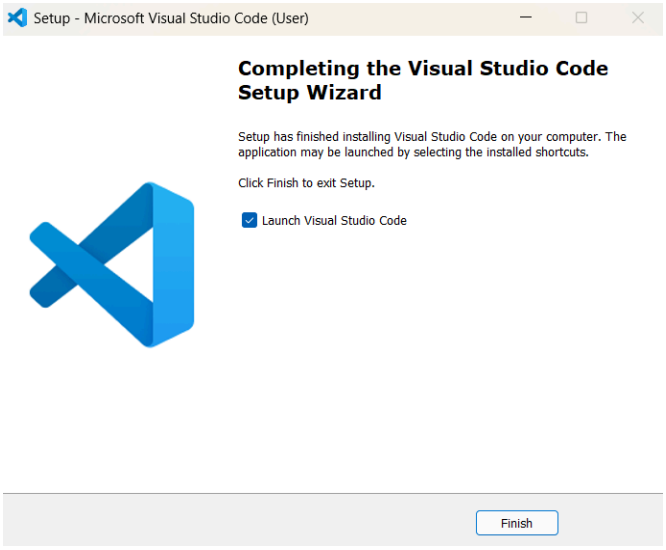
ภาพที่ ก.3 หน้าต่าง Select Additional Tasks

3. เลือก Add to PATH (requires shell restart) แล้วกด Next > ระบบจะแสดงหน้าต่างการ Install ดังรูปภาพที่ ก.4



ภาพที่ ก.4 หน้าต่างReady To Install

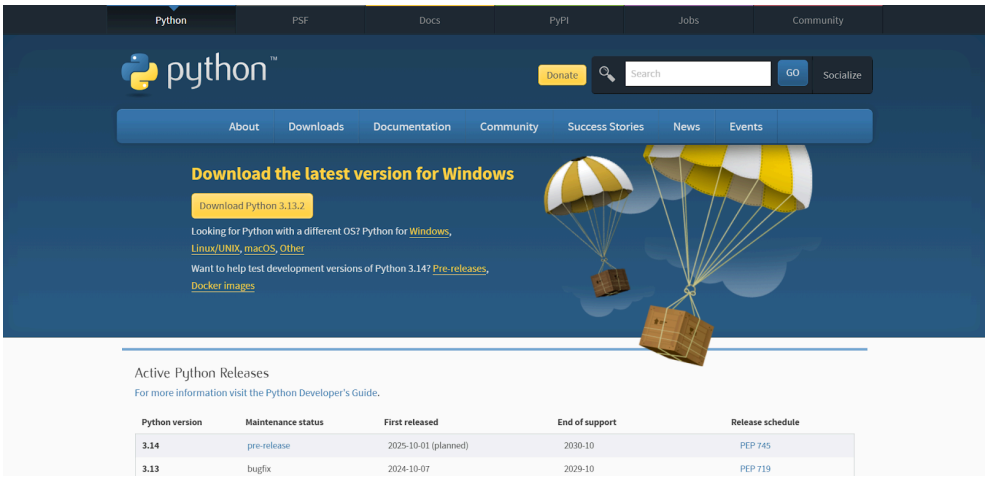
4. กด Install ในรูปภาพที่ ก.4 แล้วรอระบบติดตั้ง เมื่อเสร็จสิ้นระบบจะแสดงหน้าต่างดังภาพที่ ก.5



ภาพที่ ก.5 หน้าต่างการติดตั้งเสร็จสิ้น

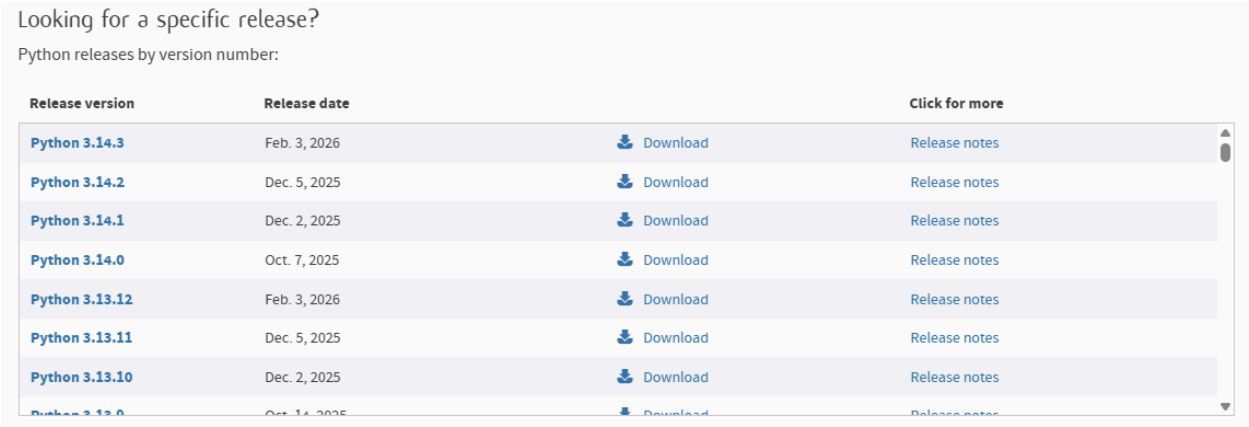
ก.2 ขั้นตอนการติดตั้ง Python

1. สามารถดาวน์โหลด Python ได้ที่ https://www.python.org/downloads/ ดังแสดงในภาพที่ ก.6



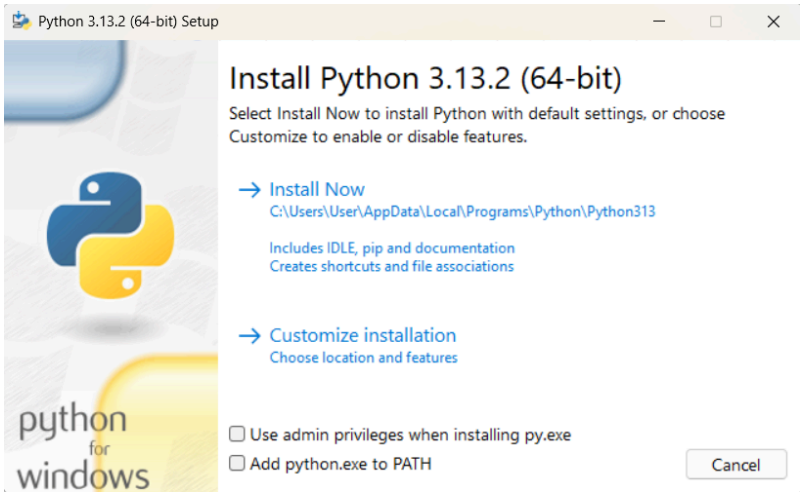
ภาพที่ ก.6 หน้าเว็บดาวน์โหลด Python

2. เลือกดาวน์โหลดเวอร์ชั้น 3.10 ขึ้นไป หลังจากกดดาวน์โหลดเลื่อนลงมาล่างสุดจะเป็นดังภาพที่ ก.7



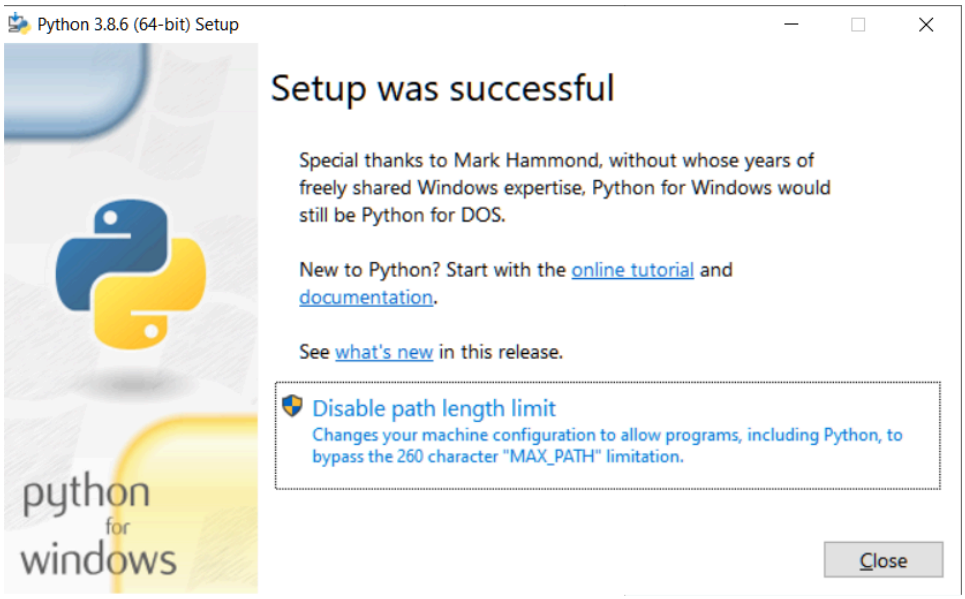
ภาพที่ ก.7 ภาพตัวเลือกที่ต้องการดาวน์โหลด

3. หลังจากกดดาวน์โหลดและทำการติดตั้ง ให้ติ้กถูกที่ Add python.exe to PATHจะแสดงดังภาพที่ ก.8



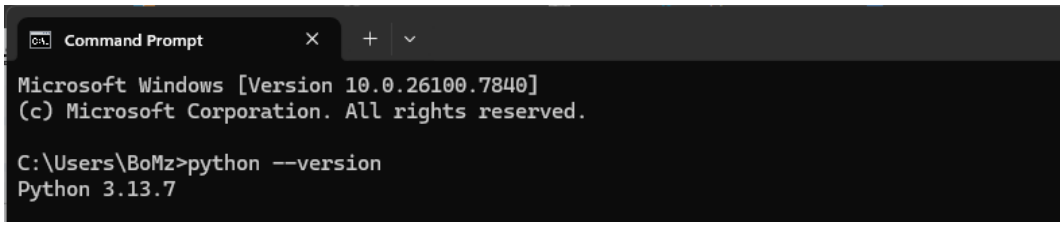
ภาพที่ ก.8 ภาพเมื่อทำการติดตั้ง Python

4. หลังจากทำการ Install Now จะแสดงดังภาพที่ ก.9



ภาพที่ ก.9 ภาพ ติดตั้งpython สำเร็จ

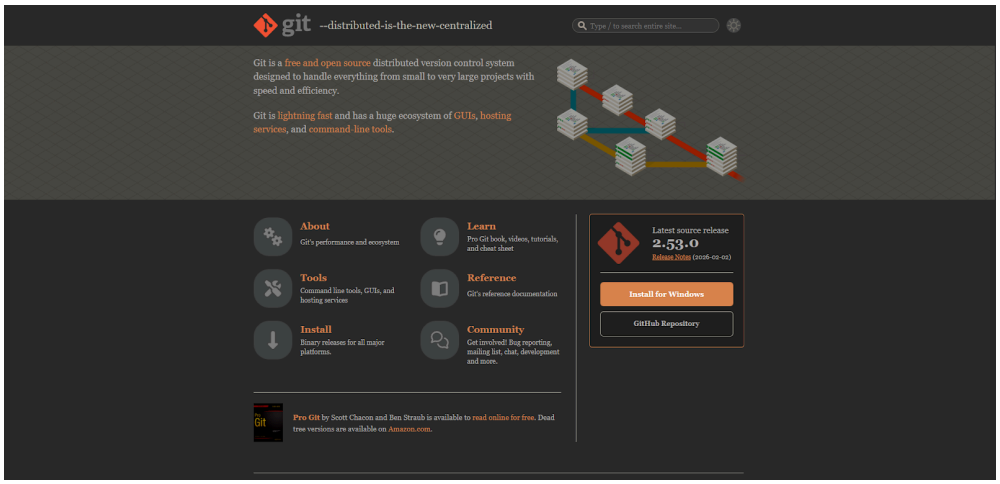
5. ตรวจสอบการติดตั้งโดยเปิด Command Prompt แล้วพิมพ์คำสั่ง python --version จะแสดงดังภาพที่ ก.10



ภาพที่ ก.10 ตรวจสอบการติดตั้ง

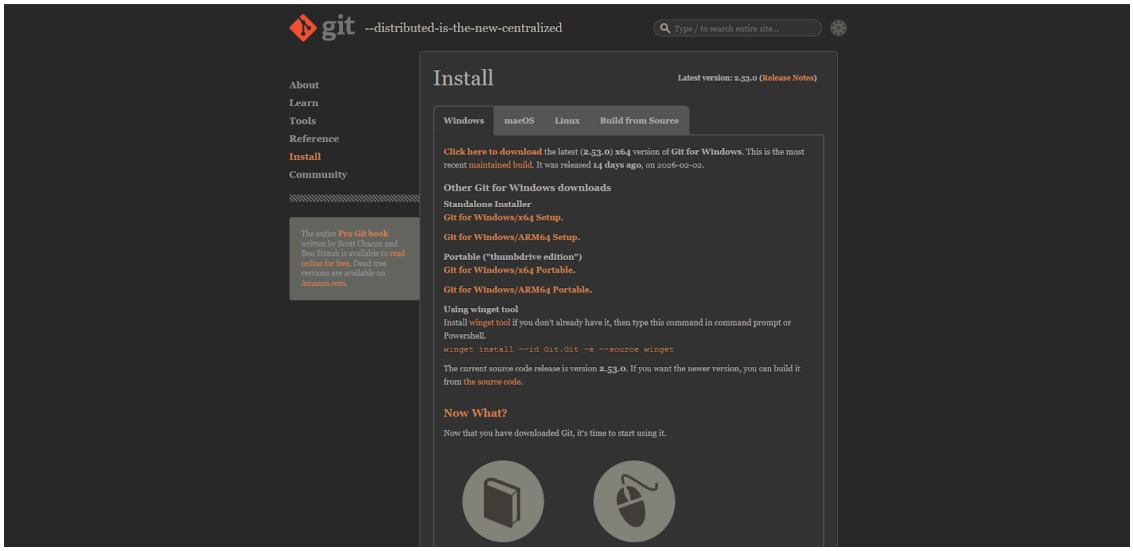
ก.3 ขั้นตอนการติดตั้ง Git

1. สามารถดาวน์โหลด Git ได้ที่ https://git-scm.com/ จะแสดงดังภาพที่ ก.11



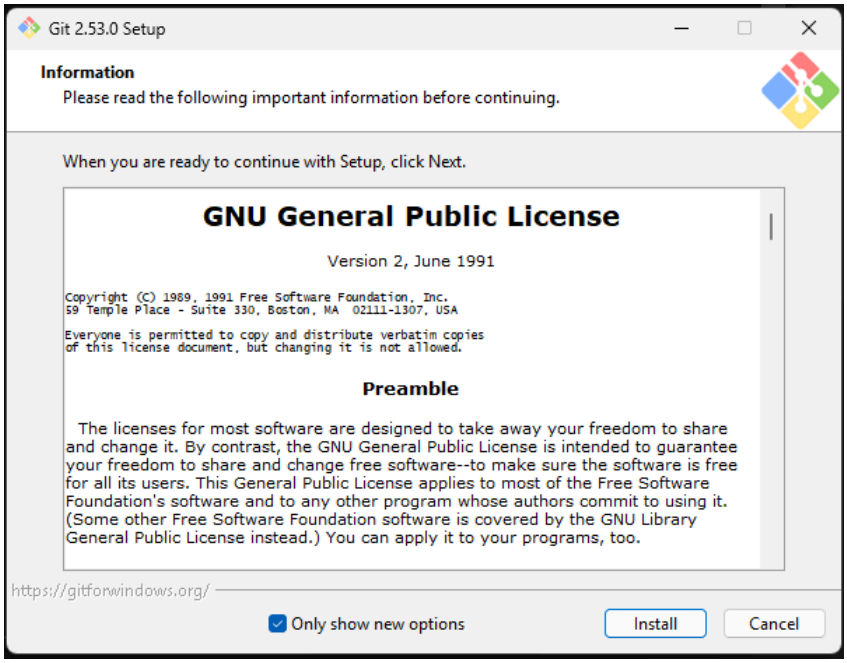
ภาพที่ ก.11 หน้าเว็บดาวน์โหลด Git

2. ดาวน์โหลด Git for Windows/x64 Setup จะแสดงดังภาพที่ ก.12



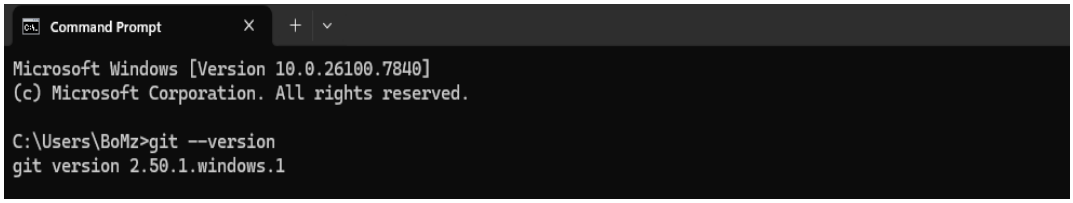
ภาพที่ ก.12 ตัวเลือกการดาวน์โหลด

3. ทำการติดตั้งตามขั้นตอนมาตรฐาน (Next จนจบ) จะแสดงดังภาพที่ ก.13



ภาพที่ ก.13 ภาพการติดตั้ง

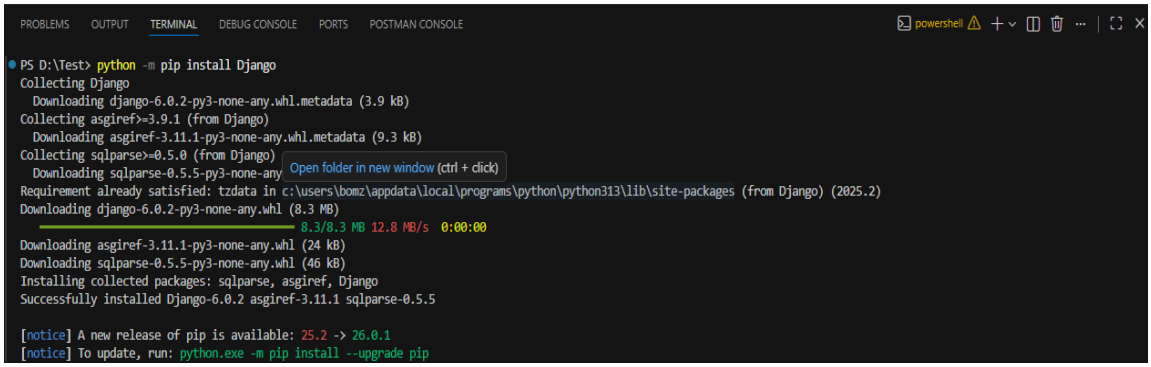
4. ตรวจสอบการติดตั้งโดยเปิด Command Prompt แล้วพิมพ์คำสั่ง git --version จะแสดงดังภาพที่ ก.14



ภาพที่ ก.14 ตรวจสอบการติดตั้ง

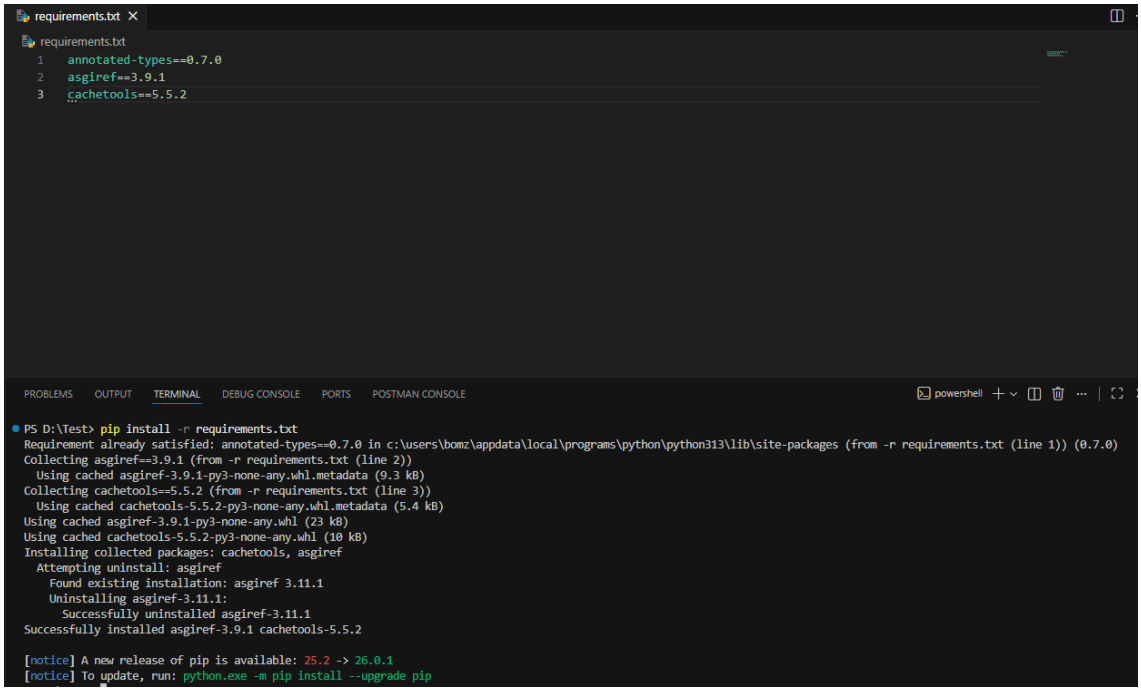
ก.4 ขั้นตอนการติดตั้ง Django และ Library ที่เกี่ยวข้อง

1. เปิด Terminal หรือ Command Prompt ในโฟลเดอร์โปรเจกต์ แล้วพิมพ์คำสั่ง python -m pip install Django จะแสดงดังภาพที่ ก.15



ภาพที่ ก.14 การติดตั้ง Django

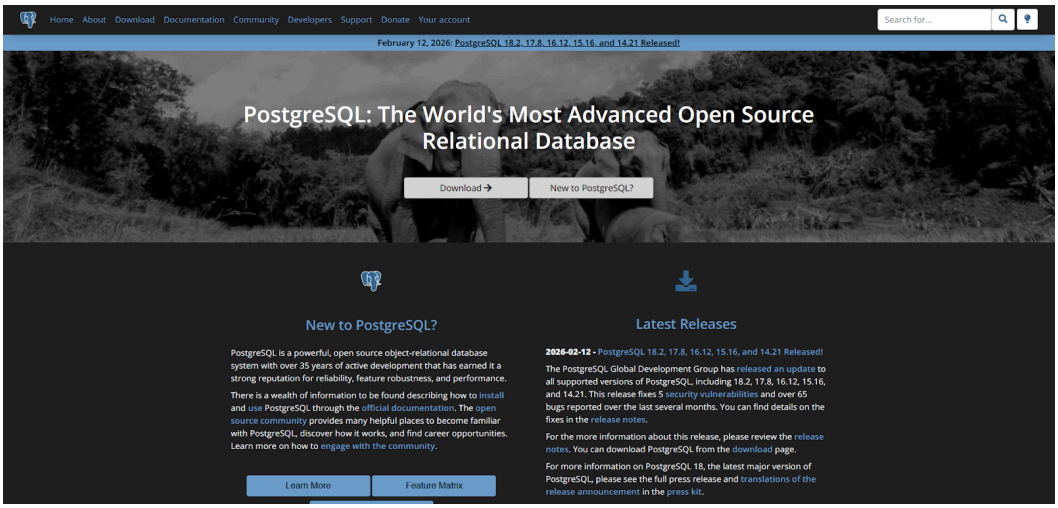
2. ติดตั้ง Library ที่จำเป็นทั้งหมดด้วยคำสั่ง pip install -r requirements.txt จะแสดงดังภาพที่ ก.15



ภาพที่ ก.14 การติดตั้ง Library

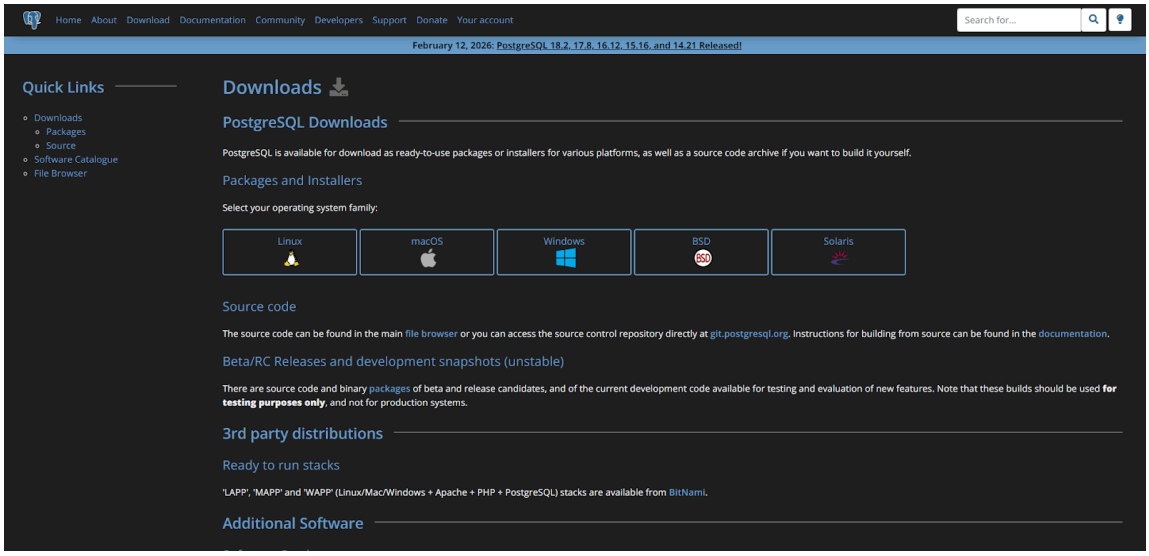
ก.5 ขั้นตอนการติดตั้ง PostgreSQL

1. สามารถดาวน์โหลด PostgreSQL ได้ที่ https://www.postgresql.org/ จะแสดงดังภาพที่ ก.16



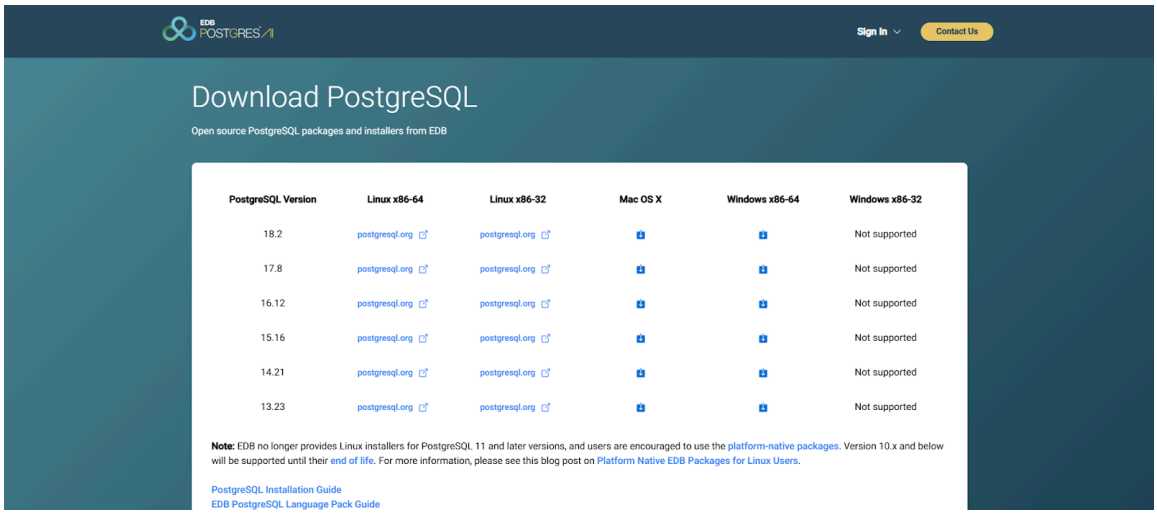
ภาพที่ ก.16 หน้าเว็บดาวน์โหลด PostgreSQL

2. ดาวน์โหลด PostgreSQL สำหรับ Windows จะแสดงดังภาพที่ ก.17



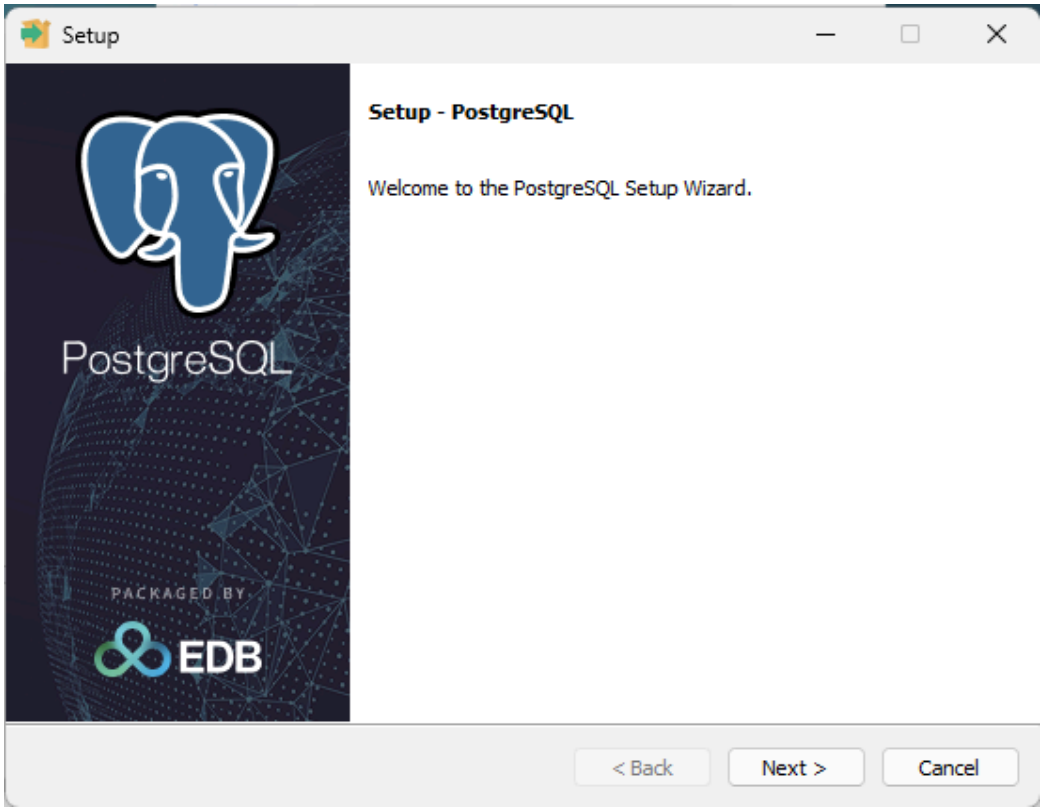
ภาพที่ ก.17 ตัวเลือกการดาวน์โหลด

3. เลือกเวอร์ชั่นดาวน์โหลด จะแสดงดังภาพที่ ก.18



ภาพที่ ก.18 ตัวเลือกเวอร์ชั่นดาวน์โหลด

4. ทำการติดตั้งตามขั้นตอนมาตรฐาน (Next จนจบ) จะแสดงดังภาพที่ ก.19



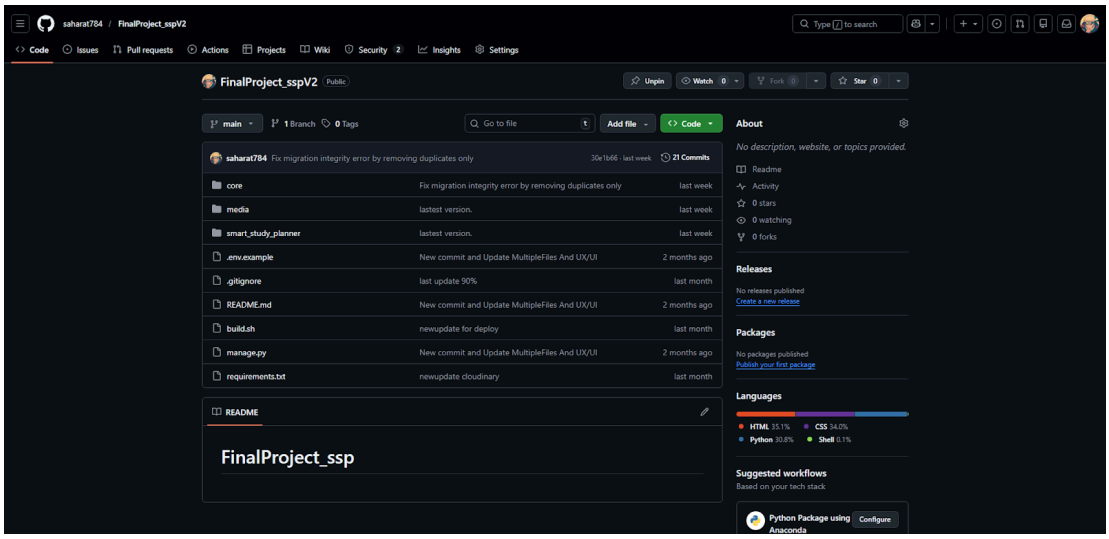
ภาพที่ ก.19 ภาพการติดตั้ง

ภาคผนวก ข
คู่มือการติดตั้งระบบ

คู่มือการติดตั้งระบบ ในการติดตั้งเพื่อใช้งานระบบจัดการลำดับเข้าสอบสัมภาษณ์แบบออนไลน์ สามารถทำได้โดยมีขั้นตอนดังนี้

ข.1 ส่วนของการดาวน์โหลดซอร์สโค้ด

1. สามารถ Clone Project ได้ที่ https://github.com/saharat784/FinalProject_sspV2.git ดังแสดงในภาพที่ ข.1



ภาพที่ ข.1 หน้าเว็บ Repository ของโปรเจ็ค

2. เปิดโปรแกรม Terminal หรือ Command Prompt ไปที่โฟลเดอร์ที่ต้องการติดตั้งโปรเจกต์ แล้วใช้คำสั่ง Clone จาก GitHub ดังแสดงในตารางที่ ข.1

Bash

```
git clone https://github.com/saharat784/FinalProject_sspV2.git
```

ตารางที่ ข.1 คำสั่งติดตั้ง โปรเจค

3. เข้าสู่โฟลเดอร์โปรเจกต์ ดังแสดงในตารางที่ ข.2

```Bash
cd FinalProject_sspV2
```

ตารางที่ ข.2 คำสั่งเข้าสู่โฟลเดอร์โปรเจกต์

ข.2 ส่วนของการสร้างและเปิดใช้งาน Virtual Environment

1. สร้าง Virtual Environment ดังแสดงในตารางที่ ข.3

```Bash
python -m venv venv
```

ตารางที่ ข.3 คำสั่งสร้าง Virtual Environment

2. เปิดใช้งาน Virtual Environment ดังแสดงในตารางที่ ข.4

```Bash
venv\Scripts\activate
```

ตารางที่ ข.4 คำสั่งเปิดใช้งาน Virtual Environment

ข.3 ส่วนของการติดตั้งไลบรารีที่จำเป็น

1. ติดตั้งไลบรารีที่จำเป็น ดังแสดงในตารางที่ ข.5

```Bash
pip install -r requirements.txt
```

ตารางที่ ข.5 คำสั่งติดตั้งไลบรารี

ข.4 การตั้งค่า Environment Variables

1. สร้างไฟล์ใหม่ชื่อ .env ในโฟลเดอร์ระดับเดียวกับ manage.py เปิดไฟล์ .env ด้วย Text Editor และใส่ข้อมูล ดังแสดงในตารางที่ ข.6

```
Bash

# Google Gemini API Config
GEMINI_API_KEY=ใส่รหัส_API_KEY_ของคุณ

# Google Calendar OAuth Config
GOOGLE_CLIENT_ID=ใส่รหัส_Client_ID_ของคุณ
GOOGLE_CLIENT_SECRET=ใส่รหัส_Client_Secret_ของคุณ
GOOGLE_REDIRECT_URI=http://127.0.0.1:8000/oauth2callback

# Django Secret Key (Optional for production)
SECRET_KEY=ใส่รหัส_Secret_Key_ของ_Django
DEBUG=True
```

ตารางที่ ข.6 คำสั่งตั้งค่า Environment Variables

ข.5 การตั้งค่าฐานข้อมูล

1. สร้างไฟล์ Migration ดังแสดงในตารางที่ ข.7

```
Bash

python manage.py makemigrations
```

ตารางที่ ข.7 คำสั่งสร้างไฟล์ Migration

2. อัปเดตฐานข้อมูล ดังแสดงในตารางที่ ข.8

```
Bash

python manage.py migrate
```

ตารางที่ ข.8 คำสั่งอัปเดตฐานข้อมูล

ข.6 การรันเซิร์ฟเวอร์

1. เริ่มการทำงานของเซิร์ฟเวอร์ด้วยคำสั่ง ดังแสดงในตารางที่ ข.8

```Bash
python manage.py runserver
```

ตารางที่ ข.9 คำสั่งรันเซิร์ฟเวอร์

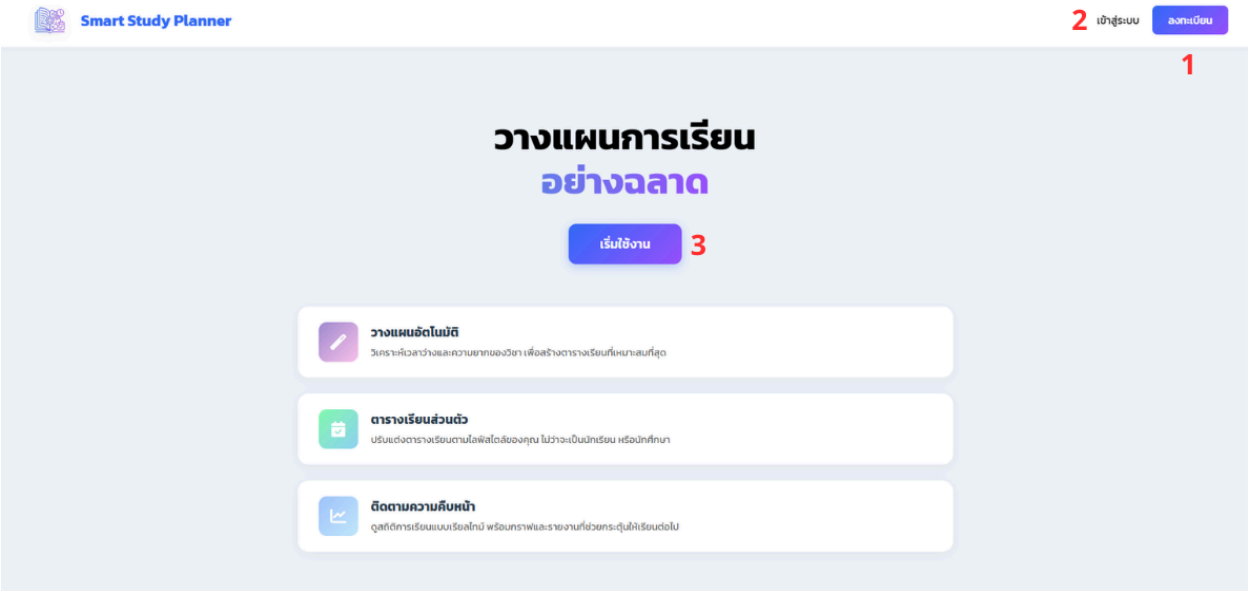
ภาคผนวก ค
คู่มือการใช้งานของระบบ

คู่มือการใช้งานทั้งหมดของระบบจัดตารางอ่านหนังสืออัตโนมัติ มีขั้นตอนการใช้งาน ดังต่อไปนี้

ค.1 ส่วนของผู้ใช้งานทั่วไป

ค.1.1 ส่วนของหน้าแรก

เมื่อผู้ใช้งานเข้าเว็บไซร์ ระบบจะแสดงผลดังภาพที่ ค.1.1



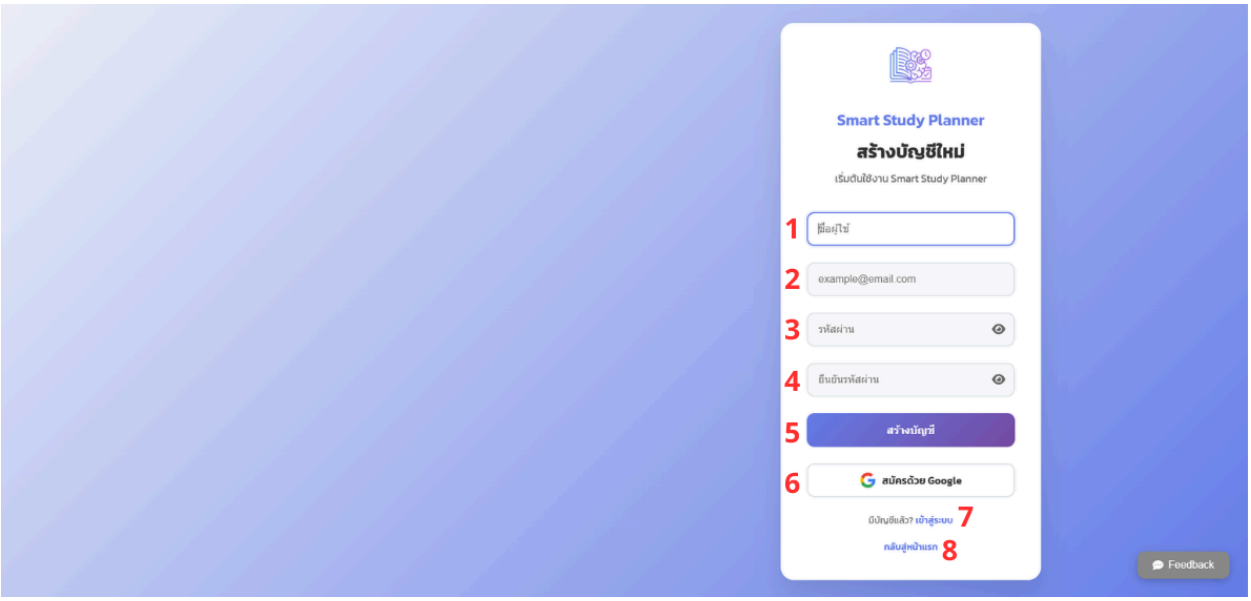
ภาพที่ ค.1.1 ภาพหน้า

จากภาพที่ ค.1.1 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ปุ่มสำหรับการกดไปหน้าลงทะเบียนหรือสร้างบัญชี
หมายเลขที่ 2 ปุ่มสำหรับการกดไปหน้าเข้าสู่ระบบสำหรับผู้ใช้งานที่เคยทำการสมัครสมาชิกแล้ว
หมายเลขที่ 3 ปุ่มสำหรับการกดไปหน้าเข้าสู่ระบบ

ค.1.2 ส่วนของหน้าสมัครสมาชิก

เมื่อผู้ใช้งานกดมาที่ปุ่มลงทะเบียน ระบบจะแสดงผลดังภาพที่ ค.1.2



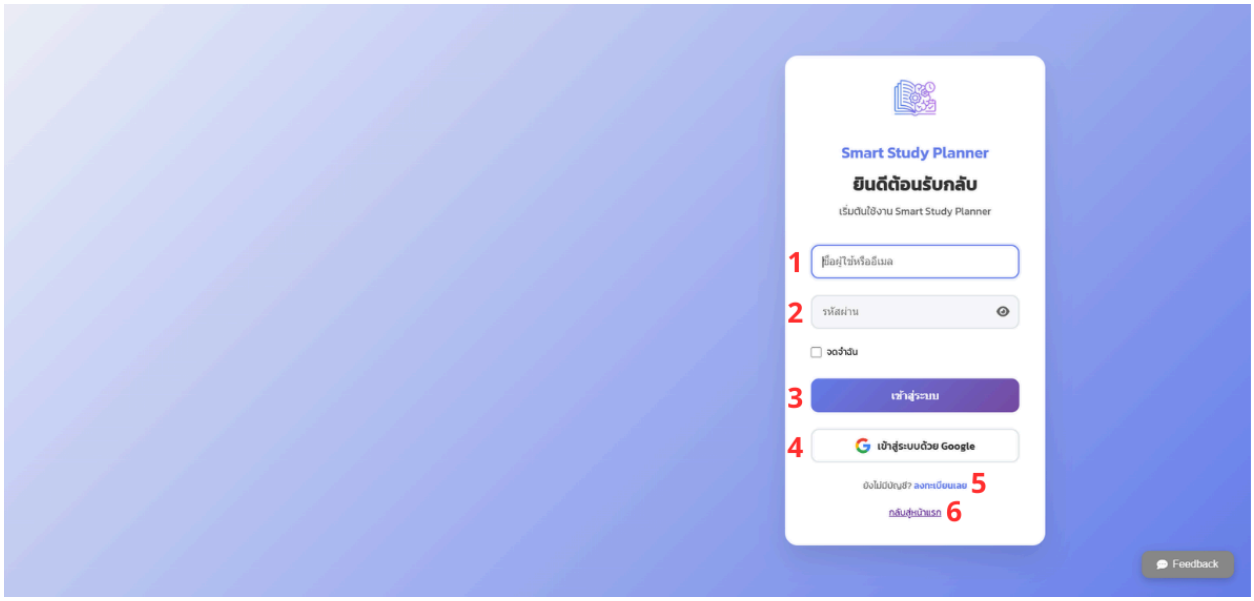
ภาพที่ ค.1.2 ภาพหน้าสมัครสมาชิก

จากภาพที่ ค.1.2 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ช่องสำหรับการกรอกชื่อที่ผู้ใช้งาน
หมายเลขที่ 2 ช่องสำหรับการกรอกอีเมลของผู้ใช้งาน
หมายเลขที่ 3 ช่องสำหรับการกรอกรหัสผ่านที่ผู้ใช้งาน
หมายเลขที่ 4 ช่องสำหรับการรอกรหัสผ่านอีกรอบเพื่อทำการยืนยันรหัสผ่าน
หมายเลขที่ 5 ปุ่มสำหรับการกดสร้างบัญชี
หมายเลขที่ 6 ปุ่มสำหรับการกดสร้างบัญชีด้วย Google
หมายเลขที่ 7 ปุ่มสำหรับการกดไปที่หน้าลงชื่อเข้าสู่ระบบสำหรับผู้ใช้งานที่เคยทำการสมัครสมาชิกแล้ว
หมายเลขที่ 8 ปุ่มสำหรับการกดไปหน้า

ค.1.3 ส่วนของหน้าเข้าสู่ระบบ

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ครั้งแรก ระบบจะแสดงผลดังภาพที่ ค.1.3



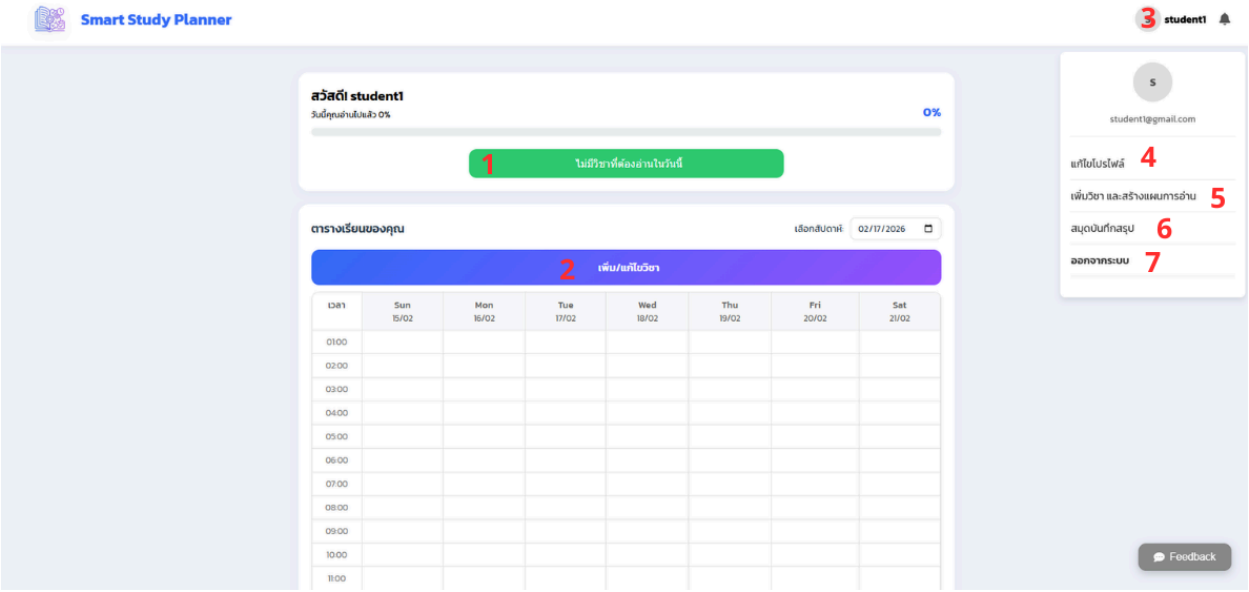
ภาพที่ ค.1.3 ภาพหน้าเข้าสู่ระบบ

จากภาพที่ ค.1.3 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ช่องสำหรับการกรอกชื่อที่ผู้ใช้งานหรืออีเมล
หมายเลขที่ 2 ช่องสำหรับการกรอกรหัสผ่านที่ผู้ใช้งาน
หมายเลขที่ 3 ปุ่มสำหรับการกดเข้าสู่ระบบ
หมายเลขที่ 4 ปุ่มสำหรับการกดเข้าสู่ระบบด้วย Google
หมายเลขที่ 5 ปุ่มสำหรับการกดไปที่ลงทะเบียนหรือสร้างบัญชีสำหรับผู้ใช้งานที่ยังไม่เคยทำการสมัครสมาชิก
หมายเลขที่ 6 ปุ่มสำหรับการกดไปหน้าแรก

ค.1.4 ส่วนของหน้า Home page

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.4



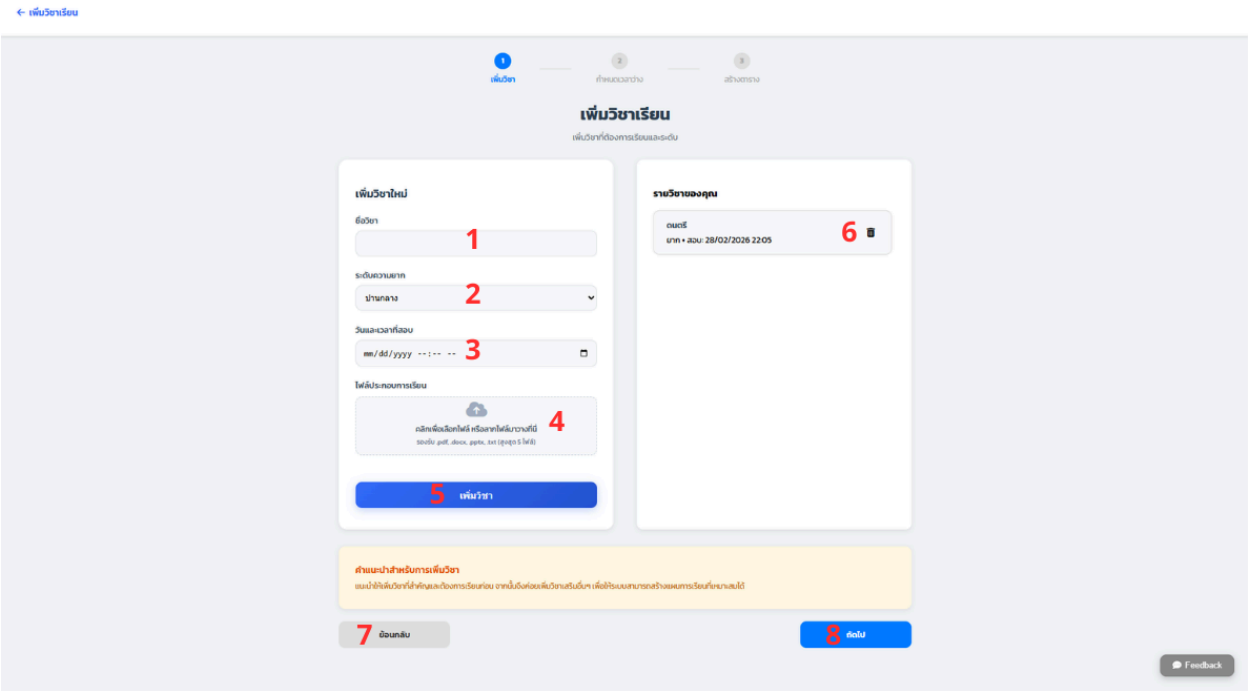
ภาพที่ ค.1.4 ภาพหน้า Home page

จากภาพที่ ค.1.4 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ปุ่มสำหรับการกดไปหน้าเริ่มอ่าน
หมายเลขที่ 2 ปุ่มสำหรับการกดไปหน้าเพิ่มวิชา
หมายเลขที่ 3 ปุ่มสำหรับการกดดูเมนูอื่นๆ
หมายเลขที่ 4 ปุ่มสำหรับการกดไปหน้าแก้ไขโปรไฟล์
หมายเลขที่ 5 ปุ่มสำหรับการกดไปหน้าเพิ่มวิชาและสร้างแผนการอ่าน
หมายเลขที่ 6 ปุ่มสำหรับการกดไปหน้าสมุดบันทึกสรุป
หมายเลขที่ 7 ปุ่มสำหรับการกดเพื่อออกจากระบบ

ค.1.5 ส่วนของหน้าเพิ่มวิชา

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.5



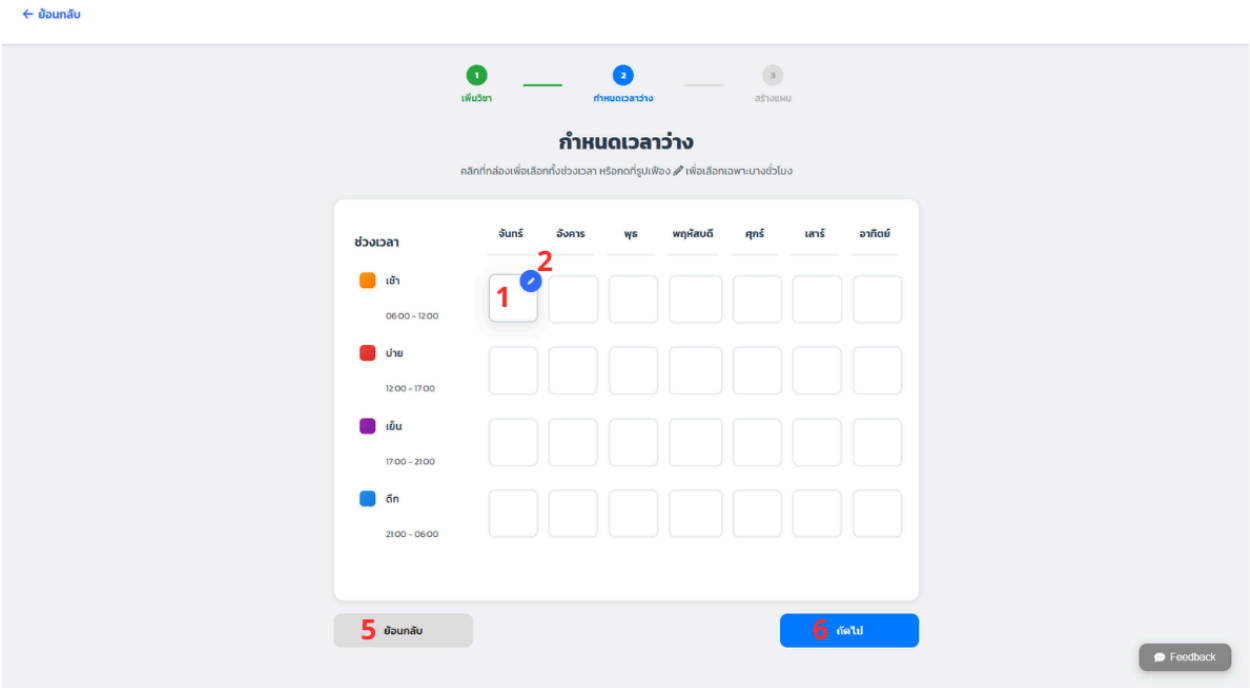
ภาพที่ ค.1.5 ภาพหน้าเพิ่มวิชา

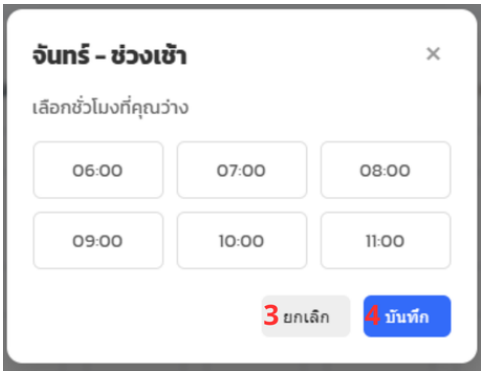
จากภาพที่ ค.1.5 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ช่องสำหรับกรอกชื่อวิชา
หมายเลขที่ 2 ช่องสำหรับเลือกระดับความอยากของชื่อวิชา
หมายเลขที่ 3 ช่องสำหรับกรอกวันและเวลาที่จะสอบ
หมายเลขที่ 4 ช่องสำหรับเพิ่มไฟล์การเรียน
หมายเลขที่ 5 ปุ่มสำหรับการกดเพิ่มวิชา
หมายเลขที่ 6 ปุ่มสำหรับการกดลบวิชา
หมายเลขที่ 7 ปุ่มสำหรับการกดไปหน้า home page
หมายเลขที่ 8 ปุ่มสำหรับการกดไปหน้ากำหนดเวลาว่าง

ค.1.6 ส่วนของหน้ากำหนดเวลาว่าง

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.6





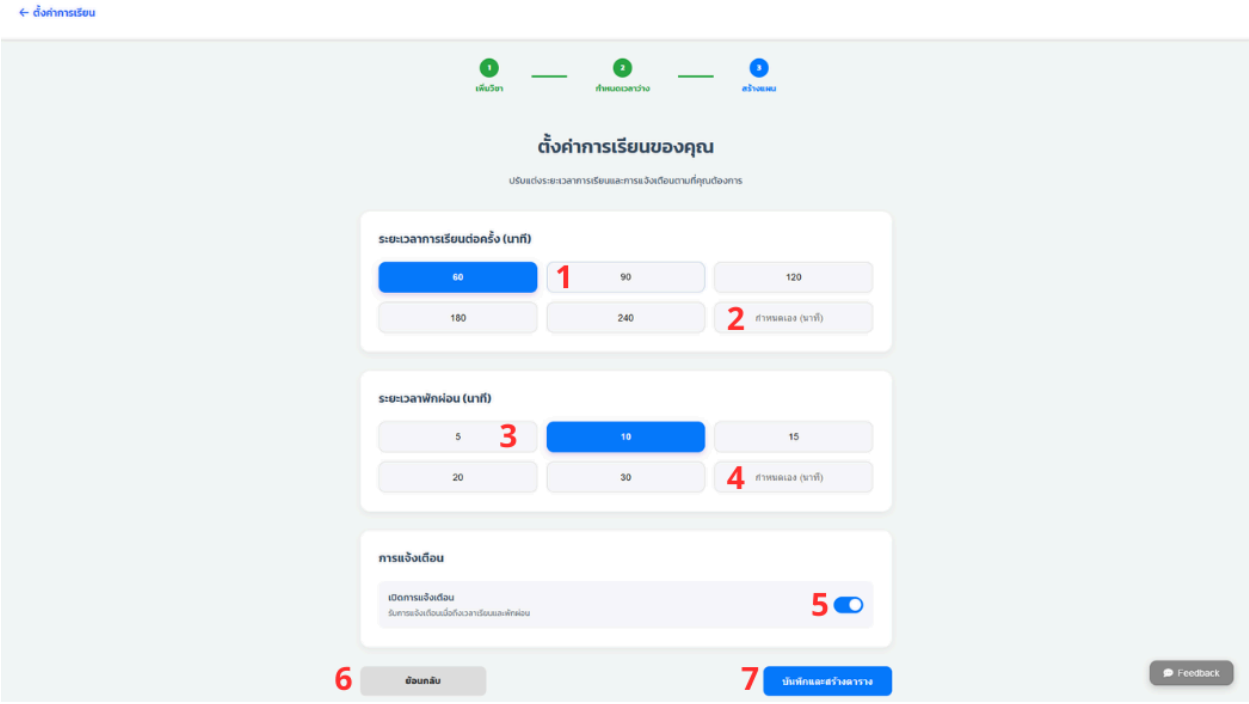
ภาพที่ ค.1.6 ภาพหน้ากำหนดเวลาว่าง

จากภาพที่ ค.1.6 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ปุ่มสำหรับการกดเพื่อกำหนดเวลาว่าง
หมายเลขที่ 2 ปุ่มสำหรับการกดเพื่อกำหนดเวลาว่างแบบรายชั่วโมง
หมายเลขที่ 3 ปุ่มสำหรับการกดเพื่อยกเลิกการบันทึกช่วงเวลาว่าง
หมายเลขที่ 4 ปุ่มสำหรับการกดเพื่อบันทึกช่วงเวลาว่าง
หมายเลขที่ 5 ปุ่มสำหรับการกดไปหน้าเพิ่มวิชา
หมายเลขที่ 6 ปุ่มสำหรับการกดไปหน้าตั้งค่าการเรียน

ค.1.7 ส่วนของหน้าตั้งค่าการเรียน

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.7



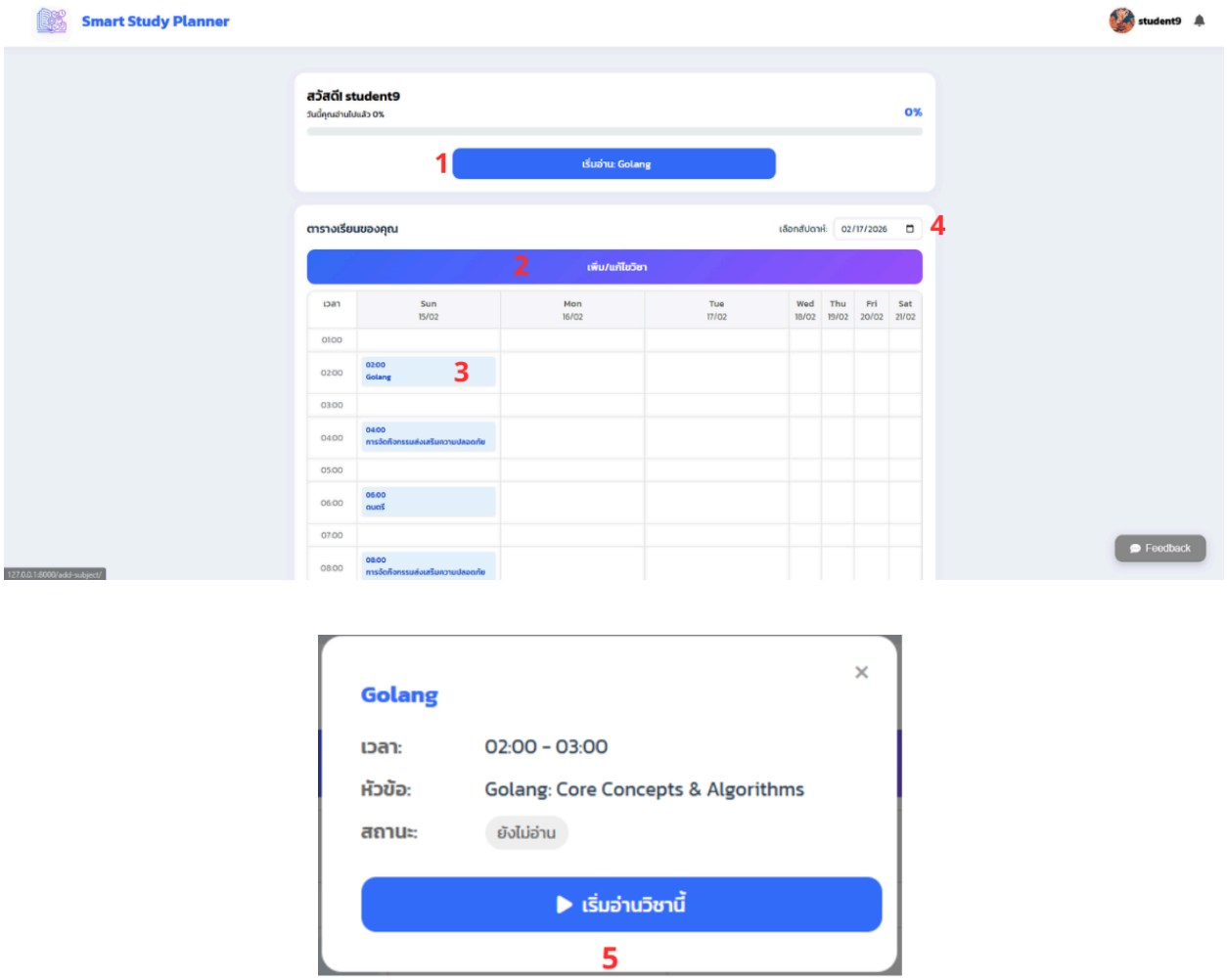
ภาพที่ ค.1.7 ภาพหน้าตั้งค่าการเรียน

จากภาพที่ ค.1.7 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ปุ่มสำหรับการกดเพื่อเลือกเวลาในการอ่าน
หมายเลขที่ 2 ปุ่มสำหรับการกดเพื่อเลือกเวลาในการอ่านแบบกำหนดเอง
หมายเลขที่ 3 ปุ่มสำหรับการกดเพื่อเลือกเวลาพักผ่อน
หมายเลขที่ 4 ปุ่มสำหรับการกดเพื่อเลือกเวลาพักผ่อนแบบกำหนดเอง
หมายเลขที่ 5 ปุ่มสำหรับการกดเปิด หรือปิดการแจ้งเตือน
หมายเลขที่ 6 ปุ่มสำหรับการกดไปหน้ากำหนดเวลาว่าง
หมายเลขที่ 7 ปุ่มสำหรับการกดสร้างตารางแผนการอ่าน

ค.1.8 ส่วนของตารางแผนการอ่าน

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.8



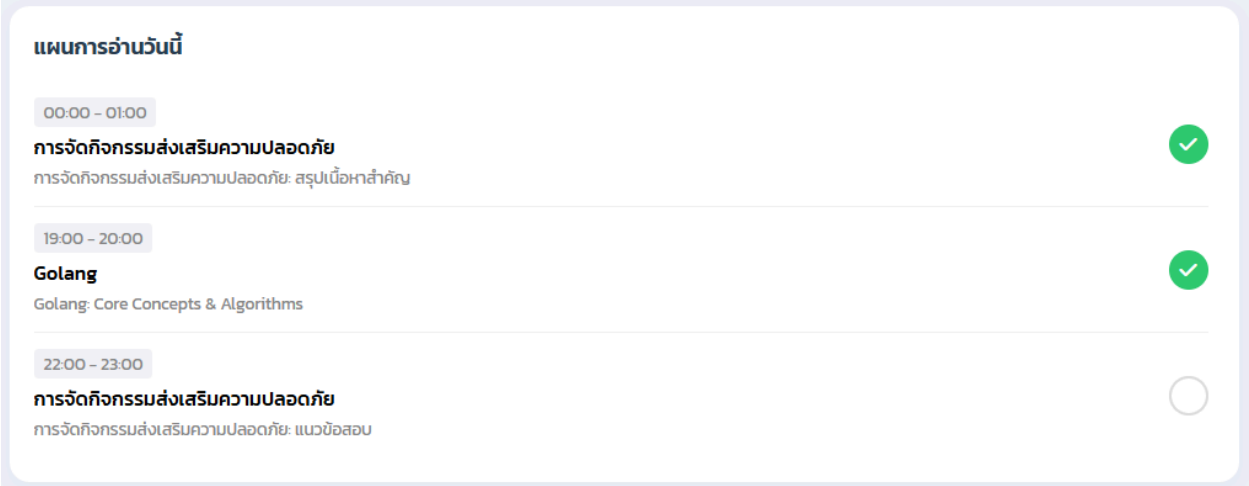
ภาพที่ ค.1.8 ภาพหน้าตารางแผนการอ่าน

จากภาพที่ ค.1.8 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ปุ่มสำหรับการกดไปหน้าเริ่มอ่าน
หมายเลขที่ 2 ปุ่มสำหรับการกดไปหน้าเพิ่มวิชา
หมายเลขที่ 3 ปุ่มสำหรับการกดดูรายละเอียดแผนการอ่าน
หมายเลขที่ 4 ปุ่มสำหรับการกดเลือกช่วงเวลาที่ต้องการดู
หมายเลขที่ 5 ปุ่มสำหรับการกดไปหน้าเริ่มอ่าน

ค.1.9 ส่วนของแผนการอ่านวันนี้

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.9



ภาพที่ ค.1.9 ภาพหน้าแผนการอ่านวันนี้

จากภาพที่ ค.1.9 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ปุ่มสำหรับการติ๊กว่าอ่านแล้ว

ค.1.10 ส่วนของสถิติภาพรวม

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.10





ภาพที่ ค.1.10 ภาพหน้าแผนการอ่านวันนี้

จากภาพที่ ค.1.10 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ปุ่มสำหรับกดดูวิชาทั้งหมด
หมายเลขที่ 2 ปุ่มสำหรับกดลบวิชา
หมายเลขที่ 2 ปุ่มสำหรับการกดเพิ่มวิชา

ค.1.11 ส่วนของการสอบที่ใกล้จะถึง

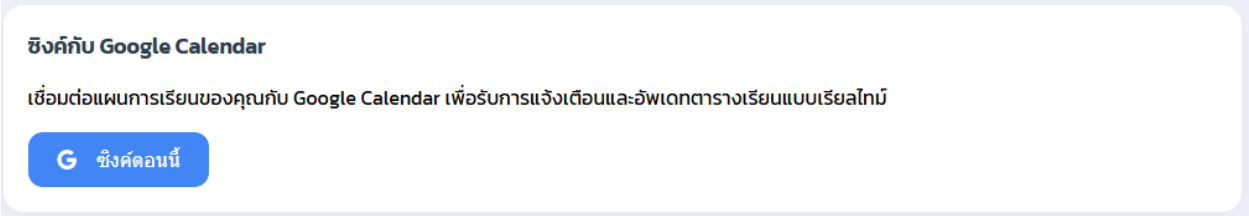
เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.11



ภาพที่ ค.1.11 ภาพหน้าการสอบที่ใกล้จะถึง

ค.1.12 ส่วนของการซิงค์กับ Google Calendar

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.12



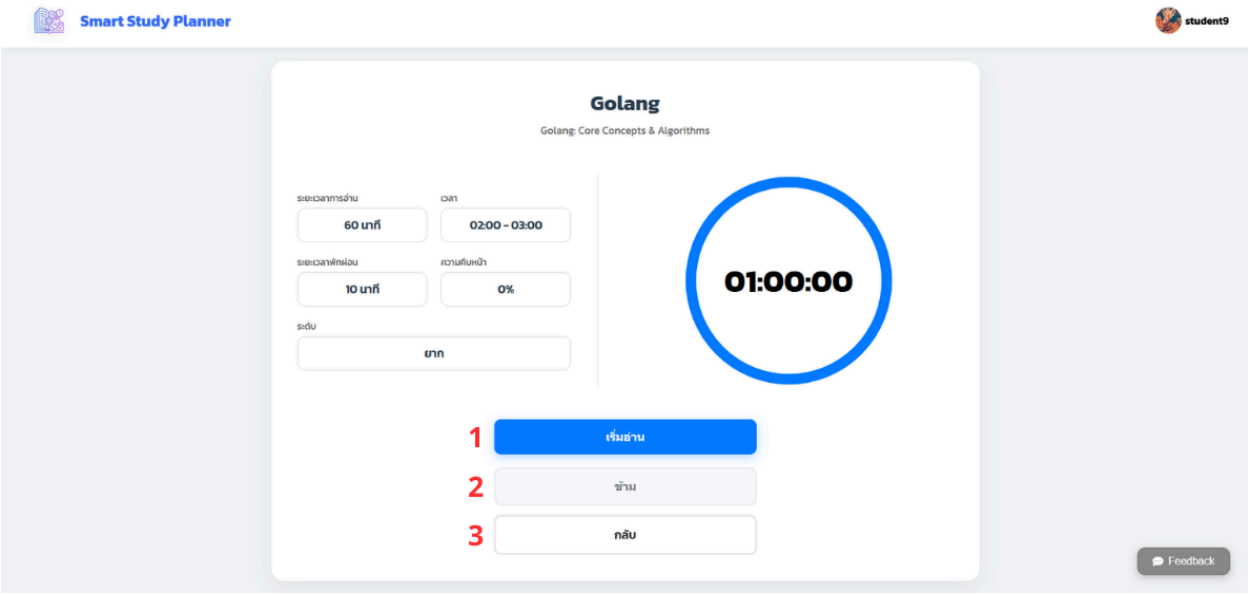
ภาพที่ ค.1.12 ภาพหน้าการซิงค์กับ Google Calendar

จากภาพที่ ค.1.12 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ปุ่มสำหรับกดเพื่อเชื่อมกับ Google Calendar

ค.1.13 ส่วนของการเริ่มอ่าน

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.13



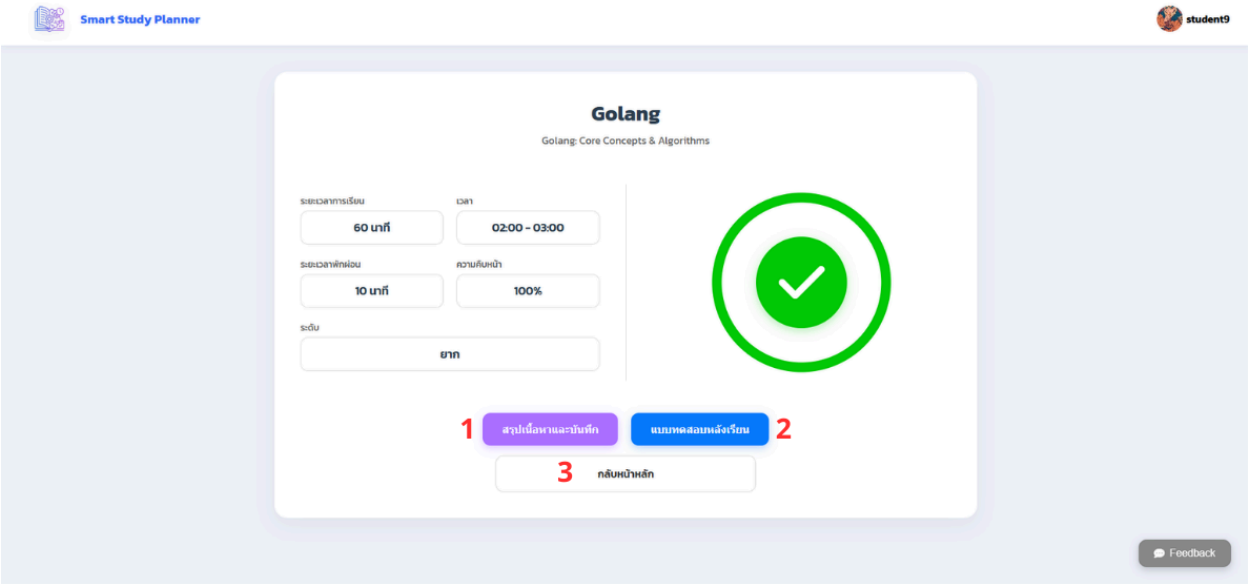
ภาพที่ ค.1.13 ภาพหน้าเริ่มอ่าน

จากภาพที่ ค.1.13 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ปุ่มสำหรับกดเริ่มจับเวลา
หมายเลขที่ 2 ปุ่มสำหรับกดข้ามการจับเวลา
หมายเลขที่ 3 ปุ่มสำหรับกดกลับไปหน้าแรก

ค.1.14 ส่วนของการสรุปเนื้อหาและการทำแบบทดสอบ

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.14



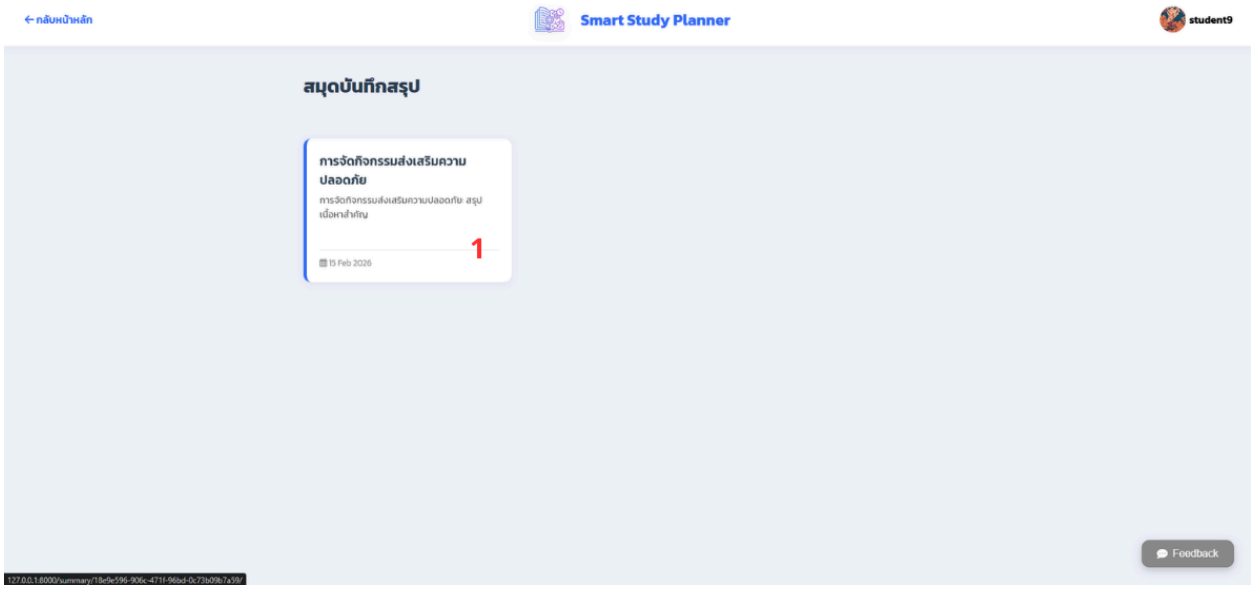
ภาพที่ ค.1.14 ภาพการสรุปเนื้อหา

จากภาพที่ ค.1.14 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ปุ่มสำหรับกดเพื่อสรุปเนื้อหา
หมายเลขที่ 2 ปุ่มสำหรับกดเพื่อทำแบบทดสอบ
หมายเลขที่ 3 ปุ่มสำหรับกดกลับไปหน้าแรก

ค.1.15 ส่วนของสมุดบันทึกสรุป

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.15



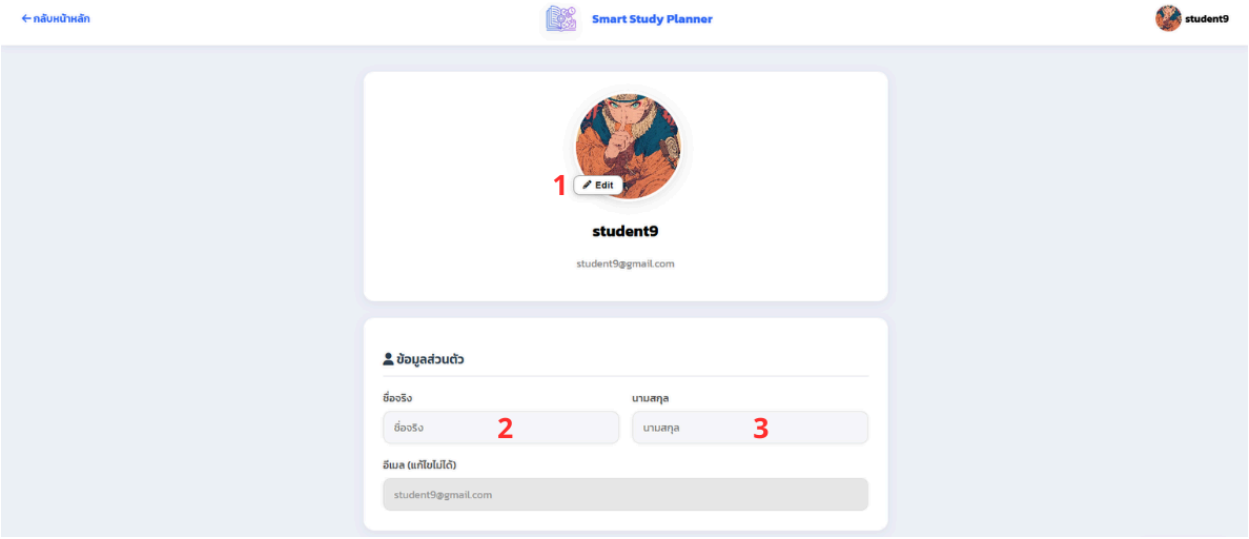
ภาพที่ ค.1.15 ภาพสมุดบันทึกสรุป

จากภาพที่ ค.1.15 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ปุ่มสำหรับกดอ่านเนื้อหาที่สรุป

ค.1.16 ส่วนของการแก้ไขโปรไฟล์

เมื่อผู้ใช้งานเข้ามาในเว็บไซต์ ระบบจะแสดงผลดังภาพที่ ค.1.16



ภาพที่ ค.1.16 ภาพการแก้ไขโปรไฟล์

จากภาพที่ ค.1.16 สามารถอธิบายการใช้งานได้ดังนี้

หมายเลขที่ 1 ปุ่มสำหรับกดเปลี่ยนโปรไฟล์
หมายเลขที่ 2 ช่องสำหรับกรอกชื่อ
หมายเลขที่ 3 ช่องสำหรับกรอกนามสกุล

ประวัติผู้พัฒนา

ชื่อ-นามสกุล : สหรัฐ เวียงแก้ว

รหัสประจำตัวนักศึกษา : 66114540784

วันเกิด : 22 กันยายน 2547

ที่อยู่ที่สามารถติดต่อได้ :

เบอร์โทรศัพท์ : 0937400770

อีเมล : saharat.wi.66@ubu.ac.th

ระดับมัธยมต้น : โรงเรียนศรีรัตนวิทยา จังหวัดศรีสะเกษ

ระดับ : วิทยาลัยเทคนิคกันทรลักษ์ จังหวัดศรีสะเกษ

ระดับอุดมศึกษา : ภาควิชาคณิตศาสตร์ สถิติ และคอมพิวเตอร์ สาขาวิทยาการข้อมูลและนวัตกรรมซอฟต์แวร์ คณะวิทยาศาสตร์ มหาวิทยาลัยอุบลราชธานี